

Sixteen Layers probed in 2 minutes

How we estimated the average activity of every neuron in a random deep network, paying for every operation

Andrei Bulzan

ARC White-Box Estimation Challenge 2026, Phase 2 · Submission number 337559 · October 2026

Contents

The story in one page 3

The problem

A network plus a question 4
What a network does to a cloud of inputs 5
How an answer is scored 6
Why not just run it many times? 7
The error is a distance 8

One neuron, then many

The mean of one neuron is an area 9
Lopsided and heavy-tailed 10
Why a neuron's sum is nearly a bell curve 11
Four numbers describe the shape 12
Neurons move together 13
Three ways two neurons are linked 14
What would have to be carried 15

Carrying the shape

New shape is made only at the kink 16
What the kink does 17
Three kinds of neuron 18
A big table from short lists 19
One bend leaves a simple pattern 20
A source and its two legs 21
The life of a source 22
How a source travels 23
What a gate reads 24
Old sources share one basis 25
One layer of the estimator 26
Young and old sources 27

Paying less

Where the FLOPs go 28
Seven multiplications instead of eight 29
How deep to go 30
Skip what is already settled 31
Iteratively reducing the FLOPs bill 32

Finding the error

Asking an oracle 33
How an error travels 34

Adding the missing physics

How one neuron's shape moves with another 35
Fourth-order births, folded into an existing product 36
A companion for what compression drops 37
Where the companion helps 38

Learning and measuring

Corrections learned from data 39
A small network reads each pass and corrects its answer 40
Measuring a one-percent change 41
Every setting sits at its knee 42

Racing the clock

Three limits, met one after another 43
The meter and the clock disagree 44
The grader's clock 45
Four tiers used for the same one model 46
Choosing the depth from the clock 47
Rehearsing slow graders 48
What we expected 49
What the grader saw 50

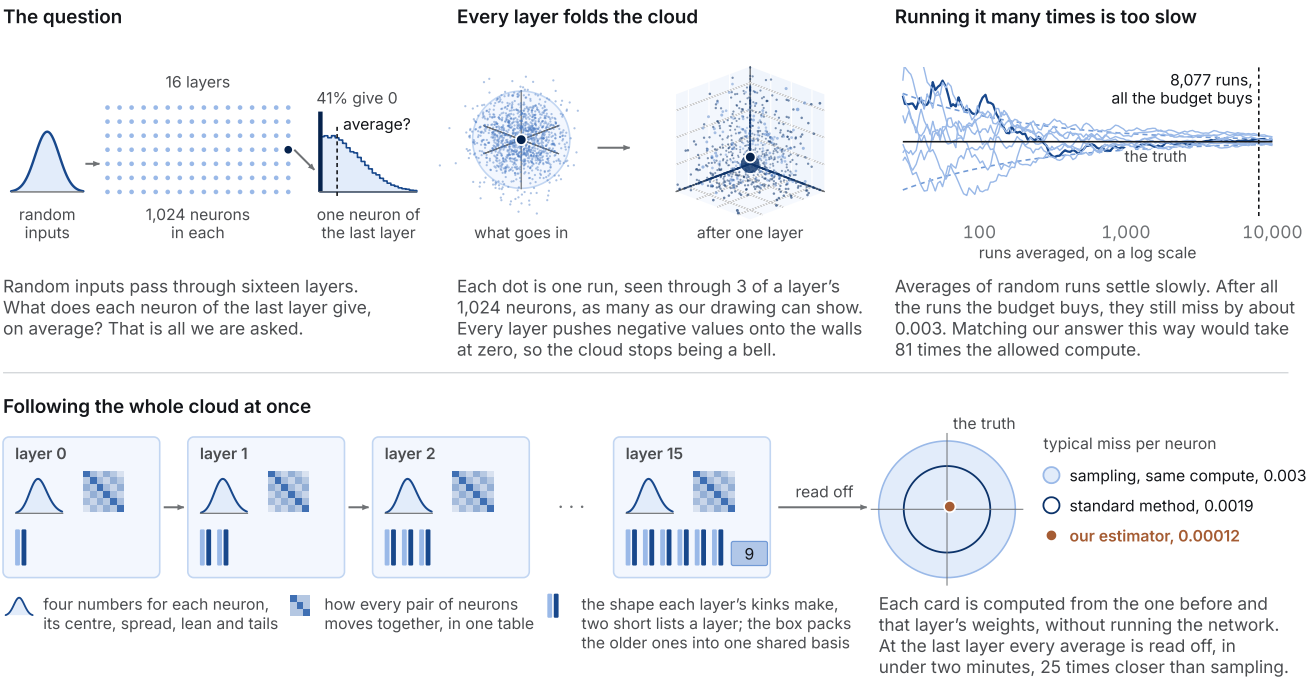
Looking back

What did not work 51
A secondary avenue with accuracy first 52
Fitting our own line to the budget 53
How the work went 54
On how we worked 55
What we built on 56
Reproducing the result 57
In Brief 58
Some ending words 59

Appendix

The effect of each change 60
Where the remaining error lies 61
The settings and their trade-offs 62
Uploads and time limits 63
Words used in this report 64

The story in one page

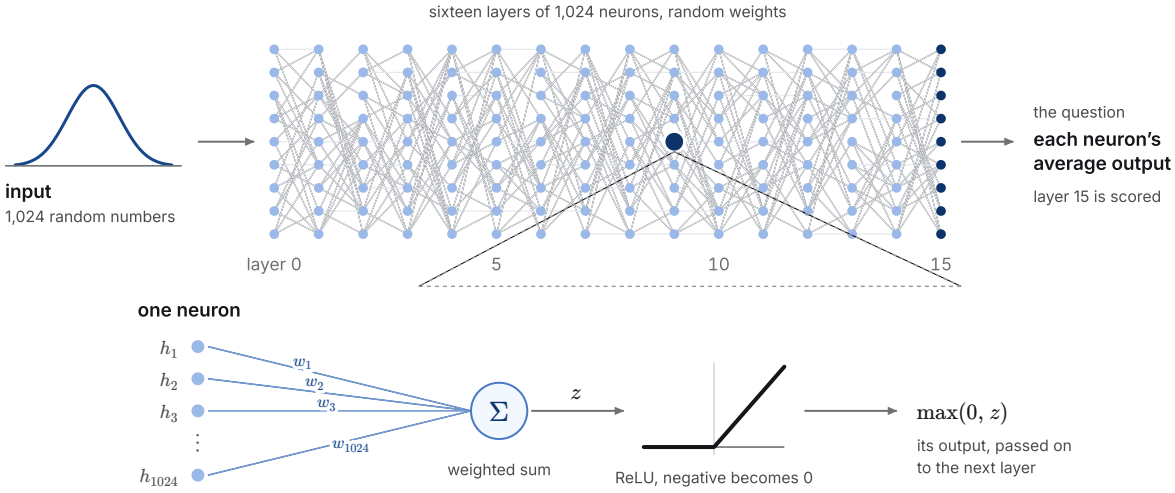


We were given random neural networks and asked for the average output of every neuron, computed from the network's weights, within a fixed budget of arithmetic and time. The figure above tells the story with real numbers, following one such network. Getting there required five large moves. We had to:

1. **Compute instead of sample.** Averaging random runs of the network would need 81 times the allowed compute to match our final error. Four numbers per neuron nearly fix its average, so we compute those instead.
2. **Carry the representation in heavily compressed form.** How far each neuron's input departs from a bell is stored where the departure arises, in a few short lists of numbers. This is Team 504aldo's method, adopted in September.
3. **Pay as little as possible for the same operations.** Strassen's fast multiplication, skipping neurons whose answer is already settled, and exact rewrites cut the compute from 735 to 294 billion operations per network; one step, on 4 October, also cut the error by a sixth.
4. **Fix the error right where it arose.** New fourth-order terms, a list of numbers that recovers part of what compression drops, and corrections learned from data cut the error by a fifth.
5. **Keep a live eye on the wall clock.** Deeper fast multiplication billed less but ran slower, and overran two minutes on the grader's slower machines until the estimator learned to set its depth, layer by layer, from the clock.

The final submission scored 1.7453×10^{-9}, an error of 1.4×10^{-8} at 271 billion operations per network, with all 100 graded networks inside every limit.

A network plus a question



Take a machine that turns 1,024 random numbers into 1,024 new ones, sixteen times over, and ask what comes out of each of its parts on average. This report is our answer, as Phase 2 of the ARC (Alignment Research Center) White-Box Estimation Challenge 2026 asks for it. "White-box" means that the estimator may read the network's weights instead of only running it.

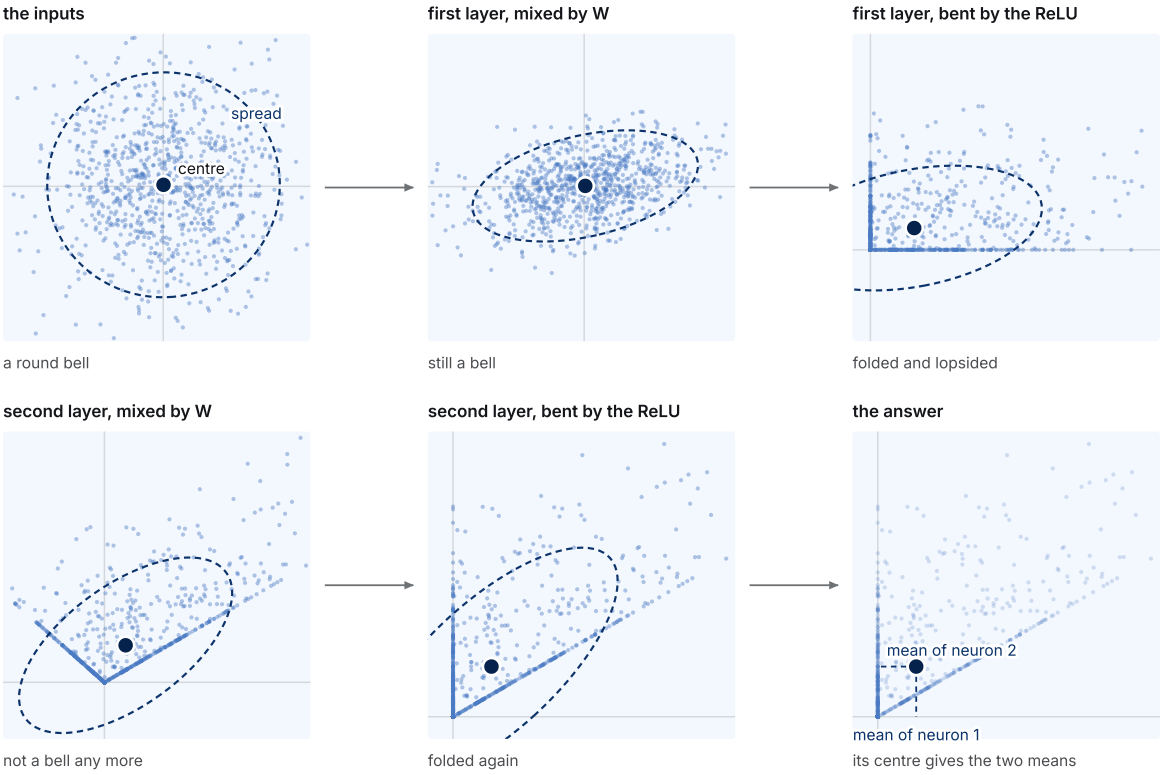
A neural network, here, is a chain of simple arithmetic. It starts from a list of 1,024 numbers, the input, and each of its 16 layers turns the list it receives into a new list of 1,024 numbers. Each number belongs to one *neuron*, and every neuron does two things. It adds up the numbers of the previous layer, each multiplied by a *weight* of its own, to get a sum z; then it applies the ReLU (rectified linear unit), which outputs z if z is positive and zero otherwise, written $\max(0, z)$. The picture enlarges one neuron, with inputs h_1 to h_{1024} and weights w_1 to w_{1024}.

The weights are fixed numbers, drawn at random once when the network is made, with a spread that keeps the numbers flowing through the network at roughly the same size (the He initialisation, after Kaiming He). The organisers give them to us as sixteen tables of 1,024 × 1,024 numbers. The input, by contrast, is drawn afresh every time the network runs, each number from the standard bell curve (the normal or Gaussian distribution, centred at 0 with spread 1), so every output is random.

The question, then, is what each neuron's output is on average over all possible inputs, its *mean activation*. There are 16 × 1,024 such means, and only the 1,024 of the last layer are scored. We are given only the weights; anything else has to be computed and paid for. The rings on the cover are the answers for one network, one ring per layer and one tick per neuron, its length the neuron's mean.

Why is this hard? For the first layer the answer is a short formula, since a weighted sum of bell-shaped inputs is itself bell-shaped. Later sums mix outputs that ReLUs have already bent and that are linked, because all respond to the same input. After sixteen layers no practical exact formula exists. Every method approximates, and the challenge is to approximate well for little compute.

What a network does to a cloud of inputs



Here is the whole problem in one picture, shrunk from 1,024 directions to two. Each dot is one run of a toy network with two neurons per layer, and many random runs make a cloud.

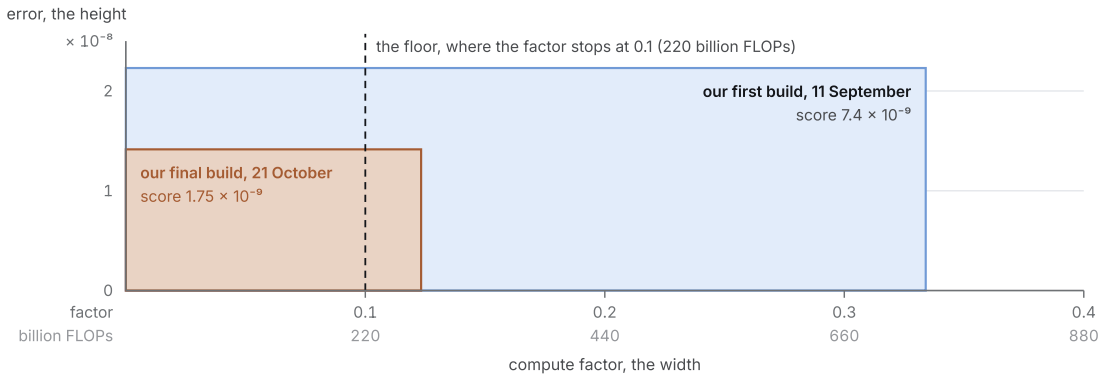
At the start the cloud is round and densest in the middle, a bell in two directions. Multiplying by the weights, $\times W$, stretches, turns and shears it, but it stays a bell, since a weighted sum of bell-shaped numbers is bell-shaped.

The ReLU then bends it. Every negative number becomes zero, so every dot left of the vertical axis is pushed onto that axis, every dot below the horizontal axis is pushed up onto it, and the dots in the lower-left quarter all land on the corner. What was a bell is now *folded*, piled up along two edges and spread out in the open quarter, *lopsided* (page 10 makes the word exact). The next layer mixes this folded cloud into something that is no longer a bell, then folds it again.

The dark dot in each panel is the centre of the cloud, the average of its dots; the dashed ellipse marks its spread. The last panel reads off the answer, since the centre's coordinates are the mean activations of the two neurons. At full size there are 1,024 of them, and they are what we are asked for. The organisers found them by running each network a billion times, drawing the cloud dot by dot. We have to find the centre without drawing the cloud.

Everything that follows is about this picture. A mix by the weights is easy to follow exactly, because the centre and spread of the cloud move by plain matrix arithmetic. The bend is the difficulty. After it, the centre of the next cloud depends on the shape of this one as well as on its centre and spread.

How an answer is scored



The organisers know the true means because they ran each network a billion times on random inputs and averaged the outputs. Our 1,024 final means are compared with theirs, each difference is squared, and the squares are averaged. This *mean squared error* is the first half of the score, and smaller is better. The final-layer means of these networks mostly lie between 0 and 2, and our final error was 1.4×10^{-8}, a typical miss of about 0.0001 per neuron (its square root).

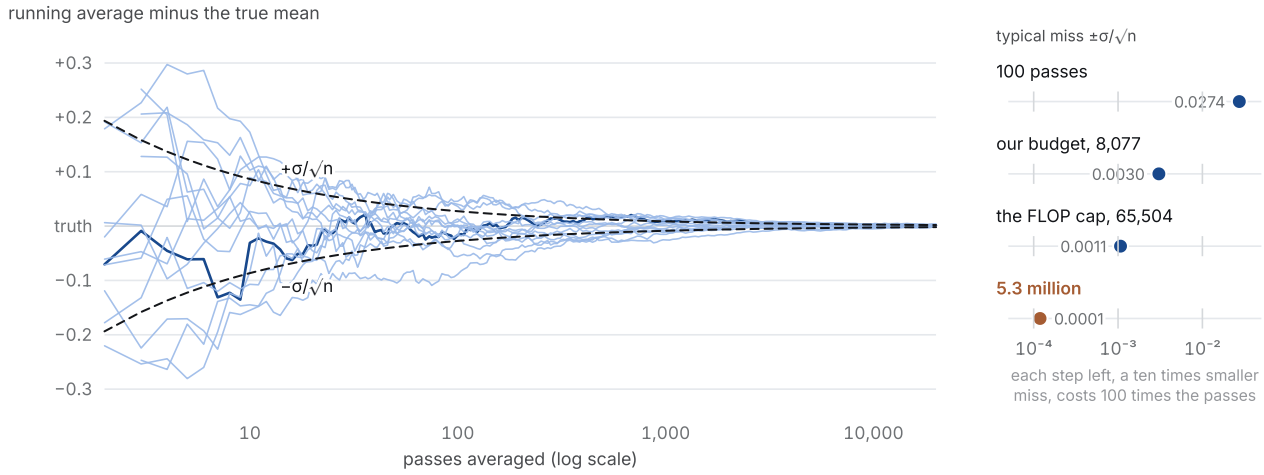
The second half is compute. Computers calculate with *floating-point* numbers, numbers with a decimal point, and every arithmetic operation on one is a *FLOP*. The organisers' library, flopscope, counts every FLOP our code performs. Operations on 64-bit numbers, with twice the digits, count double; fewer digits than the usual 32 bits earn no discount, and arithmetic outside the meter disqualifies a submission. The FLOPs per network, F, become a *compute factor*, $\max(0.1,\ F/2^{41})$, where 2^{41}, about 2.2 trillion, is the most allowed.

The score is the error times the compute factor, so it can be drawn as an area. In the picture each build is a rectangle, its height the error and its width the compute factor, also read on the lower scale in billions of FLOPs. Our first build scored 7.4×10^{-9} and the final build 1.75×10^{-9}, an area 4.3 times smaller. Narrowing a rectangle by 1% shrinks its area exactly as much as lowering it by 1%, so above the floor compute and error trade one for one; below 220 billion FLOPs the width stays at 0.1 (dashed line).

Three clocks run beside the meter, allowing 5 seconds of setup, the one-off preparation before the first network, 120 seconds per network, and 0.4 seconds per network for the program's own Python bookkeeping. A network that breaks a clock is scored as if all its answers were zero, at a compute factor of 1, and one such network would multiply our average score about ten million times. The grader runs 100 networks and the leaderboard shows the average over 50 of them. For testing, the organisers published 100 more networks with their reference answers, which we call the public networks. A smoke test also runs networks of other sizes, so the code must work for any width and depth.

The final score, an average over networks of each one's error times its own factor, was 1.7453×10^{-9}, close to the product of the averages, $1.4153 \times 10^{-8} \times 0.1233$. In the pictures, clay, the brown colour, marks our own work and results.

Why not just run it many times?



The obvious way to find the centre of the cloud is to draw it, running the network on many random inputs and averaging what each neuron outputs. This is the *Monte Carlo* method, and it is how the organisers made their reference answers. Its weakness is how slowly it improves.

The picture follows fourteen running averages for one neuron, simulated with draws of the right mean and spread, one drawn dark so that it can be followed. Each starts far from the truth, wanders, and settles. The dashed band, their typical distance from the truth, narrows only as $\sigma / \sqrt{n}$, where n is the number of runs and σ the spread of the neuron's output, so a hundred times more runs buy only ten times less distance from the truth. The score squares that distance, so a hundred times more runs buy a hundred times smaller error, and no more.

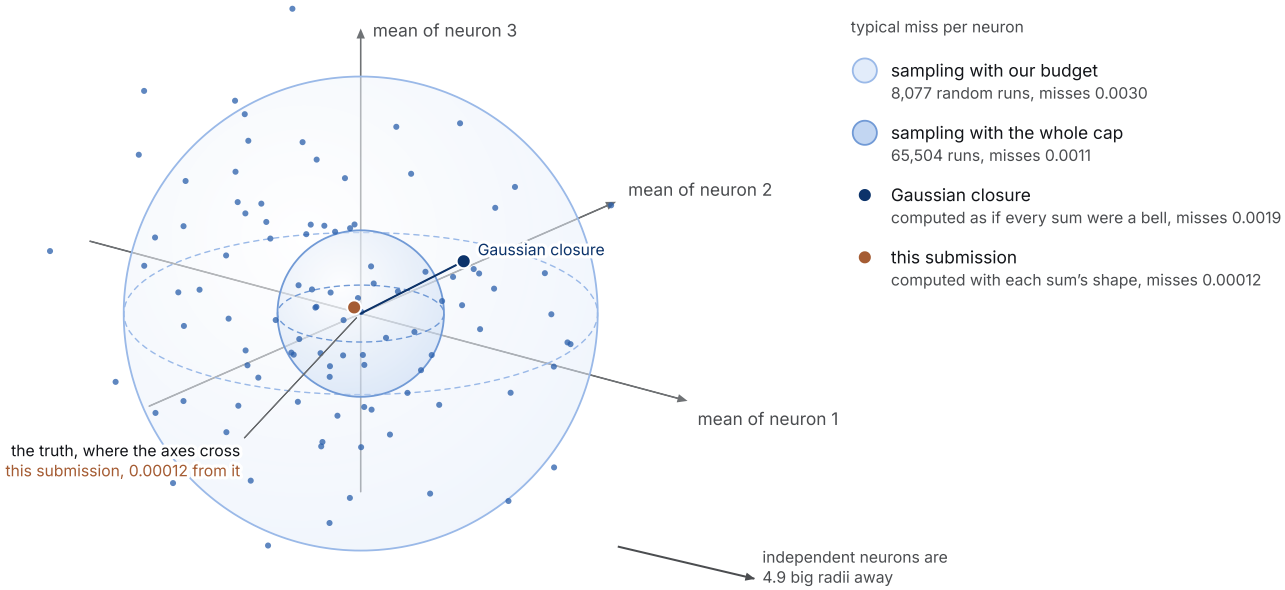
On this meter one run of the network, a *forward pass*, costs 33.6 million FLOPs. Our final budget of 271 billion buys 8,077 runs, whose average misses each final-layer mean by about 0.003, an error of 9.3×10^{-6}. The whole allowed compute buys 65,504 runs and misses by about 0.001. To miss by only 0.0001, as our estimator does, sampling would need about 5.3 million runs, 81 times the most compute allowed (right of the picture).

Cleverer sampling does not close a gap of that size. Choosing the inputs more evenly than at random made the squared error of the average 2.3 times smaller; simple linear formulas used as corrections removed only 14 to 19% of it.

Even the reference answers carry sampling noise. Averaging 10^9 runs, about 15,000 times the allowed compute, leaves an error of about 7×10^{-11} per neuron. For the networks we generated ourselves, which we call *fresh networks*, we averaged 2^{25} or 2^{27} runs, and we subtract that noise, at most about 3×10^{-9}, whenever we compare against those answers.

The means therefore have to be calculated from the weights themselves.

The error is a distance



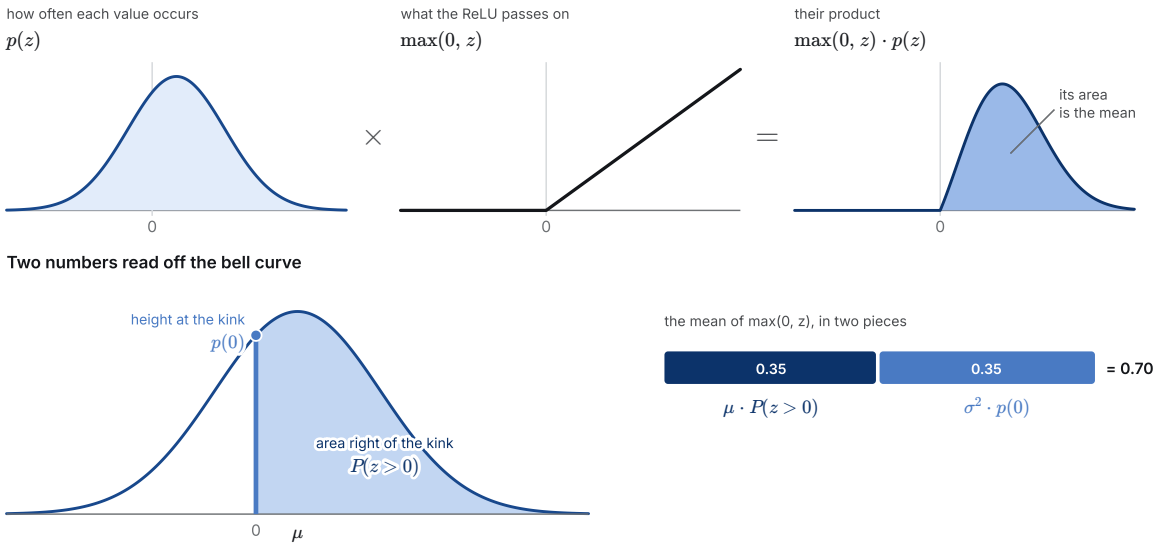
The 1,024 means of the last layer can be read as the coordinates of one point in a space with 1,024 directions, one per neuron. The truth is one such point and every estimate is another. The mean squared error is the squared distance between the two points, divided by 1,024. The picture shows three of the 1,024 directions, the means of neurons 1, 2 and 3, and the truth sits where the axes cross.

Each dot is one complete sampling estimate made with our budget, about 8,000 runs. The dots scatter around the truth in a cloud, and the large sphere marks their typical distance, 0.003 per neuron. The smaller sphere is the typical distance when the whole allowed compute is spent on sampling, 0.0011. The dark dot is the Gaussian closure, the standard method that computes the means instead of sampling them, as if every neuron's sum were an exact bell (page 15). It lands 0.0019 from the truth. Our estimator lands 0.00012 from the truth, almost on the crossing of the axes.

Because the score squares the distance, our estimate, 25.6 times closer to the truth than sampling with our budget, has an error 656 times smaller. The simplest method that computes, which treats every neuron as independent of every other, misses by 0.015 per neuron, five large spheres away and off the page.

There is a second difference. A sampling estimate is random, and repeating it lands somewhere else in the cloud. Our estimator is not random. Apart from rounding it lands on the same point every time, and its whole distance from the truth comes from its approximations. Better approximations move it closer; running it again changes nothing.

The mean of one neuron is an area



The means have to be computed, so start with one neuron. Its sum z changes from run to run, and some values are more common than others. The curve $p(z)$, top left, says how common each value is; it is the *probability density*, and the area under it is 1. For most neurons it is close to a bell curve, centred at the mean μ with spread σ, the *standard deviation* (the typical distance from the centre; its square, σ^2, is the *variance*).

The neuron outputs $\max(0, z)$, top middle. To average the output, weight every possible output by how often it occurs and add up. In the picture, multiply the two curves, and the area under the product (its integral), top right, is the mean.

$$\mathbb{E}\left[\max(0, z)\right] \;=\; \int_0^\infty z\, p(z)\, dz \;=\; \mu \cdot P(z > 0) \;+\; \sigma^2 \cdot p(0)$$

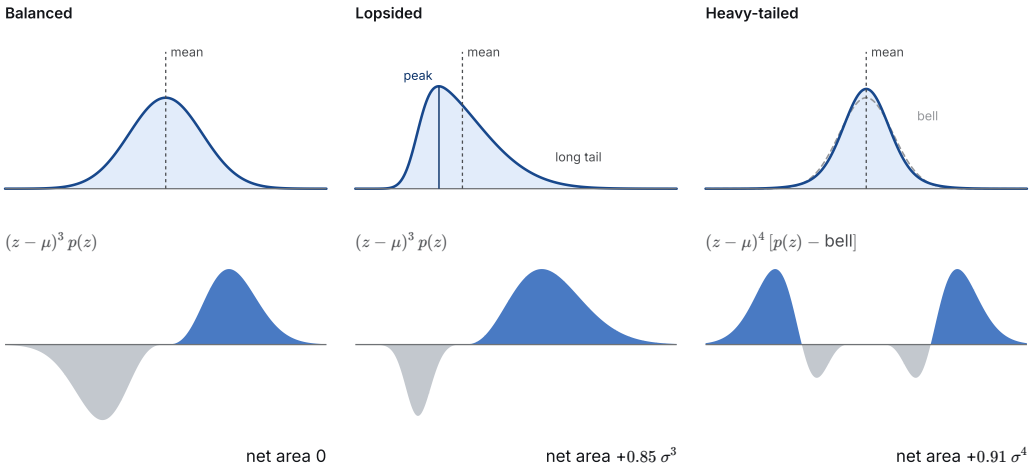
For an exact bell the area splits into two pieces read off the bell itself (bottom of the picture), the area to the right of zero, $P(z > 0)$, which is how often the neuron *fires* (gives a non-zero output), and the height at zero, $p(0)$, exactly at the kink of the ReLU. For the neuron drawn, with $\mu = 0.5$ and $\sigma = 1$, the pieces are 0.35 and 0.35, and the mean is 0.70.

Why does it split? Write $z = \mu + (z - \mu)$. The μ part gives $\mu \cdot P(z > 0)$. For a bell, the slope of the curve at any point is its height times $-(z - \mu)/\sigma^2$, that is $(z - \mu)\, p(z) = -\sigma^2 p'(z)$, so the $(z - \mu)$ part adds up to σ^2 times the height at zero. With $\alpha = \mu/\sigma$ the formula reads $\sigma\left[\alpha\Phi(\alpha) + \varphi(\alpha)\right]$, where $\Phi(\alpha)$ is the area of the standard bell left of α and $\varphi(\alpha)$ its height there.

Both numbers return later. The firing share $P(z > 0)$ is also the slope of the mean with respect to μ, so it decides how much of an error in a neuron's input passes to its output. The height $p(0)$ measures how much of the input sits at the kink, where all new shape in the network is made.

One caution. The split is exact only when the input is an exact bell, and after the first bend no input is.

Lopsided and heavy-tailed



Two words carry much of this story. The three curves in the picture have the same centre and the same spread and differ only in shape.

A curve is *lopsided*, or skewed, when one side stretches further than the other. In the middle panel most values sit left of the mean, the peak among them, while a long thin tail reaches far to the right and pulls the mean towards it. To put a number on this, take each value's distance from the mean, $z - \mu$, cube it, and average over all runs. Cubing keeps the sign, so values below the mean count against values above, and it magnifies far values most. With $\mathbb{E}[\,\cdot\,]$ for "the average over all runs" (the *expectation*) and κ, the Greek letter kappa, for these shape numbers, lopsidedness is measured by

$$\kappa_3 \;=\; \mathbb{E}\big[(z-\mu)^3\big] \;=\; \int (z-\mu)^3\,p(z)\,dz$$

The lower row draws the curve inside the integral, blue where it adds and grey where it subtracts; κ_3 is its net area. For the balanced bell, left, the two lobes are mirror images and cancel exactly. For the lopsided curve the right lobe wins, by $0.85\,\sigma^3$. Divided by σ^3, κ_3 becomes the *skewness*, here 0.85, a pure number.

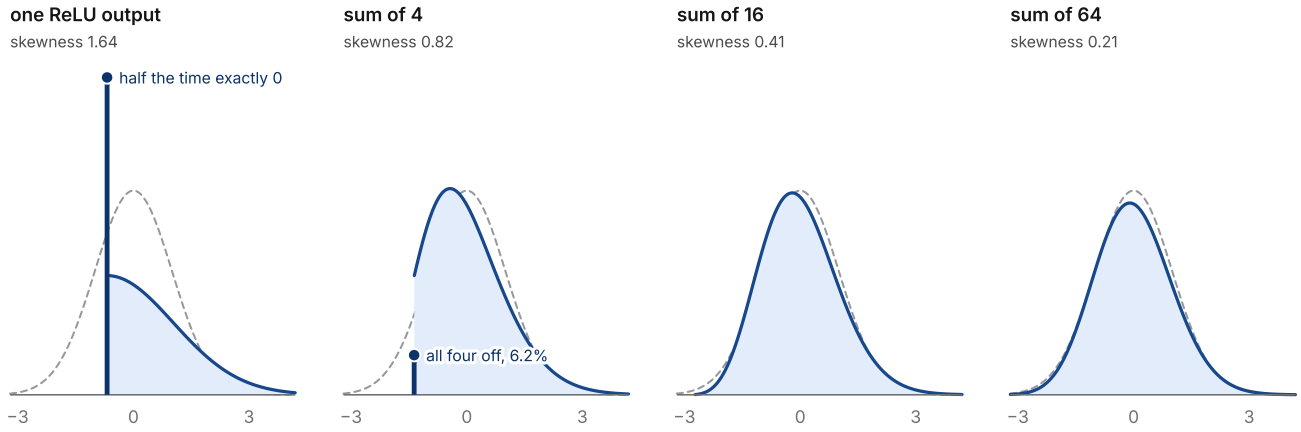
A curve is *heavy-tailed* when, against a bell of the same spread, it has more weight far out and in the middle and less in the shoulders between (right, the bell dashed). Fourth powers of the distance are positive on both sides and magnify the tails even more, so the measure compares them with the bell's own value, which is exactly $3\sigma^4$.

$$\kappa_4 \;=\; \mathbb{E}\big[(z-\mu)^4\big] - 3\sigma^4$$

The lower right draws $(z-\mu)^4$ times the gap between the curve and the bell. The shoulders, where the curve falls short, subtract; the tails, where it has extra, add; the centre counts for almost nothing, because there the fourth power is near zero. The net area is $+0.91\,\sigma^4$. Divided by σ^4, κ_4 becomes the *excess kurtosis*, here 0.91.

Both measures are zero for an exact bell, which is why they matter here. Each says, in one number, how far a neuron's input has moved from the bell curve that the formula of the previous page assumes.

Why a neuron's sum is nearly a bell curve



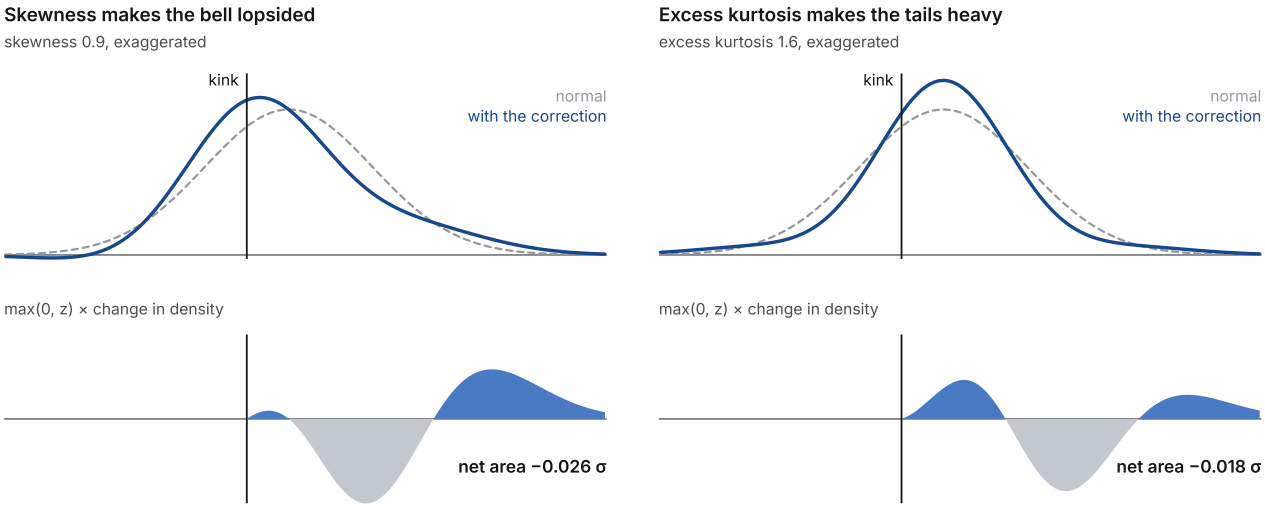
Is z a bell? At the first layer it is exactly, because a weighted sum of independent bell-shaped inputs is itself bell-shaped. After a ReLU it is not. The output of one ReLU, on the left of the picture, is exactly zero half the time and otherwise half a bell, lopsided to the right with a skewness of 1.64.

The next layer adds up 1,024 such outputs, and sums of many independent random quantities approach a bell, a fact known as the *central limit theorem*. The picture shows it at work, with each sum rescaled to the same centre and spread and the exact bell dashed. A sum of 4 outputs is already rounded, a sum of 16 is nearly a bell, a sum of 64 almost matches it. The skewness falls as one over the square root of the number of terms, to 0.82, 0.41, 0.21, and 0.05 for a sum of 1,024. Each term nudges the sum up or down a little; lopsided nudges in one direction are offset by others, and adding more terms rubs out every feature of their shape except the centre and the spread. The bell is the one shape that survives.

The outputs of a real layer are not independent, though. They all respond to the same input, so they rise and fall together, and the lopsidedness washes out more slowly. At layer 15 of our fresh networks the middle 90% of neurons have skewness between −0.17 and +0.17, and their tails are slightly heavier than a bell's (their excess kurtosis averages 0.06).

These departures are small. But our error target is near 10^{-8}, and at that scale they decide the answer. The estimator has to describe not only where each neuron's bell sits but how it is bent.

Four numbers describe the shape



The mean, the variance and the two measures of the previous pages are the first four rungs of a standard ladder of shape numbers, the *cumulants*, in which κ_1 is the mean, κ_2 the variance σ^2, κ_3 the lopsidedness and κ_4 the tail weight. Each is built from averages of products of values, and a cumulant's *order* is how many values are multiplied inside it, three for the third and four for the fourth. Every cumulant past the second is zero for an exact bell, so the higher ones measure exactly the departure from a bell.

How do κ_3 and κ_4 move the mean of a ReLU? The picture shows it, exaggerated. Top left, solid, is the bell with a skewness correction, against the exact bell with the same mean and spread, dashed. The correction is an approximation and, far out on the left, even dips below zero, which no real density does. Below it is what the ReLU sees, the change in density times $\max(0, z)$. The ReLU ignores everything left of zero, the dip included; to the right, blue areas add to the mean and grey areas subtract. Here they net to −0.026σ. On the right, heavier tails net to −0.018σ.

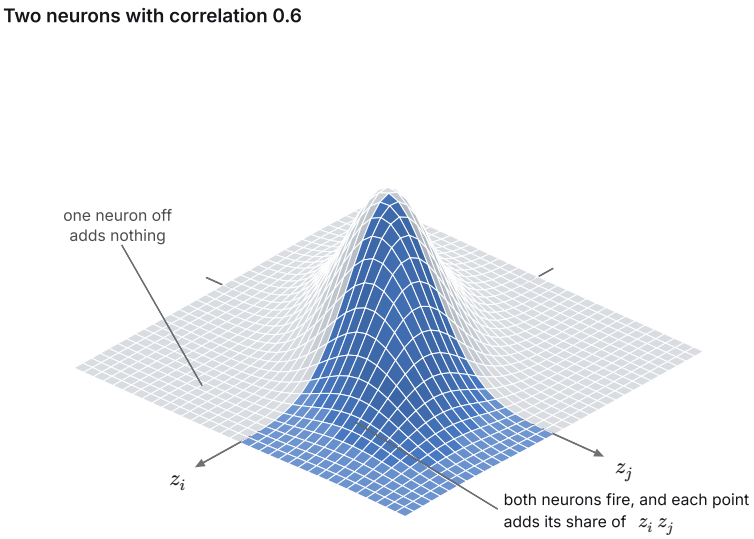
Those net areas are exactly the correction terms of the *Edgeworth expansion*, a standard way to write a nearly-bell density as a bell plus corrections in κ_3 and κ_4. Averaging the ReLU output under this corrected bell gives

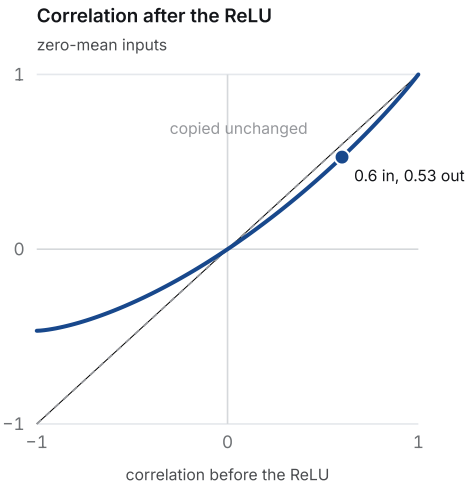
$$\mathbb{E}[\mathrm{ReLU}(z)] \approx \sigma\left[\alpha\Phi(\alpha) + \varphi(\alpha)\left(1 - \frac{\tilde{\kappa}_3}{6}\,\alpha + \frac{\tilde{\kappa}_4}{24}(\alpha^2 - 1) + \frac{\tilde{\kappa}_3^2}{72}\,\mathrm{He}_4(\alpha)\right)\right], \qquad \tilde{\kappa}_3 = \frac{\kappa_3}{\sigma^3},\ \ \tilde{\kappa}_4 = \frac{\kappa_4}{\sigma^4}$$

This is the answer of page 9 plus the two pictured corrections and a small term in the square of the skewness; $\tilde{\kappa}_3$ and $\tilde{\kappa}_4$ are the skewness and the excess kurtosis, and $\mathrm{He}_4(\alpha) = \alpha^4 - 6\alpha^2 + 3$ is a fixed polynomial.

How many shape numbers are enough? We gave this formula the exact cumulants of every final-layer neuron, measured on the same samples as our reference answers. With the mean and variance alone its squared error is 1.9×10^{-7}; with the skewness, 2.1×10^{-8}; with the kurtosis, 7×10^{-11}; later terms change it by less than 10^{-10}. We observed this on these networks and have no general proof, but it means that four numbers per neuron nearly fix its mean. The difficulty is getting the four right.

Neurons move together





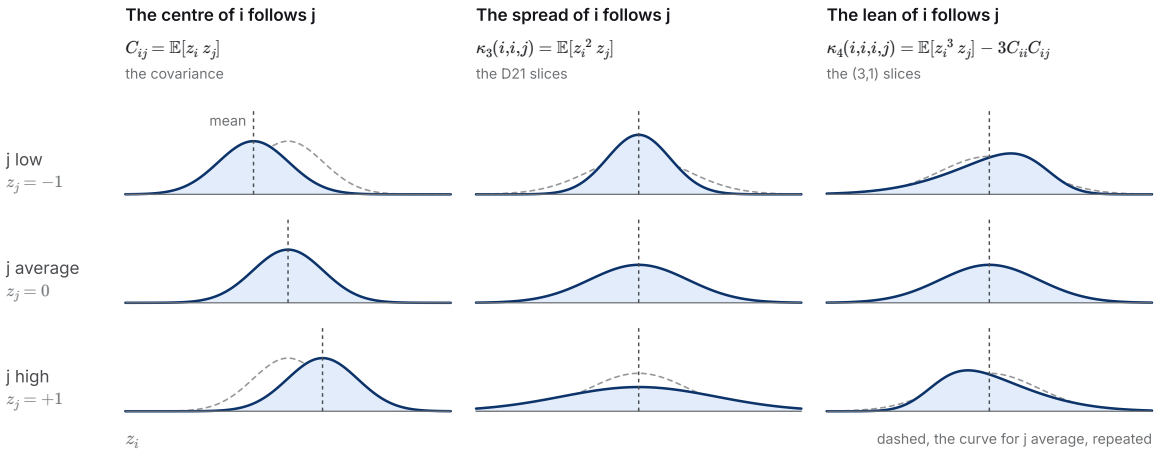
Getting them right is not a matter for one neuron alone. Two neurons of the same layer read the same previous layer, each through its own weights, so their values are linked, and when one is high the other tends to be high too, or low. The *correlation* ρ measures the link, from −1 (always opposite) through 0 (no tendency to rise or fall together) to 1 (always together). The *covariance* is the same measure in the neurons' own units, $\rho\,\sigma_i\sigma_j$, the average product of the two neurons' distances from their means. For a whole layer the covariances form a 1,024 × 1,024 table, the *covariance matrix* C; its diagonal holds each neuron's variance.

The hill on the left is the joint density of two neurons with correlation 0.6. Its height over each point (z_i, z_j) says how often that pair of values occurs, and the correlation stretches it along the diagonal. Both ReLUs act at once. Only in the blue quadrant, where both sums are positive, are both outputs non-zero, and every point there contributes $z_i z_j$ to the average product of the outputs. In the grey region at least one output is zero.

The correlation after the ReLUs is therefore a fixed function of the correlation before, drawn on the right for neurons centred at zero, where 0.6 becomes 0.53, and −1 becomes −0.47. The ReLU weakens the link because it throws away the negative half of each neuron. Whenever one of the two sums is negative, its output is zero whatever the other does. Even exactly opposite inputs give outputs correlated at only −0.47. When one output is positive the other is zero, but how large the first is says nothing more about the second.

Covariances matter for the means because the next layer's sums add up many neurons, and the variance of a sum is the sum of all the covariances of its terms. So C sets σ for every neuron of the next layer, and σ sits in the formula for the mean. Moving C exactly from one layer's ReLU outputs to the next layer's sums is just arithmetic, $C \leftarrow WCW^\top$, where W is the table of weights and $W^\top$ the same table flipped. The hard part is what the ReLU does to C, and there the shape enters again.

Three ways two neurons are linked



Covariance captures one way two neurons are linked. The picture shows it and two subtler ways. Each column is one pair of neurons, measured from their own means, and its three rows show the distribution of i only on the runs where j is low, average or high. The dashed curve repeats the middle row, so that the change stands out.

On the left the curve slides to the right as j rises, so the centre of i follows j. That is the covariance, the average of the product of the two, $C_{ij} = \mathbb{E}[z_i z_j]$.

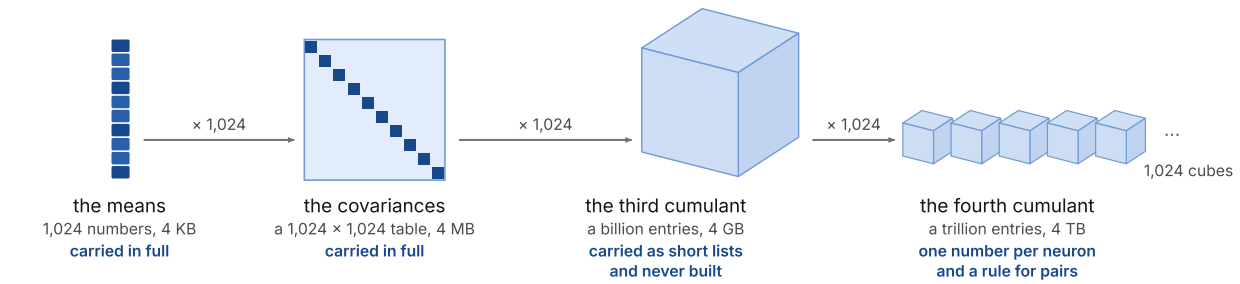
In the middle the curves keep their centre but widen, so that i varies more when j is high. The covariance of these two neurons is zero, yet they are far from unrelated. The link appears one rung higher on the ladder of cumulants, in a *joint* cumulant, one whose values come from different neurons, here $\kappa_3(i,i,j) = \mathbb{E}[z_i^2 z_j]$, with two values from i and one from j. Picture all the $\kappa_3(i,j,k)$ as a cube with one entry per triple of neurons (page 20 draws one); the entries with two equal indices form a slice through it. The estimator calls them the *D21 slices*, after their indices, two equal and one different; the diagonal $\kappa_3(i,i,i)$, all three equal, is *D3*.

On the right the curves keep their centre and their spread but *lean*, their lopsidedness flipping from one side to the other as j rises. The lean of i follows j. This is the joint fourth cumulant $\kappa_4(i,i,i,j)$, three values from i and one from j; the term $3C_{ii}C_{ij}$ removes what two exact bells with the same covariance would already show. These numbers are the *(3,1) slices* of the fourth cumulant.

Why should a mean care about such fine links? The next layer is built from ReLU outputs, and how strongly the output of i moves with j depends on how much of i's bell the ReLU cuts away. That depends on i's spread and lean; if they change with j, the cut changes with j, and so does the covariance after the ReLU. The method we built on carries the D21 slices and treats the (3,1) slices as a multiple of the covariance; a better rule for them became one of our additions (page 35).

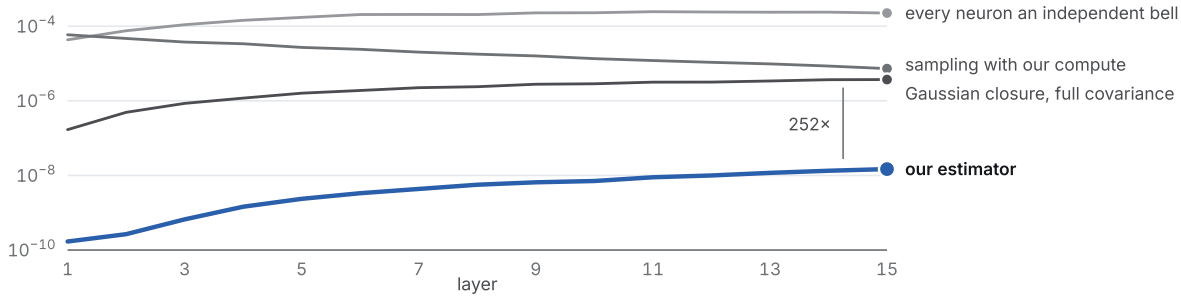
What would have to be carried

What one layer would hold



What each level of description is worth, layer by layer

squared error per neuron

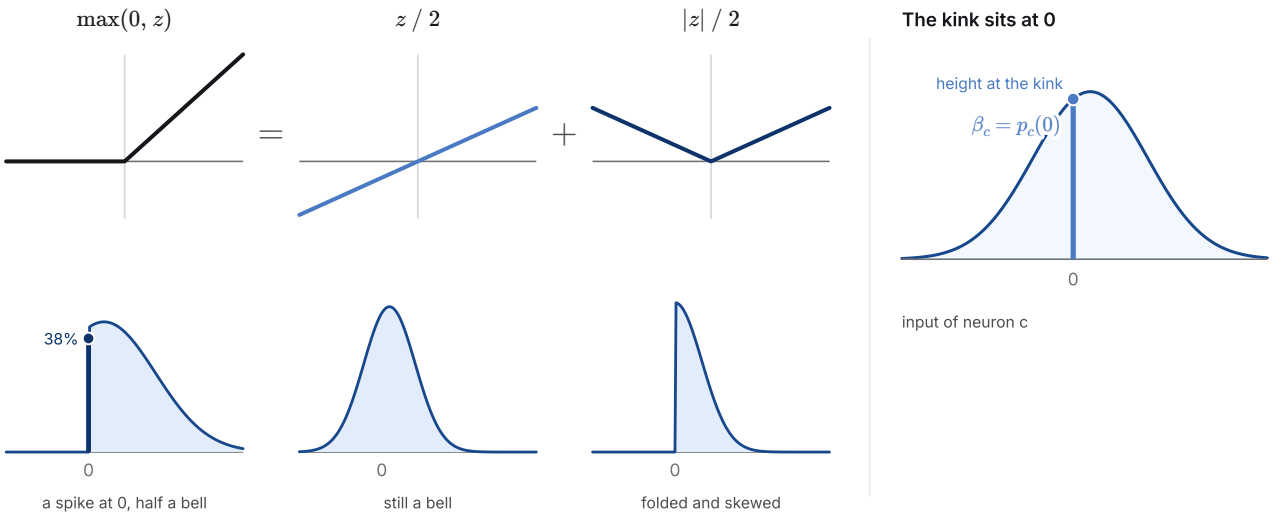


To get one neuron's mean we need four numbers about its sum, its mean, variance, κ_3 and κ_4. To get the next layer's variances we need this layer's covariances, and after a ReLU those depend on shape too, through the joint cumulants of the previous page.

Counted for one layer of 1,024 neurons, that is 1,024 means, about half a million distinct covariances, a billion entries of the third cumulant, one for every ordered triple of neurons (180 million distinct), and a trillion of the fourth, each 1,024 times the one before (top of the picture). A billion numbers take 4 GB, and moving them through one layer would cost about 6.6 trillion FLOPs, three times the whole allowed compute; the trillion cannot even be stored. Below them is what each level of description is worth, layer by layer, on two fresh networks (reference noise removed).

- Treating every neuron as an independent bell, carrying only means and variances, gives a squared error of about 2×10^{-4} at depth, worse than sampling.
- The *Gaussian closure* carries the full covariance matrix and assumes every sum is an exact bell, with every κ_3 and κ_4 zero (a *closure* is a rule that fills in the numbers a method does not carry). It is the standard estimator among those that calculate from the weights, and at the last layer its error is 252 times ours.
- Sampling with our budget, about 8,000 runs, is worse than the Gaussian closure at every layer, and about 500 times worse than ours at the last layer of these two networks.
- Our estimator also carries the third cumulant, compressed, and the fourth as one number per neuron plus a rule for pairs.

New shape is made only at the kink



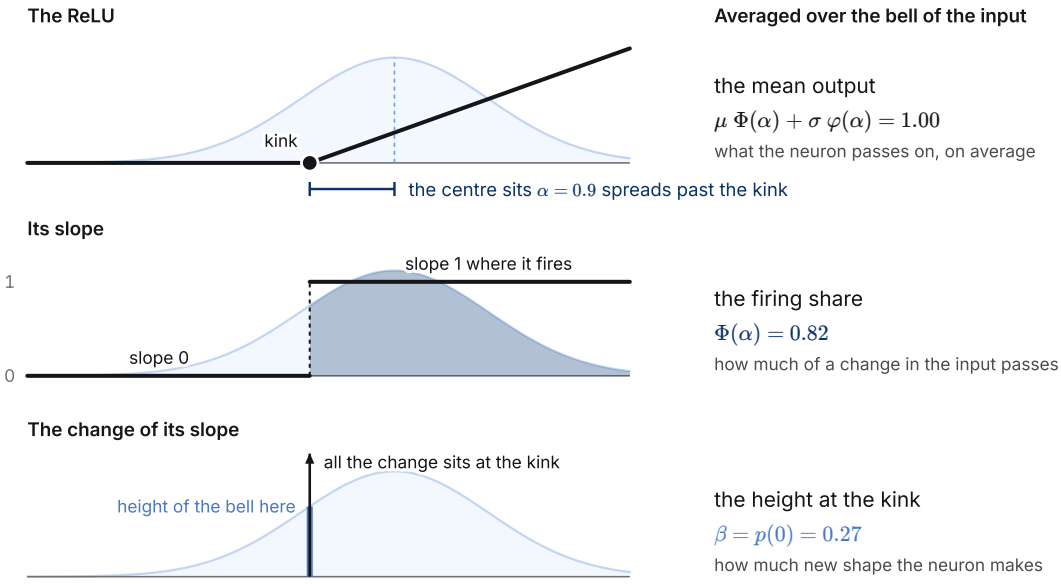
Where does the shape come from? Weighted sums of bells are bells, so only the ReLUs change shapes, and within a ReLU only one point matters, the *kink* at zero, where its slope jumps. Split the ReLU into a straight half and a V.

$$\max(0, z) = \frac{z}{2} + \frac{|z|}{2}$$

The straight half passes a bell through as a bell, its centre and spread scaled and its shape unchanged (second column of the picture). The V folds the bell at zero, and the fold is what makes the output lopsided (third column); away from zero the V is straight too. The first column shows the ReLU's own output, a spike at zero that holds the 38% of runs below zero, beside the part of the bell above it. We call this folding of a neuron's bell at the kink a *bend*.

So the two straight pieces of the ReLU leave a shape as it was, and everything new comes from the single point where they meet. How much shape it passes on to other neurons depends, to leading order (keeping only the largest terms), on one number, the height of the bell at the kink, drawn on the far right and written β. The next page shows why.

What the kink does



Three things a neuron does can be read off this picture, each by averaging something over the bell of its input. The bell drawn has its centre 0.9 spreads past the kink. That distance, counted in spreads, is $\alpha = \mu/\sigma$, the position of the bell relative to the kink.

Averaged over the bell, the ReLU's output gives the neuron's mean output, the area of page 9, here 1.00.

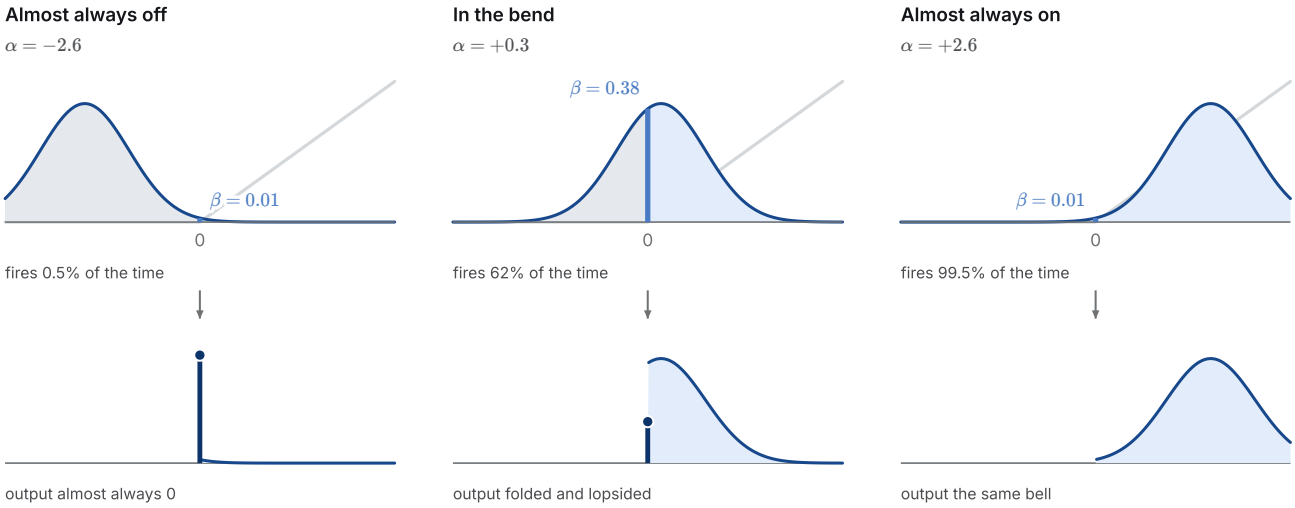
Averaged over the bell, its slope gives the share of runs in which the neuron fires, $\Phi(\alpha)$ = 0.82. The slope is 0 left of the kink and 1 right of it, so its average is the area of the bell to the right (shaded). This share decides how much of a small change in the input reaches the output, and so how errors travel (page 34).

Averaged over the bell, the change of its slope gives the height of the bell at the kink, β = 0.27. The slope changes in one place only, all at once, so its change is a single spike at the kink, and averaging a spike over the bell picks out the bell's height there. This is the number that decides how much new shape the neuron makes. A classical result of *Gaussian calculus*, the rules for averaging functions of bell-shaped variables, makes this exact when the inputs are jointly bell-shaped (Price's theorem, a form of Stein's lemma).

$$\operatorname{cum}\big(\mathrm{ReLU}(z_c),\, z_j,\, z_k\big) \;=\; \beta_c\, C_{cj}\, C_{ck}, \qquad \beta_c = p_c(0)$$

Here cum is a joint cumulant, as on page 14, measuring how the output of c and the sums of two other neurons j and k are lopsided together. In words, it is the height of c's bell at the kink, times how strongly j moves with c, times how strongly k moves with c. It is exact while j and k have not yet passed through their own ReLUs; page 20 takes that step.

Three kinds of neuron



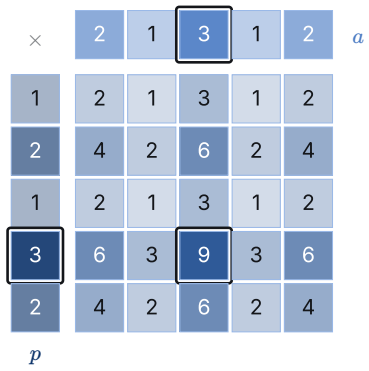
The position α of a neuron's bell sorts neurons into three kinds. At $\alpha = 0$ the bell is centred on the kink, and at $\alpha = -2$ its centre lies two spreads below it. The picture shows three neurons and, under each, what comes out of its ReLU.

- **Almost always off** (left, $\alpha = -2.6$). Almost the whole bell lies below zero, where the ReLU outputs 0, so the neuron fires in only 0.5% of runs. Its output is a spike at zero with a sliver beside it. Its β is 0.01, so it makes almost no new shape.
- **In the bend** (middle, $\alpha = 0.3$). The bell straddles the kink, and the ReLU bends it. The 38% of runs below zero collapse onto the spike at 0, while the other 62% pass unchanged, as the part of the bell above zero. The output is folded and lopsided. Its β is 0.38, close to the highest point of the bell, 0.40 (both for a bell of spread 1, as drawn). This is where new shape is made.
- **Almost always on** (right, $\alpha = 2.6$). Almost the whole bell lies above zero, where the ReLU is the straight line $z \mapsto z$. The bell passes through unchanged, shape and all, and β is again 0.01.

So every neuron is roughly one of three kinds, and the estimator treats each differently, with the full machinery for neurons in the bend, roughly $-2 < \alpha < 2.3$, far less for neurons that are almost always off, and somewhat less for neurons that are almost always on (page 31). The same position decides what happens to a mistake, which reaches the output in proportion to the firing share, hardly at all when the neuron is almost always off and nearly in full when it is almost always on. From here on the story follows the shape that neurons in the bend create.

A big table from short lists

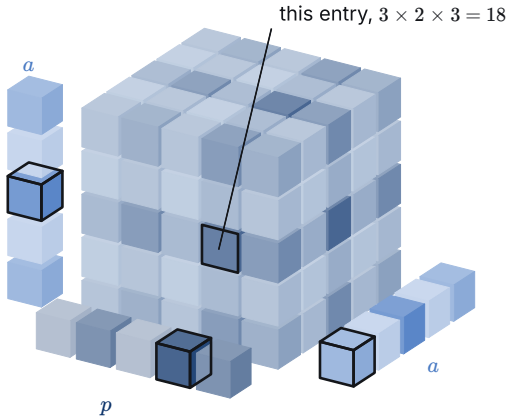
Two lists make a table



entry $(i, j) = p_i\, a_j$

10 numbers stored, 25 entries

The same lists make a cube



entry $(i, j, k) = p_i\, a_j\, a_k$

the same 10 numbers, 125 entries

The shape shared between neurons lives in tables far too big to store. The third cumulant of a layer has an entry for every triple of neurons, $\kappa_3(i, j, k)$, a cube of $1{,}024^3$, about a billion numbers. Yet the cube that one bend creates fits in two short lists, by an old and simple trick, shown here at toy size.

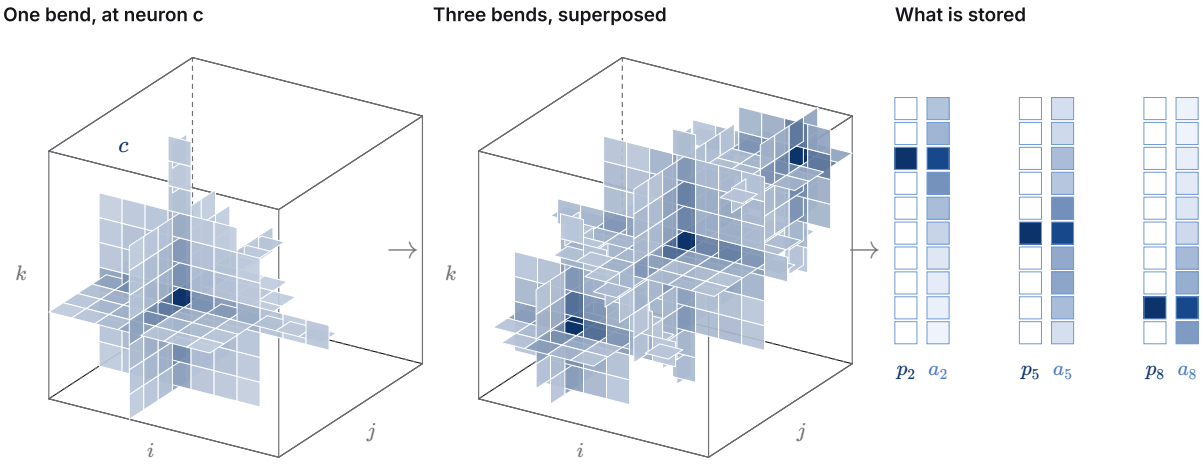
On the left are two short lists, p down the side and a across the top, and the table they make. Each entry is the number at the start of its row times the number at the head of its column, as in a school multiplication table; the entry outlined, 9, is 3 × 3. Ten stored numbers describe all 25 entries, and any entry can be produced the moment it is needed, with one multiplication.

On the right the idea gains a dimension. Multiply every entry of the table by the numbers of a third list, here a again, laid along the depth, and the table becomes a cube whose entry (i, j, k) is $p_i\, a_j\, a_k$. The entry outlined is 3 × 2 × 3 = 18, one number taken from each of the three rods. The same ten numbers now describe 125 entries.

A table or cube built this way is an *outer product*, written $p \otimes a$ for two lists and $p \otimes a \otimes a$ for three; we call the three-list kind a *triad*. At full width the saving is enormous, since two lists of 1,024 numbers describe all $1{,}024^3$ entries of a cube, a ratio of half a million to one.

Most cubes are not triads, and a sum of a few triads can describe only cubes with that much structure. A bend, as the next page shows, creates this structure, to leading order.

One bend leaves a simple pattern



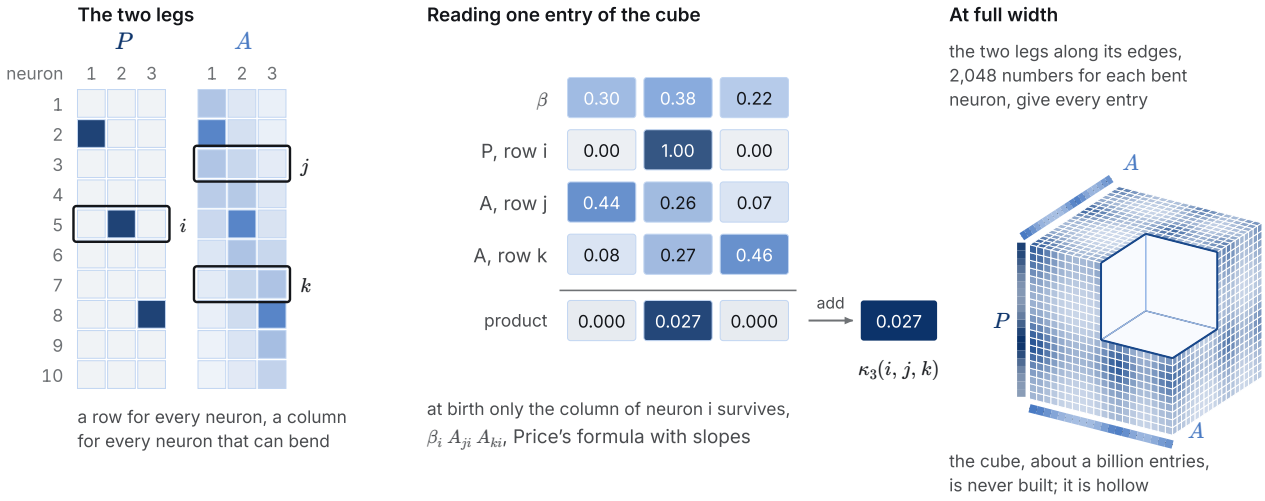
Here is the third cumulant of a layer drawn as a cube, for a toy layer of ten neurons. Entry (i, j, k) records how neurons i, j and k are lopsided together.

Which entries can one bend, at neuron c, fill? To leading order the new shape is the formula of page 17, $\beta_c C_{cj} C_{ck}$, so c must be one of the three indices. The bend writes only three planes through c, the planes $i = c$, $j = c$ and $k = c$ (left), each carrying the same pattern, c's covariance column multiplied by itself. Several bends write several sets of planes, and their sum is the dense-looking cube in the middle.

In the language of the previous page, each bend writes a triad. Its first list, p_c, marks neuron c with a 1 in place c and zeros elsewhere. Its other two lists are both a_c, c's covariance column with each entry multiplied by that neuron's own slope, since the other neurons pass through their ReLUs too. This is the *leading* part of the new lopsidedness; terms in which two or three kinks act together carry higher powers of the covariances, and the triads leave them out. The bend adds β_c times $p_c \otimes a_c \otimes a_c$, once with p in each of the three places (hence the three planes). For the toy layer the cube has 1,000 entries and three bends store 60 numbers (right); at full width a bend stores 2,048 numbers against a billion entries, and the cube is never built.

This is the central idea of the method we built on. Team 504aldo released that method as an open-source estimator, V29, under the MIT licence; it reimplements the factored algorithm of arXiv:2605.05179. Everything we added was built on top of it.

A source and its two legs



All the triads made at one layer's *gate*, the estimator's step for that layer's ReLUs, form a *source*, the lopsidedness that the layer adds to the network. A source is stored as two tables, its *legs*, drawn on the left for a toy layer of ten neurons in which three bend. A triad stands on its three lists as a stool on three legs, and two of its lists are the same, so a whole source needs only two tables.

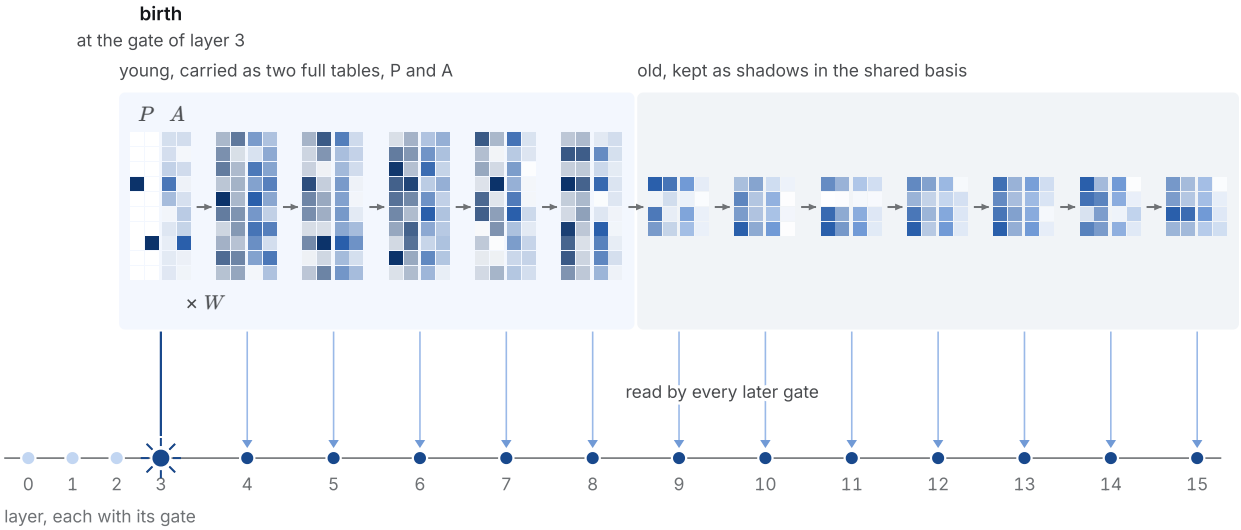
Each leg has a row for every neuron and a column for every neuron that can bend, all but those almost always off. In the leg P each column holds a single 1, in the row of the neuron that bent, so P records where the bends happened. In the leg A each column holds that neuron's covariance column, each entry times its own neuron's slope, so A records how strongly every neuron moves with the one that bent. Drawn this way, a column's darkest cell is the bent neuron's own row, its variance. The stored leg leaves that cell at zero, because the entries in which the bent neuron appears more than once, its own lopsidedness among them, are computed directly and enter through further terms of the full rule (page 24). Since 4 October each column also carries the shape that the input of the bent neuron already had (page 53).

The cube is never built, because any entry can be read straight from the legs. To read $\kappa_3(i, j, k)$, take row i of P and rows j and k of A, multiply them column by column together with each bend's β, and add (middle). At birth only the column of neuron i survives, since P has zeros everywhere else, and the reading gives back the formula of page 17, slopes included. Later the legs are mixed by the weights and P fills in, but the reading stays the same. The full rule adds the same products with the row of P moved to the second and the third place.

At full width, right, the legs of one bent neuron lie along the edges of the cube they describe, drawn a few layers after birth, once P has filled in. The faces show their products; the cut-away corner shows the cube hollow, as it is never built. The legs hold 2,048 numbers against a billion entries.

The moment a layer's ReLUs create a source is its *birth*; its age is counted from there.

The life of a source



Think of a stone dropped into a pond. The splash happens at one place and one moment, and the ripples then spread over the whole pond. A birth is the splash and the source is the ripple it sets off; the picture follows one from birth to the last layer.

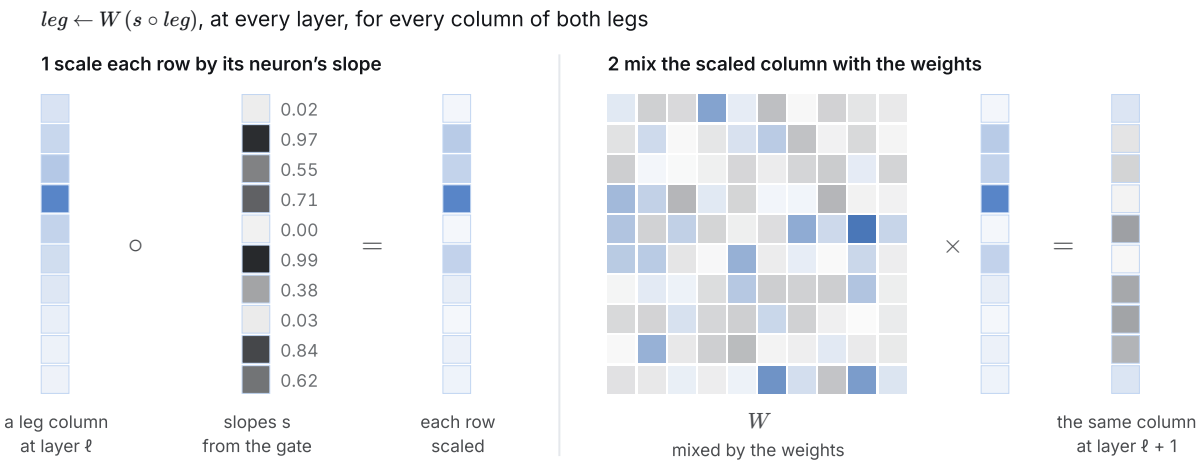
It is born at the gate of layer 3, where a few neurons in the bend, two in this toy, fold their bells. At birth its legs are two thin tables, P, with a single 1 in each column marking a neuron that bent, and A, with the covariance columns of those neurons. Before layer 3 the source does not exist, and the pale gates of layers 0 to 2 know nothing of it.

At every later layer both tables are multiplied by the weights, $\times W$, just as the numbers of the network are. Watch P. After one layer its two dark cells have spread down whole columns, because every neuron of the next layer reads both of the neurons that bent, and the lopsidedness created at two neurons now reaches all of them.

At every later gate the source is read. A gate has to know how lopsided its neurons' inputs are, and the answer is a sum over every source alive at that point. The arrows from the tables down to the gates of layers 4 to 15 are these readings, which the estimator calls *queries*. Meanwhile each gate makes new shape of its own, and so a new source, which means the gate of a deep layer reads many sources, each born at a different gate.

For its first five layers after birth the source is *young* and travels as its two full tables. After that it is *old*, and its tables are replaced by small summaries in a shared set of directions (page 25), shadows that cost far less to keep and are rebuilt when a gate needs them. Old sources still count, all the way to the last layer.

How a source travels



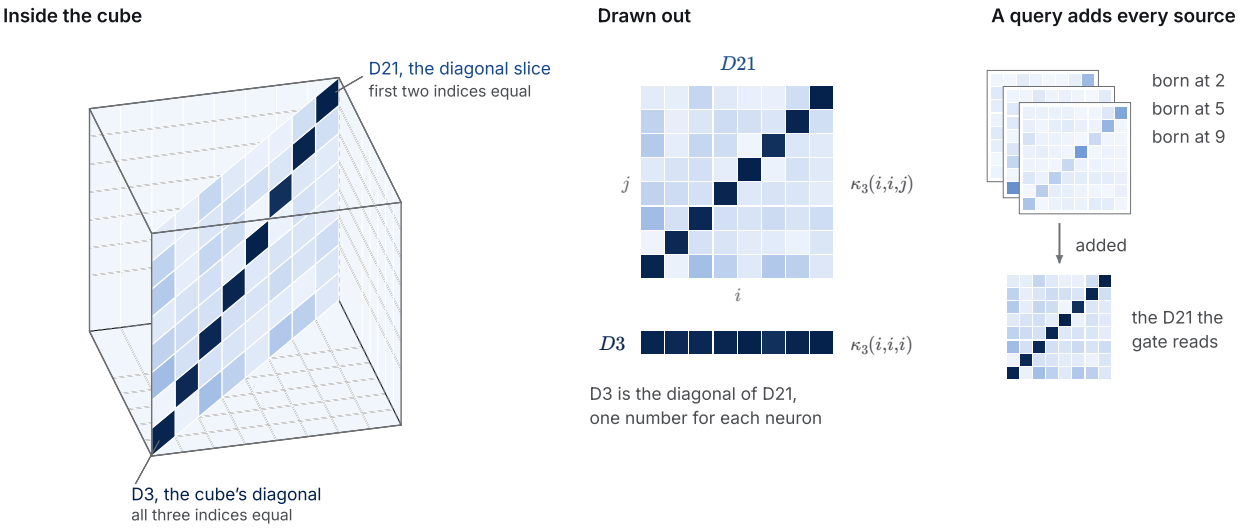
A source born at layer 3 does not stay at layer 3. Every later layer multiplies by its weights and applies its ReLUs, and the shape born at layer 3 is carried along and mixed. The method moves the legs instead of the cube, one column at a time, in the two steps of the picture. First each row is scaled by its neuron's slope, the firing share of page 17, so that the rows of neurons that are almost always off are all but erased while the rows of neurons that are almost always on pass nearly whole. Then the scaled column is mixed by the weights, exactly as the network's own numbers are. In symbols,

$$\text{leg} \;\leftarrow\; W\,(s \circ \text{leg})$$

where s holds the slopes and $\circ$ multiplies entry by entry.

Why does this work? The next layer's sums are a fixed mix of this layer's outputs, and cumulants pass through a fixed mix exactly. The cube of the next layer is this layer's cube transformed by W along each of its three sides, and for a triad that means each list is transformed on its own, $(Wp) \otimes (Wa) \otimes (Wa)$. The ReLU in between passes each neuron's part on in proportion to its slope, which is the first step. That step holds *to leading order*, keeping the largest terms and dropping much smaller ones. So a source born at layer 3 still describes, at layer 12, the shape it contributes there.

What a gate reads



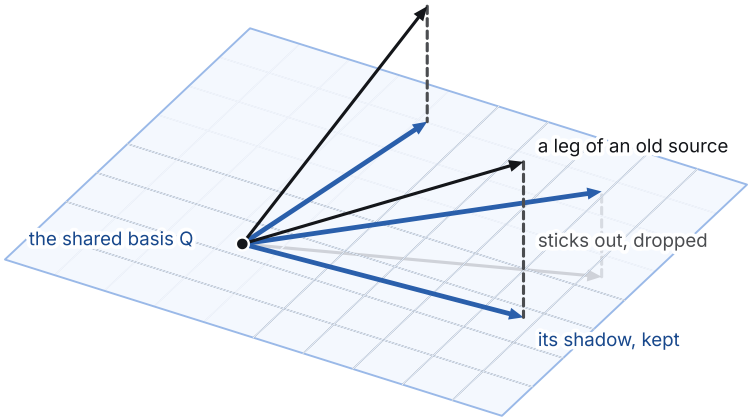
A gate does not need the whole cube either. It needs the two views of it drawn on the left. **D3** is the cube's diagonal, $\kappa_3(i,i,i)$, each neuron's own lopsidedness, which enters the neuron's mean through the formula of page 12. **D21** is the diagonal slice, $\kappa_3(i,i,j)$, how the spread of neuron i moves with neuron j (page 14), which enters the new covariances. In the cube D21 stands as a wall from the nearest edge to the farthest, and its own diagonal is the cube's diagonal, D3. Drawn out, D21 is a table with one number per pair.

Every source alive contributes to both, so before each gate a *query* reads each source's share from its legs and adds the shares up (right). Written out for any entry, the reading rule of page 21 is a weighted sum over the stack of triads,

$$\kappa_3(i,j,k) \;\approx\; \sum_c \beta_c \left(P_{ic}A_{jc}A_{kc} + A_{ic}P_{jc}A_{kc} + A_{ic}A_{jc}P_{kc}\right)$$

with one term for each place of P. For D3 all three indices are equal, so it costs only element-by-element work. For D21 two are equal, and the sum over c becomes one product of a 1,024 × b matrix with a b × 1,024 matrix per source, where b, its number of columns, counts the neurons kept at its birth, all but those almost always off, rounded up to a multiple of 64. That product, repeated for every source still in use at every layer, is the largest single share of the compute (page 28). The full rule in V29 has two more kinds of term, for the entries in which a bent neuron appears more than once, its own lopsidedness and how its spread moves with each other neuron. The first is computed exactly at birth, the second exactly in its main part with the rest compressed to six directions, the *extra piece* (64 directions made no measurable difference on 16 fresh networks), and both are stored as thin matrices of the same shape, so the same reading applies.

Old sources share one basis



Sixteen layers can mean fifteen sources at the last layer, one born at each earlier gate. Each young source costs two large products per layer, one per leg, and moving all of them through every layer would cost more than the budget allows. So after five layers a source is compressed.

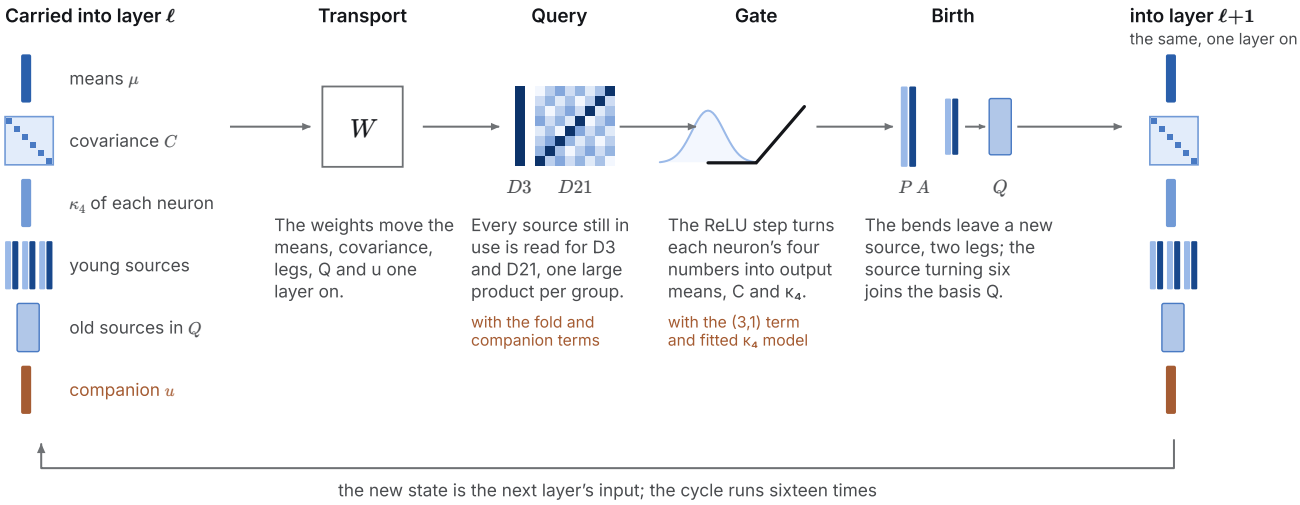
The picture shows the idea in three dimensions. The real space has 1,024 directions, one per neuron. The method keeps 320 well-chosen directions, shared by all old sources, which form the *basis* Q, drawn here as a plane. A basis is a set of directions from which others are built; an arrow lying in the plane is so much of one direction plus so much of another. Each leg column, a list of 1,024 numbers, is an arrow in the 1,024-direction space of page 8. Once a source has been young for five layers, each of its leg columns is replaced by its shadow on the plane, its *projection*, and the part sticking out of the plane is dropped. From then on the source is stored as small 320 × b factors that say how much of each basis direction its legs contain.

This saves work because the basis is moved through the weights once per layer, in the same product that moves the covariance, while the factors of every old source stay valid without being touched. When a gate needs an old source's legs, they are rebuilt from the basis and the factors.

The basis is chosen once, from the legs of the first source to be compressed, by a standard randomised method for finding the directions that carry most of a matrix. It then moves with the covariance, and each later source is projected onto it as it joins.

What sticks out of the plane is lost. It is small, which is why compression costs only about 3% of the error, and a later addition, the companion (page 37), recovers part of it.

One layer of the estimator



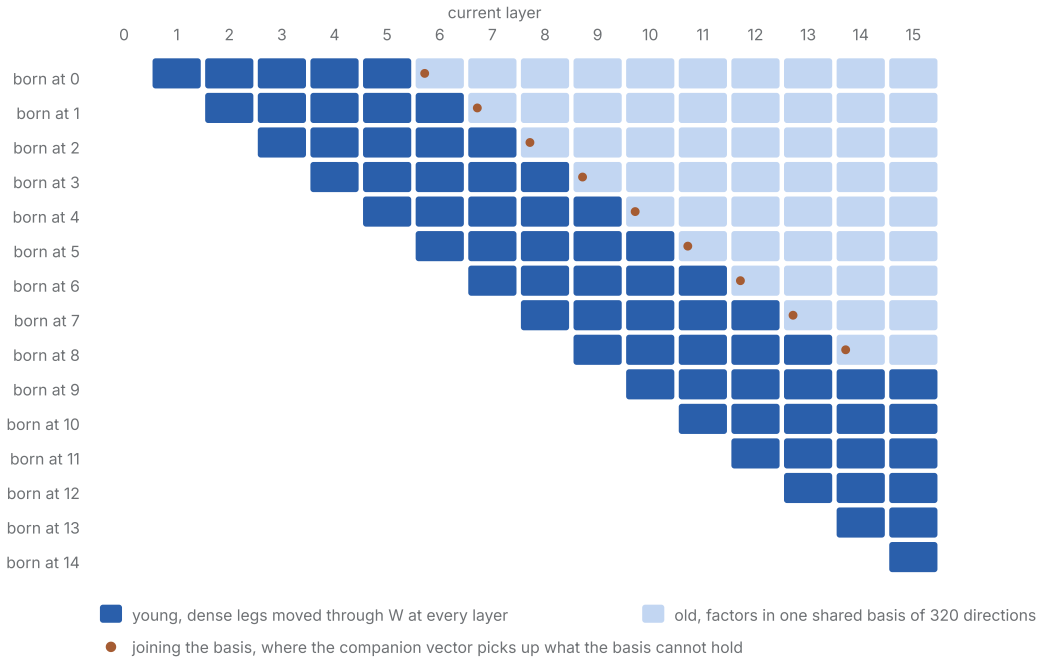
Put together, one layer of the estimator holds the means μ, the covariance C, the sources still in use and an estimate of each neuron's κ_4, and moves all of it forward by one layer in four steps.

- **Transport.** Means and covariance move exactly through the weights, as $\mu \leftarrow W\mu$ and $C \leftarrow WCW^\top$. Young sources move as legs; old ones are held more cheaply (page 25). The κ_4 estimate moves by a rule inherited from V29, which takes the fourth cumulant shared by two neurons as a multiple of their covariance, one number per layer retuned to each network as it runs, so each new κ_4 is a mix of the old ones through $W \circ W$, the table of squared weights, plus that multiple of the covariance received. A fitted model then corrects it.
- **Query.** For every neuron that can still fire, the estimator assembles D3 and D21 from all sources, in one large product per group of sources.
- **Gate.** The estimator's ReLU step. Formulas from Gaussian calculus turn each neuron's μ, σ^2, κ_3 and κ_4 into its output mean, the output covariance and the output κ_4. To leading order the output covariance of two neurons is $s_i s_j C_{ij}$, their input covariance times both slopes; the D21 slices and the rule's (3,1) and (2,2) slices, $\kappa_4(i,i,i,j)$ and $\kappa_4(i,i,j,j)$, add the corrections of pages 14 and 35.
- **Birth.** The gate records the new source it has just created.

Three habits keep this affordable. Neurons almost always off are left out of queries, transport and births, those almost always on are left out of most queries (page 31), and the last layer needs only means, so its covariance and births are skipped.

On the left of the picture is what one layer holds; the four steps run across, and the long arrow along the bottom hands the new state back as the next layer's input, sixteen times in all. Clay marks our team's additions, such as the companion u (page 37). Only four things are approximate, the gate's formulas, the leading-order rules that move sources and κ_4, the cuts that skip settled neurons, and the compression of old sources and of part of each birth. Everything else is exact up to rounding.

Young and old sources



The picture shows the life of every source in a 16-layer network. Each row is one birth and each column the layer being computed. A source is carried as full legs (dark) for its first five layers and then as factors in the shared basis (light); the dot marks the layer at which it joins. At the last layer, which skips the join, six sources are young and nine are old. The last layer makes no source of its own, because only its means are needed.

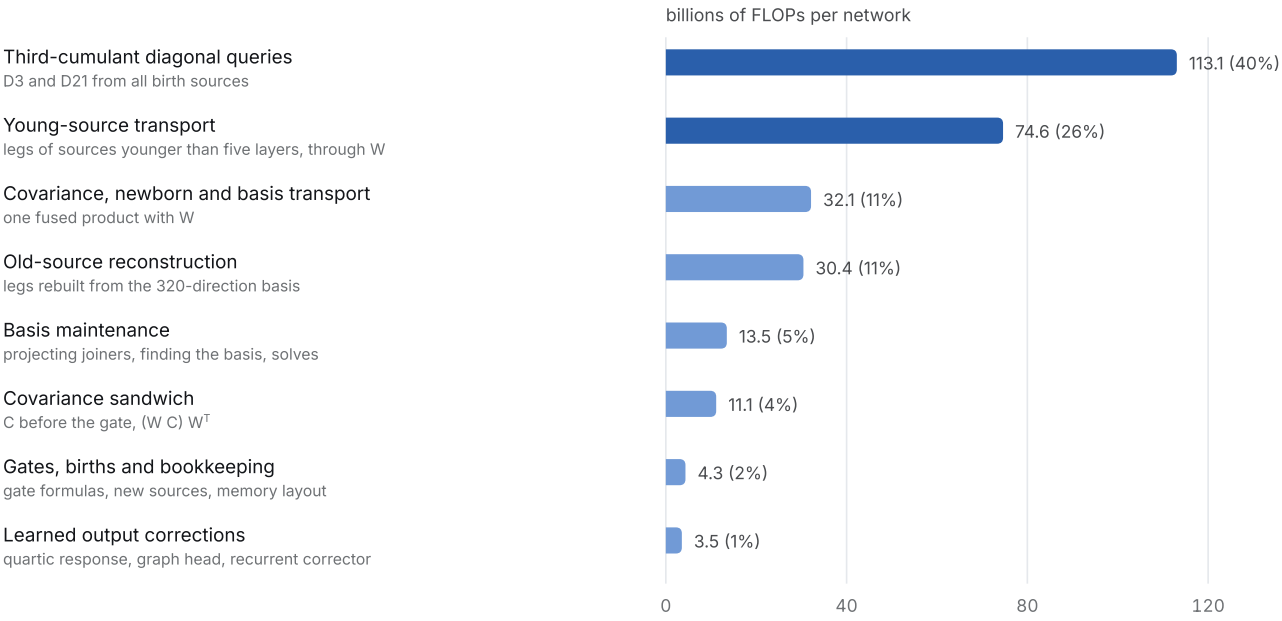
In numbers, a young source with b born neurons is two $1{,}024 \times b$ matrices, and moving it costs two products of the $1{,}024 \times 1{,}024$ weights with a $1{,}024 \times b$ matrix at every layer. An old source is two $320 \times b$ factors that never move; only the basis, $1{,}024 \times 320$, is moved, once per layer for all of them.

These settings sit at a measured *knee*, the point beyond which pushing further buys little. A basis of 256 directions instead of 320 saved about 5% of the compute and raised the error by 10 to 12%; compressing after four layers instead of five saved 3 to 5% and cost 11 to 13%; keeping every source in full would remove only about 3% of the error for 30% more compute.

This is V29's design. Rebuilt on it, with only the depth of its fast multiplication (page 30) changed, our first build scored 7.4×10^{-9} on the 100 public networks on 11 September, against 3.5×10^{-8} for our best method until then. That gain belongs to Team 504aldo.

It spent 735 billion FLOPs per network, more than three times the 220-billion floor. Above the floor error and compute trade one for one, so the next target was the same answer for fewer FLOPs.

Where the FLOPs go



To pay less, first find where the FLOPs go. Every operation reaches the meter as a *call* to flopscope; we intercepted each call and charged it to the line of our code that made it. The picture shows the whole bill of one fresh network at the full settings of the final estimator, 283 billion FLOPs (the graded average over the 50 leaderboard networks was 271 billion).

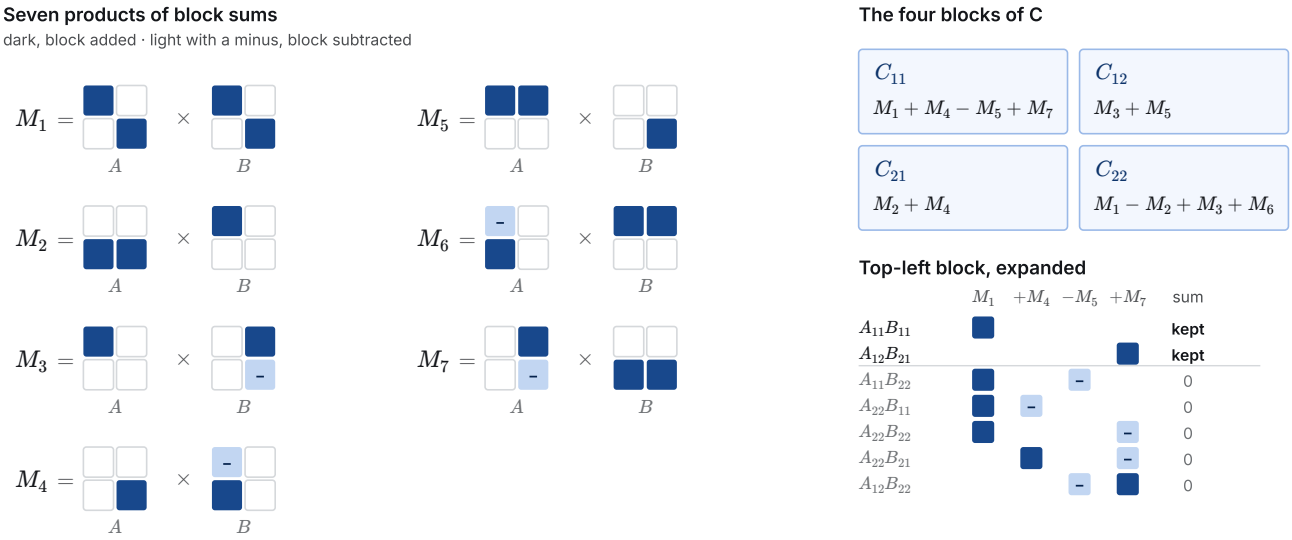
Of that bill, 40% assembles the third-cumulant views D3 and D21, and 26% moves young sources from layer to layer. Matrix products are 81% of the bill, almost all of them these bulk products. The gates, the births and all the per-neuron formulas together cost under 2%, and the learned corrections 1%.

The rows are the jobs of the loop. Queries assemble D3 and D21; transport moves young legs; one combined product moves the covariance, the newborn legs and the basis through W; reconstruction rebuilds old legs from the basis; basis maintenance refreshes it; the covariance sandwich forms $WCW^\top$; then come gates, births and bookkeeping, and the learned corrections.

So the price of a matrix product decides the bill. The meter charges a product by its shape, whatever the numbers in it, and charges every element-by-element operation one FLOP per element, even a reshape, which only rearranges numbers.

That leaves three ways to pay less, by making each product cheaper, skipping work whose result is already known, and removing work that is done twice. Our later additions were designed around this picture too. The quartic fold (page 36) rides a product that is made anyway, the companion (page 37) costs only matrix–vector work, and the learned corrections (page 39) run once, after the pass. The dearest item we could still change was the number of young sources, and it was not worth changing, since one fewer saves 4.6% of the FLOPs and costs 12.9% in error.

Seven multiplications instead of eight



Most of the bill is matrix products, so start there. Multiplying two $n \times n$ tables the schoolbook way, row times column, takes n^3 multiplications and about as many additions, about $2n^3$ FLOPs in all. In 1969 Volker Strassen found a cheaper way.

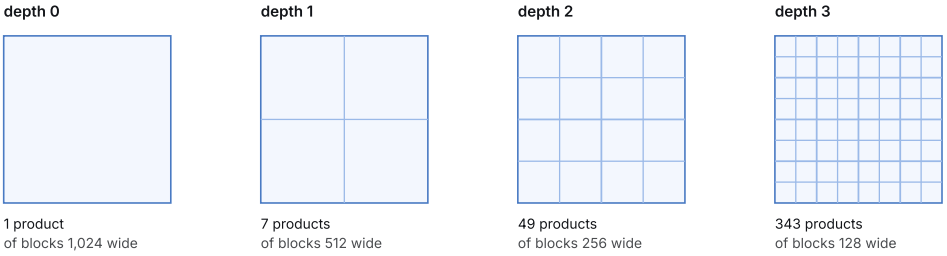
Cut each table into four blocks, so that A has blocks $A_{11}, A_{12}, A_{21}, A_{22}$ and B likewise (on this page A, B and C stand for any two tables and their product, unrelated to the legs and covariance of earlier pages). The schoolbook product needs eight products of blocks, since each block of the answer C is a sum of two. Strassen's needs seven, M_1 to M_7 in the picture. Each multiplies a sum of blocks of A by a sum of blocks of B; a dark square means the block is added, a light square with a minus that it is subtracted. M_1, for example, is $(A_{11} + A_{22})(B_{11} + B_{22})$. The four blocks of C are then sums and differences of the seven products (right). Expanded, every unwanted term cancels. Take the top-left block, which should be $C_{11} = A_{11}B_{11} + A_{12}B_{21}$. M_1 supplies $A_{11}B_{11}$ along with three unwanted terms; $M_4 = A_{22}(B_{21} - B_{11})$, $M_5 = (A_{11} + A_{12})B_{22}$ and $M_7 = (A_{12} - A_{22})(B_{21} + B_{22})$ supply $A_{12}B_{21}$ and cancel the rest, as the table at the bottom right shows.

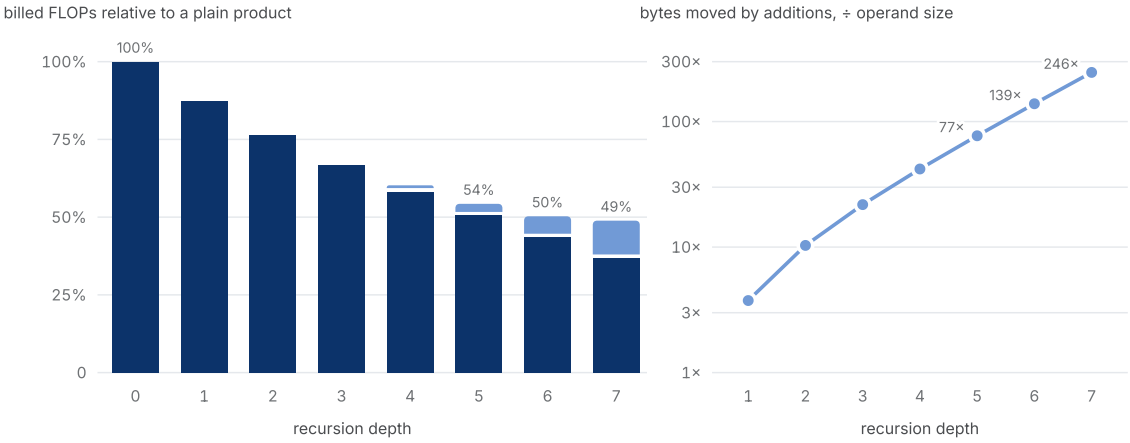
$$M_1 + M_4 - M_5 + M_7 \;=\; A_{11}B_{11} + A_{12}B_{21} \;=\; C_{11}$$

This is cheaper because multiplying two blocks of size $n/2$ costs about $2(n/2)^3$ FLOPs, while adding them costs $(n/2)^2$, one per element. Trading one block product for a handful of block additions saves almost an eighth of the work when the blocks are large.

Winograd's variant of the same idea arranges the sums so that 15 block additions suffice instead of Strassen's 18. The code we ship goes one step further. It rewrites both tables in another basis, after which each level needs only 12 block additions, and converts the answer back at the end, the alternative-basis method of Karstadt and Schwartz.

How deep to go





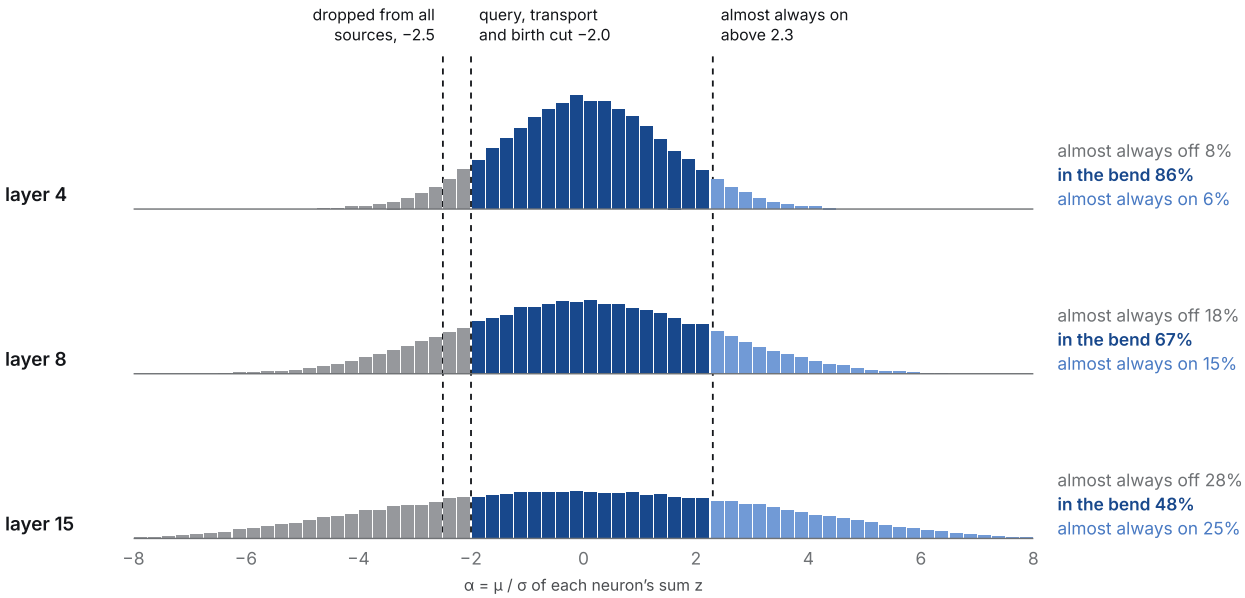
The trick applies again inside each of the seven products, which are themselves products of tables, and again inside those. The top picture shows the cutting. Each level cuts every block into four and replaces eight block products by seven, so at depth L one product becomes 7^L products of blocks 2^L times smaller. The number of levels is the *recursion depth*.

The lower picture shows the bill for one product of two 1,024 × 1,024 tables in Winograd's form, counted as the meter counts it; the alternative basis we ship bills somewhat fewer additions. On the left are the billed FLOPs relative to the schoolbook product, split into multiplications (dark) and additions (light). The dark part shrinks by about 7/8 per level, to 0.435 of the schoolbook FLOPs at six levels. The light part, the additions, grows with every level, to 0.068 at six levels (0.503 in all) and 0.120 at seven (0.489 in all); beyond the picture, at eight, it reaches 0.212 (0.513 in all) and the total turns back up.

On the right is a hidden cost. Every level's additions sweep the whole set of blocks through memory, and at depth 6 they move 139 times the size of the operands. The meter does not charge for moving data, but the clock does (page 44).

The estimator picks a depth for each shape of product, shallow for small or thin products and up to seven levels for some square ones, and runs the 7^L block products of a level in a few batched calls to the meter. Batching keeps the number of metered calls near 24,500 per network, which matters for the 0.4-second Python limit, since every call costs a little Python time. The tiers of page 46 are choices of this depth. The covariance products use another scheme of this kind, found by computer search, the flip-graph method of Khoruzhii, Gelss and Pokutta (flip-cpd, MIT licence).

Skip what is already settled



Page 18 sorted neurons into three kinds by $\alpha = \mu/\sigma$, almost always off, in the bend and almost always on. Only neurons in the bend make new shape, and a neuron that is almost always on passes its bell through what is, for it, nearly a straight line. Much of the work spent on the other two kinds changes nothing.

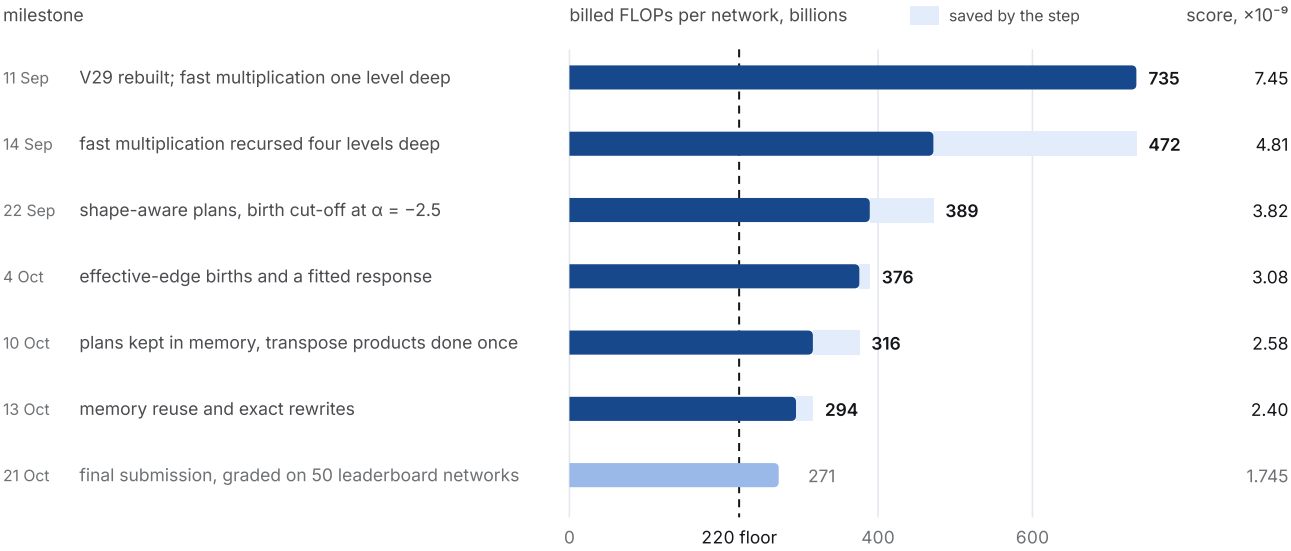
The picture shows α for all neurons of 64 fresh networks at layers 4, 8 and 15. The spread widens with depth, because a deep ReLU network maps different inputs to increasingly similar patterns, so each neuron varies less from input to input compared with how far its mean sits from zero. The settled share grows from 14% of neurons at layer 4 to 53% at layer 15.

The estimator draws lines, called *cuts*, dashed in the picture.

- below $\alpha = -2.0$ a neuron is left out of queries, transport and births, and below −2.5 the sources stop carrying it at all, its κ_3 diagonal then estimated by a small regression from quantities already at hand. At $\alpha = -2$ a neuron fires 2.3% of the time;
- above $\alpha = 2.3$ a neuron is negative only 1.1% of the time and makes almost no new shape, so whole blocks of such neurons are left out of the D21 queries, with a cheap correction in their place.

Layers 1 to 5 use slightly tighter cuts, which the picture leaves out. Each cut is an approximation, set by measurement. Moving the birth cut from −2.5 to −2.0 on 19 October saved 1.9% of the FLOPs and cost 0.8% in error, a net gain of 1.1% on 63 fresh networks. Lowering the almost-always-on cut to 2.0 went the other way, with 1.7% fewer FLOPs and 9.5% more error. The cuts save most where the network is deepest and the sources most numerous.

Iteratively reducing the FLOPs bill



In short sessions between 11 September and 13 October the bill fell from 735 to 294 billion FLOPs per network, by 60%, while the error held or improved. The picture shows each step. Every bar but the last is a complete run of the organisers' *harness*, the program that runs an estimator the way the grader does, on all 100 public networks; the last is the graded submission, which also carries the accuracy gains of pages 35 to 40. A change was kept only if every network passed every clock and the score improved.

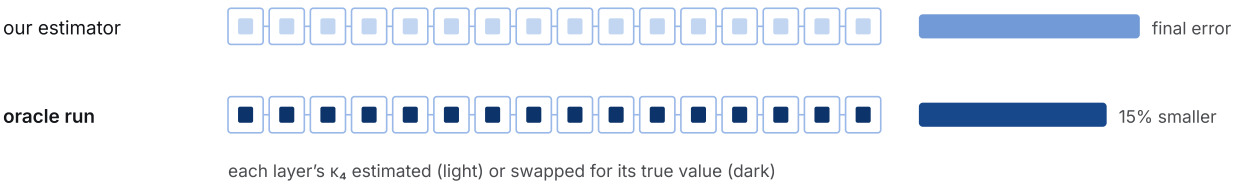
The largest step came first. Running the fast multiplication four levels deep instead of one cut the bill by 36% on 14 September. Multiplication plans for each shape (instructions prepared in advance), keeping track of which neurons are in the bend, and the first birth cut, at $\alpha = -2.5$, brought it to 389 billion by 22 September.

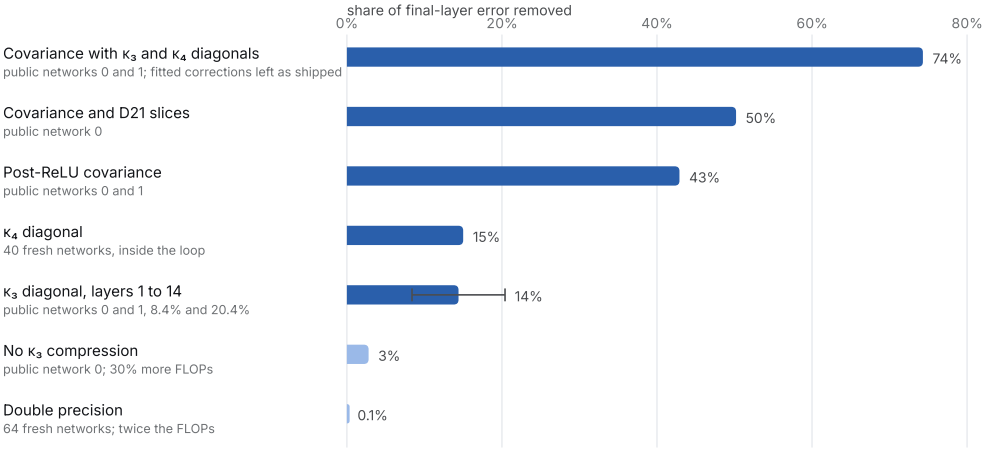
On 4 October two changes landed together. When a neuron bends, the lopsidedness its input already carries also feeds the new source. The original design carried that feedback as a separate thin source. The *effective-edge* birth, from our second line (page 53), folds it into the new source's leg A, whose entries, the links between a bent neuron and the others, become effective links that include that shape. The other change was a frozen fitted response, an early, fixed version of the deferred quartic response (page 39). Together they saved FLOPs and cut the error by a sixth; on eight matched networks the new leg alone cut it by 11%.

In October the work turned to the cost of each product, with plans kept in memory between calls, memory reused instead of reallocated, products of a matrix with its own transpose done once, and exact rewrites that removed duplicated work. Several promising candidates in between failed the test, most of them on the Python clock.

By 13 October the bill was 294 billion, a third above the floor, and the score 2.40×10^{-9}. Cutting much further needed deeper fast multiplication, which ran slower on the clock (page 44), so the error became the target.

Asking an oracle





Which of the estimator's approximations causes the error? An *oracle run* answers that (top picture). Pick one quantity the estimator computes, say each neuron's κ_4, and at every layer replace it by its true value, measured by sampling the same network many millions of times, while everything else runs as before. The share of the final error that disappears is what a perfect version of that quantity would be worth.

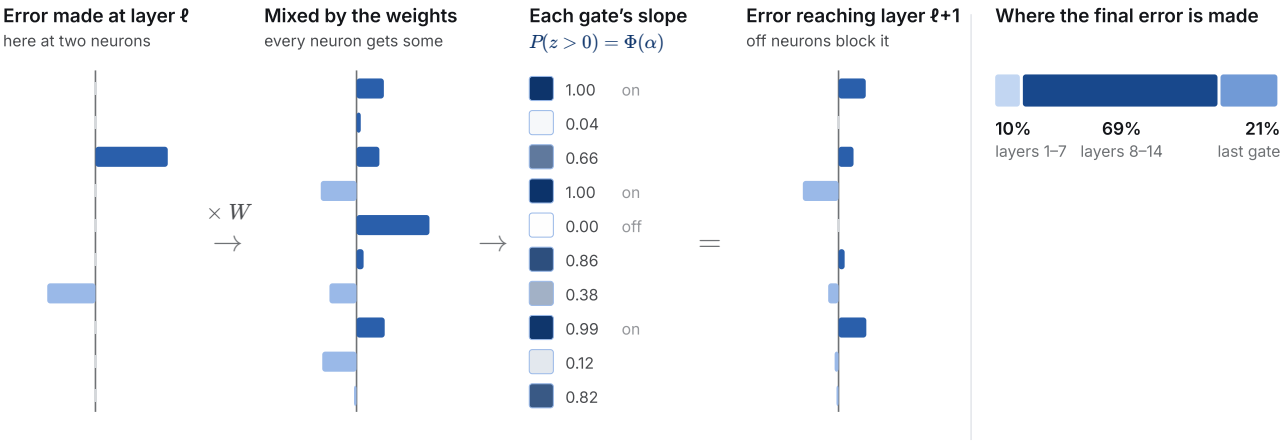
The lower picture lists the oracles we ran. Exact post-ReLU covariances at every layer would remove 43% of the error, exact D21 slices on top of them about half, and exact κ_3 and κ_4 diagonals on top of the covariances three quarters. The κ_4 diagonal alone, the least accurate quantity the estimator carries, is worth 15%.

Two suspected sources of error matter little. Keeping every source in full, instead of compressing the old ones, would remove 2.8% of the error for 30% more FLOPs. Computing with twice the digits (double precision) would remove 0.12% at twice the FLOPs, so rounding is not a meaningful source of error, though it jitters each network's error slightly (page 41).

The shares do not add up, because the parts overlap, and making one exact changes what the next is worth. Most oracles ran on one or two networks, so their numbers indicate orders of size. They show where the error is and say nothing about how to remove it.

Why not simply use the true values? Because they come from sampling the very network we are asked about, millions of runs of it, which is exactly what the budget cannot pay for.

How an error travels



We also asked where in the network the error is made. An error in one layer's means does not stay put; follow it in the picture. Say the estimator gets two neurons of layer ℓ slightly wrong (left). The next layer's sums mix every neuron through the weights, so the error spreads to all of them. Then, for a small error, each ReLU passes on a share equal to its slope, the firing share $\Phi(\alpha) = P(z > 0)$ of a bell from page 9, nearly all of it where a neuron almost always fires and almost none where it seldom does. What reaches layer $\ell + 1$ is the product

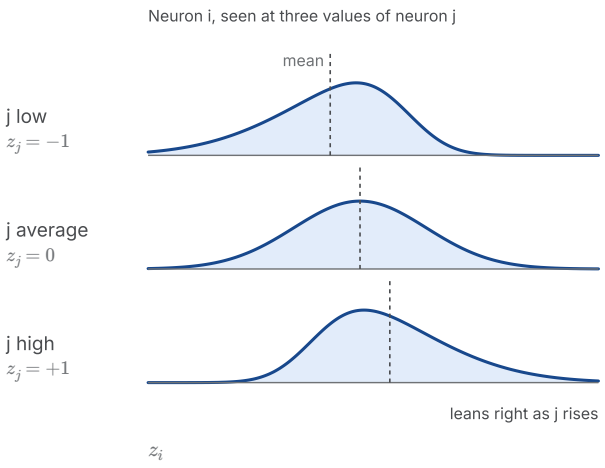
$$\delta_{\ell+1} \;=\; \Phi(\alpha) \circ (W\,\delta_\ell) \;+\; \text{new error made at layer } \ell + 1$$

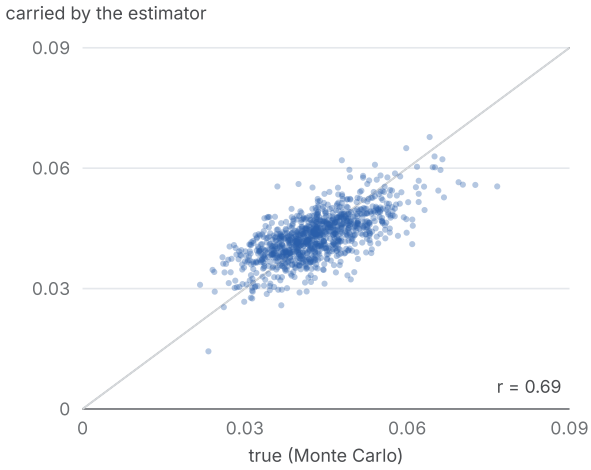
Repeated layer after layer, and to first order, the error that reaches the output is the sum of the errors made at every layer, each carried forward by the weights and slopes of the layers after it.

To measure where the error is made, we ran the estimator against the truth on 160 fresh networks, took the new error it made in the means at each layer, and carried each layer's new error to the output with the slopes and weights of the later layers, as in the formula. Traced this way (right of the picture), layers 1 to 7 account for 10% of the final error, layers 8 to 14 for 69%, and the last gate for 21%. This tracing is linear, with the slopes of bells, so it gives an approximate split of the error, useful as a diagnostic. The middle layers are where the departures from a bell have built up while many neurons are still in the bend. At the last gate, errors in the variance cost most, then the κ_4 diagonal, then the κ_3 diagonal, in the ratio 1.9 to 1.2 to 1.0. A formula we fitted on the last layer's own quantities predicted only 0.3% of that share, so a closer look at the last layer alone did not help.

So the work that followed aimed at the covariances and the κ_4 diagonal, above all in the middle layers, without adding much to the bill.

How one neuron's shape moves with another





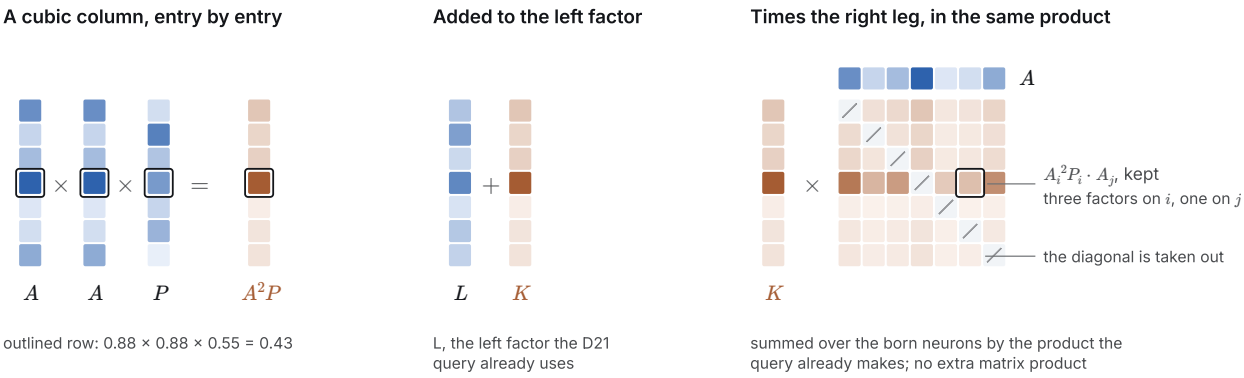
Our first additions changed the *physics*, our word for the mathematical model the estimator carries. The covariance update at each gate has fourth-order terms, and the most important are the (3,1) slices of page 14, $\kappa_4(i,i,i,j)$, which say how the lean of neuron i follows neuron j. The left picture shows a pair where two links act at once. On runs where j is low, average or high, i's bell shifts with j, which is the covariance, and also leans further right as j rises. As a rule of thumb, $\kappa_4(i,i,i,j)$ is j's variance times the average rate at which i's lopsidedness changes with j, exactly so when j is a bell and i's centre and variance change at most linearly in j, as in the picture. Oracle runs on two public networks put the exact (3,1) slices at about 21% and 12% of the error.

The participant kai_leal_gonzalez published a formula for these slices on the challenge forum, a closure in the sense of page 15. Beside a multiple of the covariance, set per neuron from V29's layer value and its own κ_4, we took from it the term $\kappa_3(i,i,i)\,\kappa_3(i,i,j)/\sigma_i^2$, i's lean times the pair's D21 link over i's variance, weighted by 0.75 as we calibrated. Together with a refit of the fitted response it improved the score by 1.3% on held-out public networks (ones not used to calibrate it), 50 of 58 of them better.

The second addition corrected the κ_4 diagonal that the inherited rule moves, with a *fitted* correction, a formula whose numbers were chosen to match measured examples. It reads 22 *features* of each neuron, quantities the estimator already computes, such as the rule's own κ_4, the neuron's κ_3 and sixteen fourth-order sums of the sources' legs, with separate numbers for each layer, 352 in all.

Together the two, with the response refitted to match, improved the score by 3.4% on 58 held-out public networks, 53 of them better, and by 3.9% on fresh networks. Submitted on 16 October, they scored 2.2×10^{-9}, down from 2.38 for our first upload the day before. The right picture shows what is still missing. In the final estimator the carried kurtosis of the layer-8 neurons of one network tracks the truth only loosely (correlation 0.69), and over 64 networks the median correlation through the middle layers is 0.54 to 0.68. Oracle runs put that gap at 15% of the error.

Fourth-order births, folded into an existing product



Fourth-order structure, like the third, is born at the gates. To leading order a bend makes it in four patterns of a source's legs, written A^3P, A^2P^2, AP^3 and P^4, each a product of four leg entries. The covariance update needs it in the (3,1) slices of page 14, where three of the four factors sit on neuron i and one on neuron j.

The fold builds these patterns inside a product the query already makes. To read D21, the query multiplies left factors L^A and L^P by the right legs A and P, a matrix product that sums over the born neurons. The fold adds to each left factor a few *cubic* columns, such as A^2P, whose entry i is $A_{ic}^2 P_{ic}$, made entry by entry from legs the query already holds and scaled by coefficients γ from a fitted rule. The product with the right leg then supplies the fourth factor,

$$\sum_c \left(L^A_{ic} + K^A_{ic}\right) A_{jc} \;+\; \left(L^P_{ic} + K^P_{ic}\right) P_{jc},$$

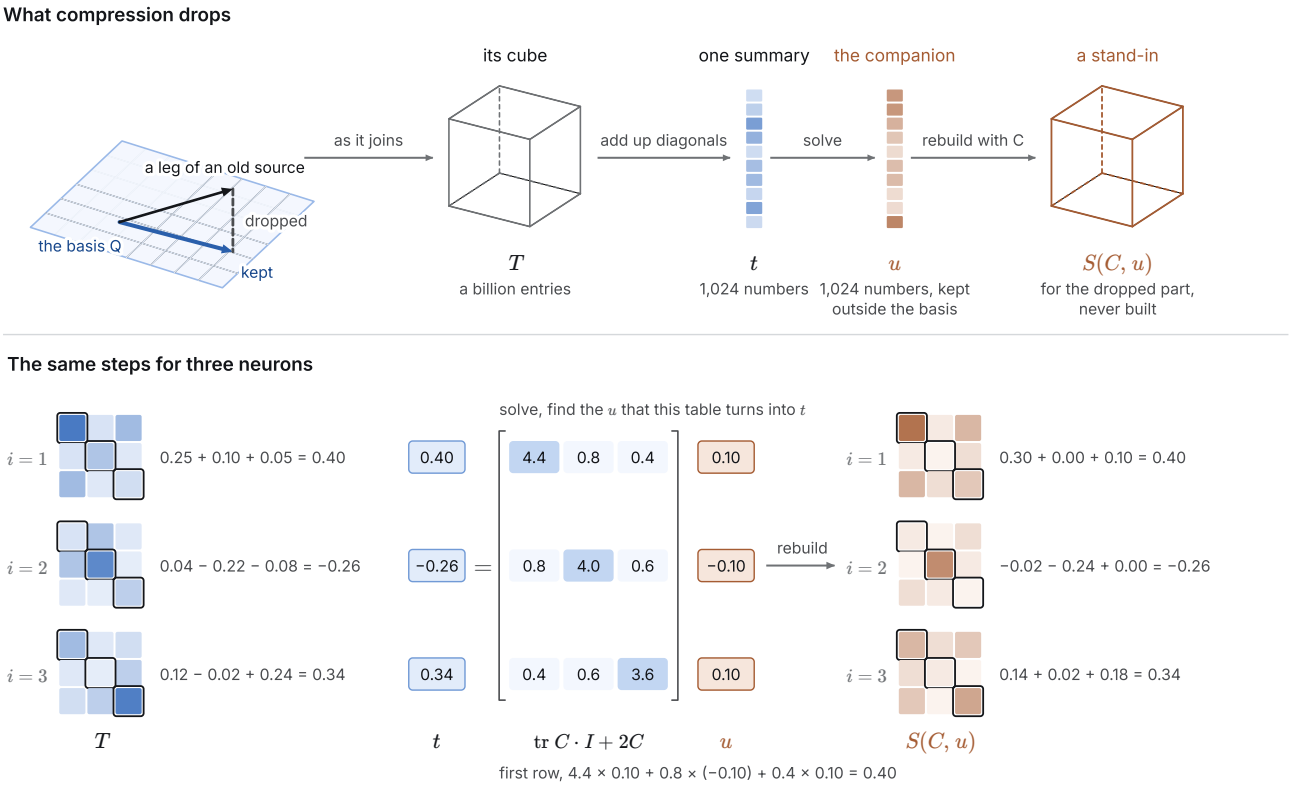
$$K^A = 3\gamma_0 A^2P + 3\gamma_1 AP^2 + \gamma_2 P^3, \qquad K^P = \gamma_0 A^3 + 3\gamma_1 A^2P + 3\gamma_2 AP^2 + \gamma_3 P^3,$$

with products of legs taken entry by entry. Where $j \neq i$, each added term has three leg factors on i and one on j, the young sources' share of the (3,1) slices, and it reaches the covariance update inside D21, scaled by $-\alpha_i/3\sigma_i$. Where $j = i$ the terms would come to $4\gamma_0 A^3P + 6\gamma_1 A^2P^2 + 4\gamma_2 AP^3 + \gamma_3 P^4$, the four patterns on the κ_4 diagonal, which the estimator carries by other means (page 35), so the code takes them out again. The picture follows one cubic column through these steps. The one product the query already makes delivers the fourth-order part together with the third-order values, without a single extra matrix product.

The coefficients come from a rule fitted on four public networks. It uses twelve features of the born neuron (its α, firing share, kink height at spread 1 and products of them, its own κ_3 and κ_4, and the sizes and overlap of its leg columns), each multiplied by 1, t and t^2 where t is the layer of birth, with separate coefficients for each of the four patterns and for each of the source's first three layers, 432 coefficients (458 values in its file). On the 100 public networks the first build with the fold improved the score by 3.0%, for 0.4% more FLOPs, all of it element by element.

With the graph head of page 39 it reached the grader on 17 October, and after a failed smoke test and a failed setup, the third upload that day scored 2.01×10^{-9}.

A companion for what compression drops



Outlined, the diagonal of each slice. Its sum is the same in both cubes, although the entries differ.

Compression costs about 3% of the error (page 33). That is small, but by October every percent was hard-won, and part of it can be recovered for almost no compute. When a source grows old, each of its leg columns is replaced by its shadow on the shared basis and the part sticking out is dropped (page 25, top left of the picture), and with it part of the source's cube T. The *companion* brings some of it back with one extra list of 1,024 numbers per network, u.

First, summarise the joining source's cube by one list, its *partial trace* t. For each neuron i, add up the entries T_{iij} along the diagonal of i's slice, as the trace of a table adds up its diagonal; the legs give this list directly, without the cube. Then look for a cube that is cheap to describe and has the same summary. The cube built from the covariance and one list,

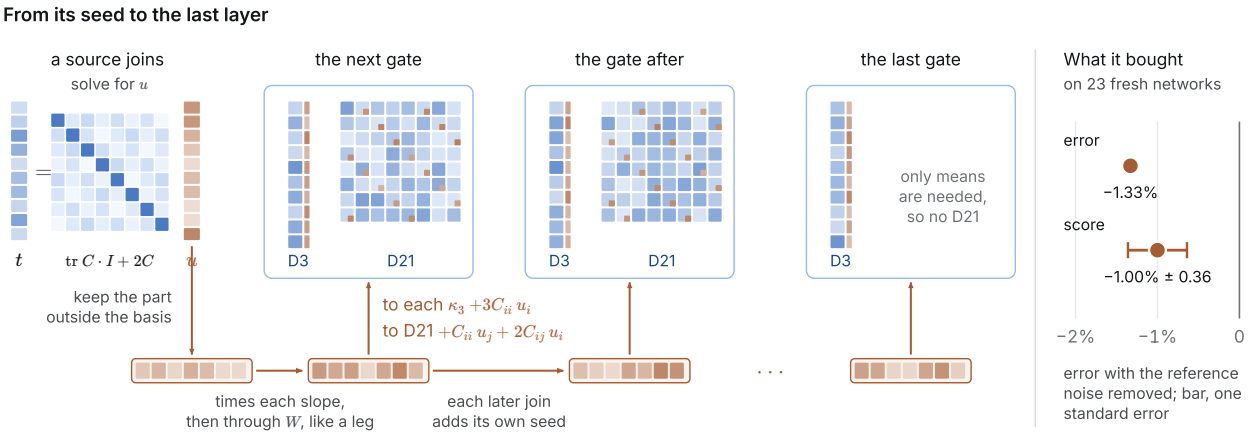
$$S(C,u)_{ijk} \;=\; C_{ij}\,u_k + C_{ik}\,u_j + C_{jk}\,u_i,$$

has partial trace $(\operatorname{tr} C \cdot I + 2C)\,u$, where $\operatorname{tr} C$ is the sum of the variances and I the identity table (ones on the diagonal, zeros elsewhere). Matching the summaries therefore means solving

$$(\operatorname{tr} C \cdot I + 2C)\,u \;=\; t.$$

Solving means finding the one list u that this table turns into t, one equation per neuron, as at the bottom of the picture. The two cubes differ entry by entry, yet every slice's diagonal adds up to the same total. The estimator finds u approximately, from a starting guess and six rounds of matrix–vector work, and keeps only its part outside the basis, which stands in for what the basis drops.

Where the companion helps



Each gate reads only the pieces of $S(C, u)$ it needs; the cube itself is never built.

Once born at a join, the companion lives like a leg (picture). At every gate each neuron's entry is multiplied by its slope, and the list is carried through the weights, $u \leftarrow W(s \circ u)$, the rule of page 23. Each later join adds a seed of its own, so one list serves every old source.

At every later gate the companion is read like a source. S itself is never built; the gate takes from it only the pieces it needs, $3C_{ii}u_i$ for each neuron's κ_3, which is the diagonal of S, and $C_{ii}u_j + 2C_{ij}u_i$ for the D21 slices. Because S is tied to the full covariance, it reaches every pair of neurons for the cost of matrix–vector work. Birth terms are re-matched so that structure a gate renews is not counted twice, and the last gate, which computes only means, takes the κ_3 part alone.

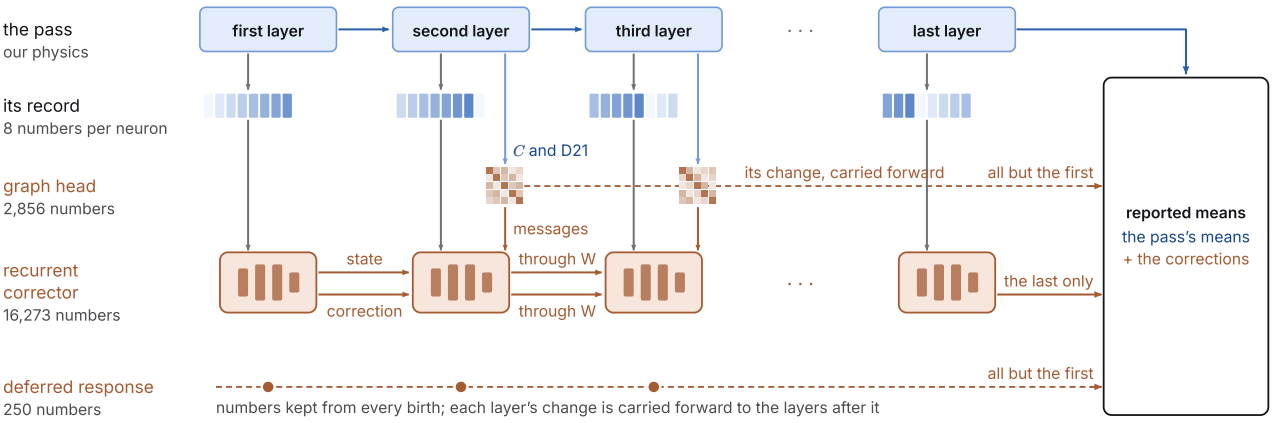
One list cannot describe the lost cube; it is enough here only because the lost part is small. Tested as a component on 23 fresh networks, it lowered the error by 1.33% once the reference answers' own noise is removed, and the score by 1.00% ± 0.36 (one standard error, the uncertainty of the average; right of the picture). A second list added almost nothing, and retiring old sources entirely into companions made the error twelve times larger. The algebra was first checked by 417 explicit tests at small width. It shipped on 20 October together with the −2.0 birth cut and the recurrent corrector of page 40, and that upload scored 1.83×10^{-9}.

The solve itself is cheap. Starting from $u = 0.85\, t/\operatorname{tr} C$, the estimator repeats six times

$$u \;\leftarrow\; u + 0.85\left(\frac{t - 2Cu}{\operatorname{tr} C} - u\right),$$

each round one product of C with a list, about two million FLOPs, against 271 billion for a whole network. Six rounds do not solve the equation exactly, and the gain measured above is that of this approximate solution.

Corrections learned from data



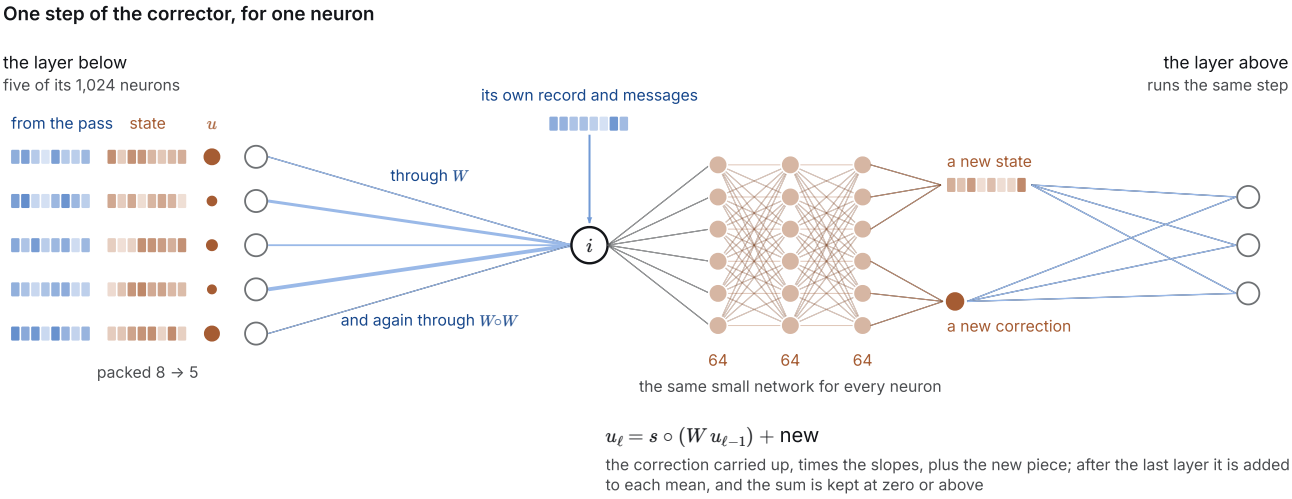
Clay marks the fitted parts. The pass itself never changes; only the means it reports do.

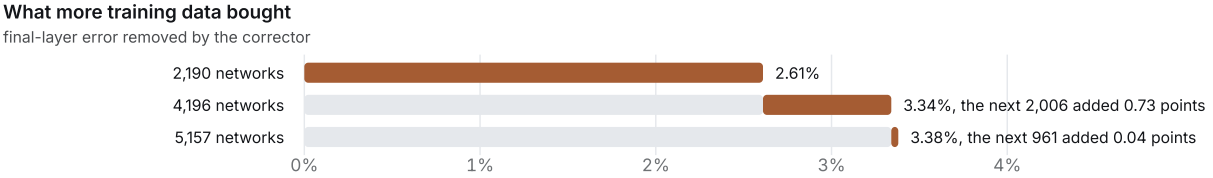
What is left after the physics can be learned from data. Three fitted corrections ride along the pass (picture). They read what the pass writes down at every layer, its *record*, eight numbers per neuron, the mean, variance, κ_3 and κ_4 of its input, its α and σ, the height at the kink of its bell scaled to spread 1, and its firing share. They change only the means the estimator reports; the moments used inside the pass are never touched. On 64 fresh networks they remove 9% of the final-layer error.

- The **deferred quartic response**, 250 fitted numbers (303 in its file). At every birth the pass keeps a few numbers per bent neuron, and at each later layer it uses them to take four fourth-order sums of that source's legs as they are then, structure the moments it carries leave out. After the pass a fitted rule turns these sums, with each neuron's own α, κ_3 and κ_4, into a small change of its mean, and each change is carried to the later layers through the weights, scaled by the slopes, as an error travels (page 34). It is *deferred* because it acts after the pass and *quartic* because its sums are of fourth order in the legs.
- The **graph forcing head**, 2,856 fitted numbers (3,264 in its file; a *head* is a small fitted add-on). It treats the neurons of a layer as a *graph*, linked by their correlations, their D21 slices and products of the two. Along every link each neuron sends a *message*, a few numbers made from its own α, κ_3 and κ_4, and each neuron turns what it receives into a change of its mean. A *forcing* is a term that drives a recurrence from outside; these changes are added at every layer but the first and the last and carried forward like an error.
- The **recurrent corrector**, 16,273 numbers in use (19,145 in its file), a small neural network of its own (next page). It reads the record and the messages, carries a state of its own from layer to layer, and corrects the means of the last layer.

The three were fitted together, so each relies on the others. Switching off the quartic response alone raises the error by 53%, because the corrector downstream reads its output. They also learned the estimator's own old errors, which made later improvements to the physics harder to add (page 51).

A small network reads each pass and corrects its answer





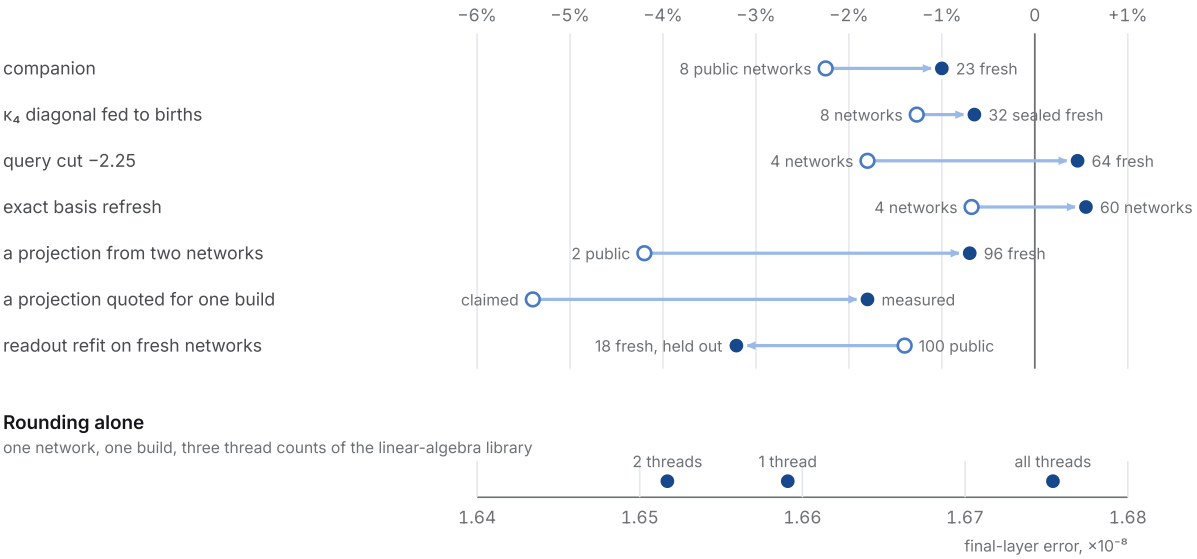
The recurrent corrector is the largest of the three corrections and the only neural network. *Recurrent* means that the same small step, with only a bias and a few scales set per layer, runs at every layer and hands something on to the next. The picture shows one step for one neuron; all 1,024 run it at once, with the same weights.

Each step reads four kinds of input. The first is the neuron's own record and what the other corrections made of it, 36 numbers including the graph head's messages and the neuron's *reach*, how strongly a change of its mean would move the last layer, found by running the weights backwards. The second is twelve numbers of the layer below, carried through W and through $W \circ W$. The weights carry signed amounts and the squared weights sizes, since independent errors add through their squares. The last two are what the corrector itself carries, an eight-number *state* per neuron, packed into five before it moves through W, and the correction so far.

A network of three layers of 64 turns these into a new state and a new piece of correction, scaled for the layer. The correction travels like the error of page 34, multiplied by the slopes, corrected for shape, and carried through the weights, so the corrector builds it up over all sixteen layers and adds it to the means only at the end, where each neuron's final mean is the larger of zero and its corrected value.

Its design follows the corrector Susupro published on the challenge forum (MIT licence). It was fitted in advance on about 5,000 networks with sampled true means, most of them from Keenan Pepper's public collection. Its gain saturated with data (bottom of the picture). Its shipped weights come from brief further training on records of the estimator's own passes, companion included; longer training on the original collection did not help (page 51).

Measuring a one-percent change



Rounding alone

one network, one build, three thread counts of the linear-algebra library

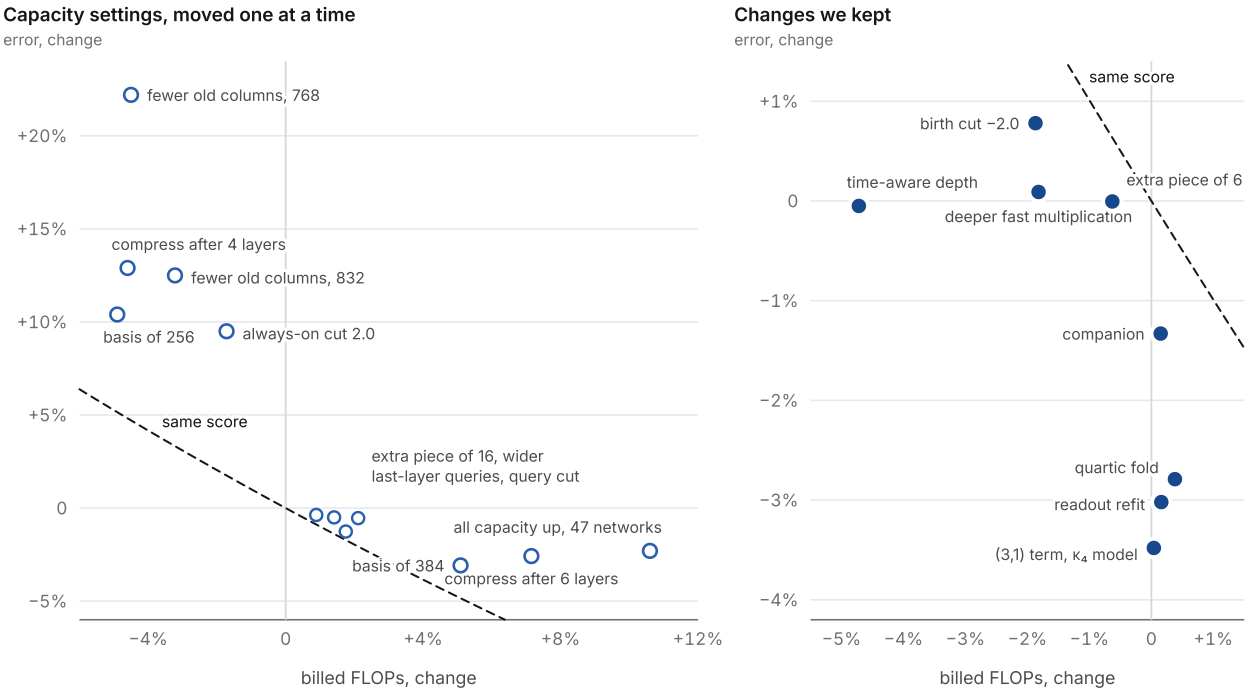
Most late changes were worth about 1%. That is less than the differences between networks, and measuring it took attentive care.

Rounding alone moves a network's error. With everything else fixed, changing how many processor threads the linear-algebra library uses moved one network's final error by 1.4% (bottom of the picture), and the noise per network is about ±0.5%. Every comparison was therefore *paired*, the same networks measured with and without the change, and a claim below 1% needs 40 or more paired networks.

The public networks had a second problem, since parts of the estimator had been fitted on them, which flatters any change measured there, and choosing the best of several candidates on a small panel flatters it again, because the best of several noisy results is usually a lucky one. From 18 October every decision used fresh networks, with true means sampled 2^{25} or 2^{27} times on a graphics card, and one *panel*, a fixed set of test networks, was kept sealed from the agent building the candidates, so that its results could not steer the building. The picture shows why, with first estimates on a few networks (open circles) against later measurements on more or fresh ones (filled). Several shrank or changed sign. The first decision made the new way, a refit of the fitted corrections (the readout) on fresh networks, was uploaded on 18 October and scored 1.93×10^{-9}.

Forecasts from two networks overstated gains by about 3.5%, and the slow first network of each process, a missing precomputed multiplication template or a busy machine could move a measurement by more than the change being tested. And a build can pass every clock and still be wrong. One candidate on the last day passed the official harness with an error 17 times too large, from an array left over from an earlier run, and only the accuracy check caught it.

Every setting sits at its knee



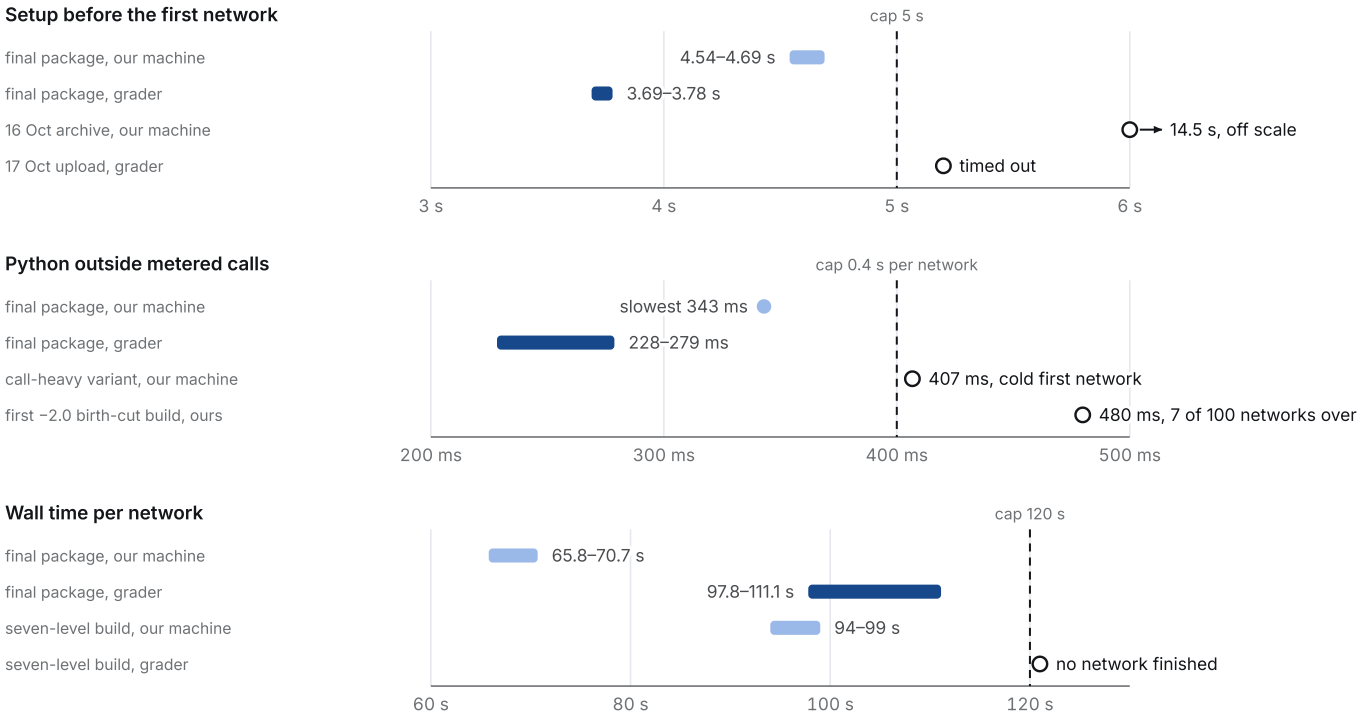
The corrector's gain had saturated with data (page 40). Why not buy accuracy with compute instead, with a larger basis, more young sources, a wider query? Because every setting already sat at its knee. Moving any one of them away from the shipped value lost score.

Read the picture like this. Each point is one change, placed by its effect on FLOPs (across) and on error (up), both in percent, with the reference answers' own noise removed and on the panel used when the change was decided, so figures quoted on other pages can differ slightly. The score is the product of the two, so the dashed curve marks changes that leave the score as it was, and points below it improve the score. A point at (+5%, −3%) costs 5% more FLOPs and removes 3% of the error; it lies above the curve and loses.

Shrinking a setting is costly. Compressing after four layers instead of five, or shrinking the basis to 256 directions, saved about 5% of the FLOPs and cost 9 to 13% in error; keeping fewer columns for old sources cost four to five points of error for each point of FLOPs saved. Enlarging a setting pays little. Compressing after six layers, a basis of 384, a larger extra piece or wider queries at the last layer each removed less error than they added in FLOPs; all of them together, on 47 fresh networks, cut the error by 2.3% for 10.6% more FLOPs.

The changes we kept lie close to the axes (right). They are savings that left the error unchanged, such as the deeper multiplication and the time-aware depth (page 47), and accuracy gains at almost no cost, such as the quartic fold, the readout refit and the companion. So the late gains had to come from new structure, or from the same arithmetic for fewer FLOPs.

Three limits, met one after another

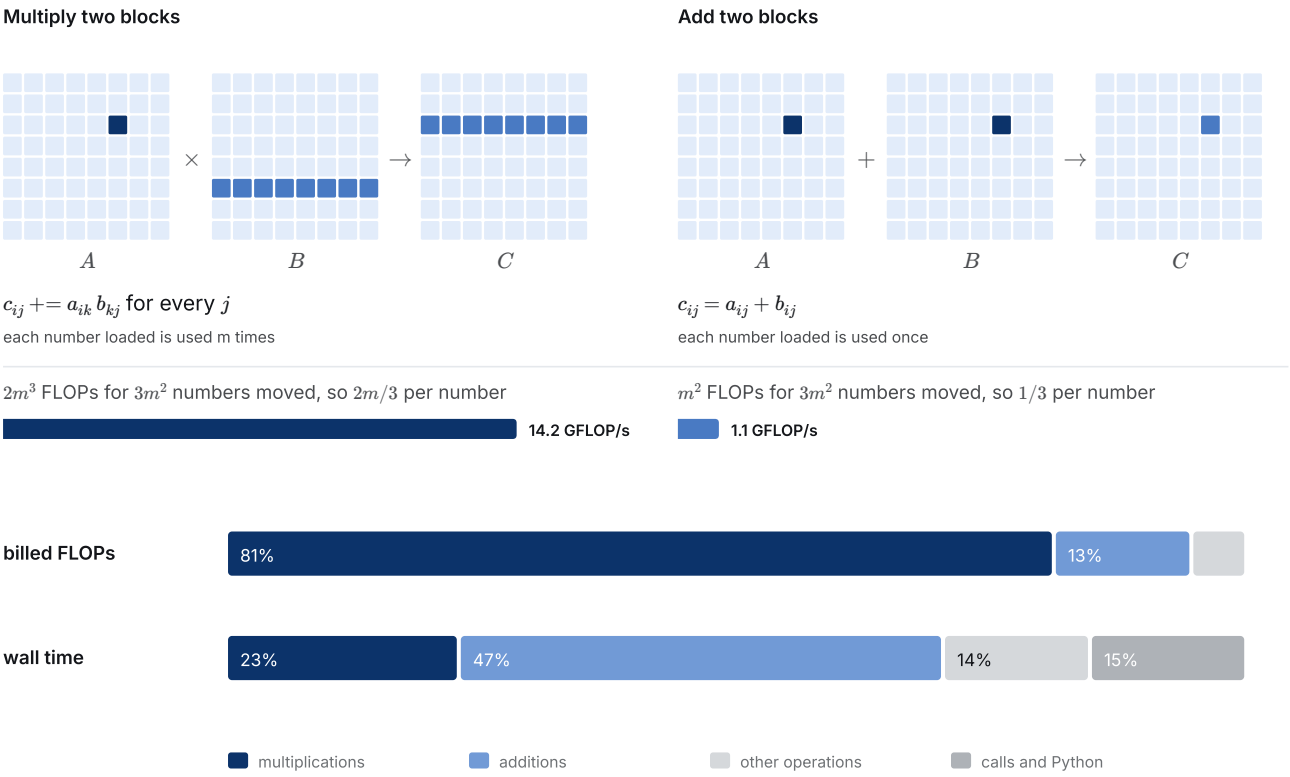


Three clocks run beside the meter (page 6), and each stopped one of our builds, one after another; none could be judged on our machine alone. On the organisers' harness the estimator runs as a client that sends each metered operation to a separate server process, which does the arithmetic and counts it. The picture shows measured values against each limit, in the order we met them, with light marks from our machine, dark marks from the grader's records, open circles where a limit broke.

- **Setup.** The 5-second setup includes preparing the multiplication plans, which takes many client–server round trips, about 1.2 ms each. Our 16 October archive took about 2 seconds in our own test runner and 14.5 seconds on the organisers' *stock harness* (their unmodified client–server runner), and an upload on 17 October failed setup on the grader. Since then every candidate has been checked on the stock harness.
- **Python time.** Each metered call costs a little Python time in the client, and only 0.4 seconds per network may be spent outside the metered work. Our first build with the −2.0 birth cut failed it on 7 of 100 networks, because uneven numbers of births per layer had knocked out an optimised route in the code. An exact rewrite fixed it. The final package makes about 24,500 calls; its slowest network spent 343 ms locally at normal speed and 279 ms on the grader.
- **Wall time**, the real seconds a network takes. Our deepest engine, at a fixed depth, took 94 to 99 seconds per network locally. On the grader its first networks ran out of time about three quarters of the way through, and the run was stopped.

All three taught the same lesson. Our machine could measure how much work a build does, but only the grader could show how close that work came to a limit. The wall clock took the longest to understand, and the pages that follow are about it.

The meter and the clock disagree

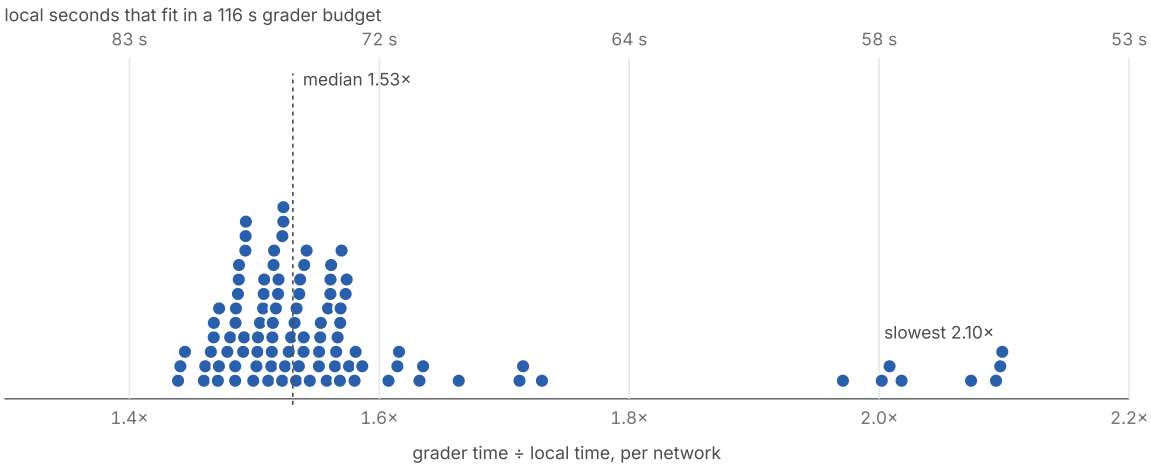


The meter counts FLOPs; the clock counts seconds; and the two are not proportional. The top picture shows why. To multiply two blocks of $m \times m$ numbers, each number loaded from memory meets a whole row of the other block, so one number of A is used m times. Adding two blocks uses each number once. Both move three blocks through memory, but the product does $2m/3$ FLOPs per number moved and the sum only 1/3. A processor calculates much faster than it can fetch numbers, so additions spend most of their time waiting.

On the organisers' harness, on one processor core, matrix products ran at 14 GFLOP/s (billion FLOPs per second) and the additions of the fast multiplication at 1.1. The lower picture shows what that means for one network's *wall time*, the real seconds that pass. Products are 81% of the FLOPs and 23% of the time; additions and subtractions are 13% of the FLOPs and 47% of the time; another 15% is spent passing some 24,500 metered calls between the organisers' client and server (page 43).

So fast multiplication trades fast FLOPs for slow ones. Our deepest multiplication engine, the seven-level build of 21 October, saved about 5% of the FLOPs and nearly doubled the time spent in arithmetic on our machine, from 45 to 85 seconds; going from six levels to seven saved only 0.24 to 0.68 billion FLOPs per extra second. Layout mattered as much. A seven-level product reading numbers scattered through memory with gaps (a strided view) ran at 0.36 GFLOP/s, the same product with its numbers side by side at 2.1. From here on every speed-up was about memory traffic or the number of calls.

The grader's clock



The limit is 120 seconds per network, measured on the grader's machines, which are slower than ours by an amount we could not know in advance. The picture shows the grader's time for each of the 100 networks of our upload of 20 October, a build with a fixed depth, divided by our own time for that build, 55 seconds per network (58 for a worker's first). The ratio lies between 1.44 and 2.10.

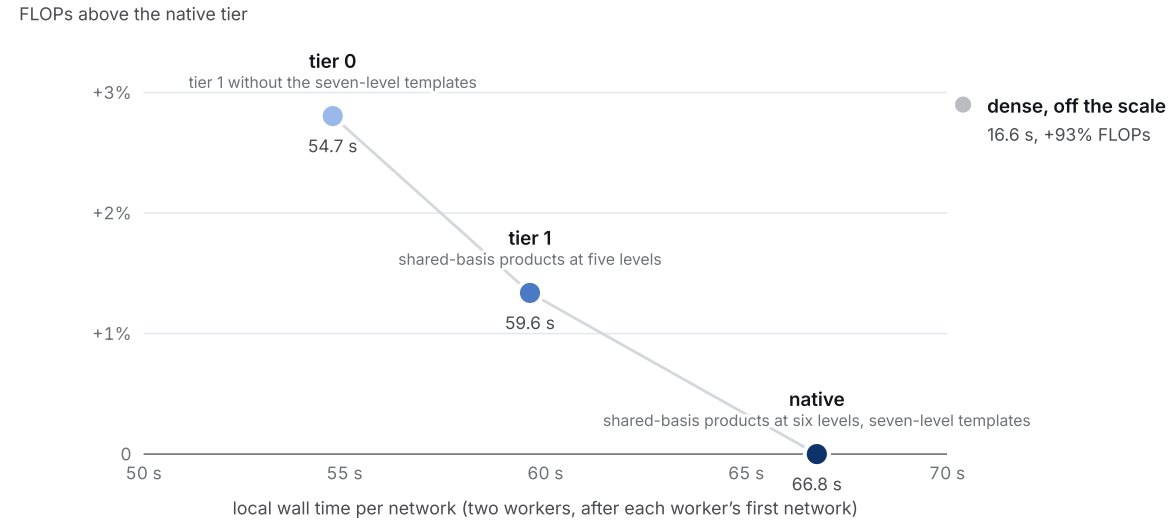
The grader runs five *workers*, separate processes that each take 20 networks in turn. Most networks ran 1.44 to 1.70 times slower than on our machine, with a median of 1.53; 11 ran more than 1.7 times slower. The slow ones were not the hard networks. They used ordinary FLOPs and landed on busy workers.

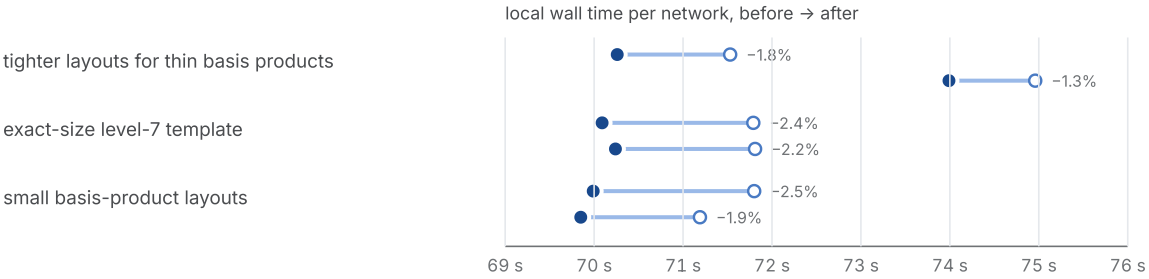
For a build with a fixed recursion depth, the slowest network sets the budget. Surviving a 2.1-fold slowdown inside 120 seconds allows at most 57 seconds of total time on our machine; keeping 4 seconds in hand, as the scale at the top of the picture does, leaves 55. The build in the picture sat at that edge. Our deeper engines needed 67 to 99 seconds. The deepest, a seven-level variant of the full estimator, needed 94 to 99 seconds and forecast 1.734×10^{-9} on fresh networks; submitted on 21 October, it ran out of time on the grader.

From the grader's records in the picture we fitted a simple rule, under which the slowest network takes about 2.4 times our local arithmetic time plus 9 seconds, so a fixed build is safe only with at most about 42 seconds of local arithmetic (time inside the metered operations). Setup and Python time, by contrast, ran faster on the grader than on our machine. Each worker's first network is also slower, because multiplication plans are prepared on first use.

So the deep engine could not be used at a fixed depth, however good its score. The depth could no longer be fixed before seeing how fast the grader is.

Four tiers used for the same one model





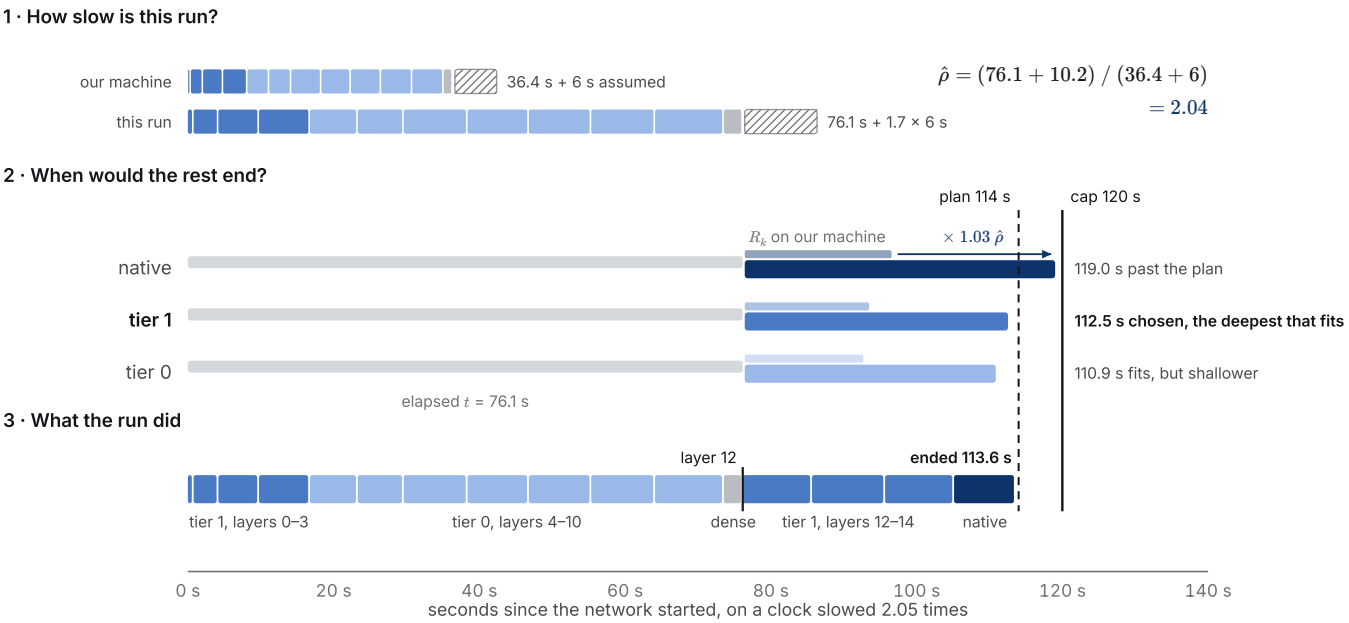
Every recursion depth computes the same model on the same state; the depths differ only in the order of the arithmetic, and so in rounding. So the depth can be chosen while a network runs, layer by layer, from a menu. We built four *tiers*.

- **native**, the estimator's own and deepest setting, with shared-basis products at six levels and seven-level templates (prepared in advance) for square shapes;
- **tier 1**, with shared-basis products at five levels and the seven-level templates kept, the policy of an earlier, faster build;
- **tier 0**, tier 1 with the seven-level products switched off;
- **dense**, plain metered products without fast multiplication, for emergencies.

The top picture shows the trade on our machine. Falling back from native to tier 0 saves 12 seconds per network for 2.8% more FLOPs; dense is three times faster still and costs 93% more.

Before the controller, which picks a tier for each layer (next page), could use the native tier, it had to get cheaper in time. Several seven-level products were padded out to larger templates and read through strided views; exact-size templates and tighter layouts cut its wall time by about 6% with identical results (lower picture; work of argon_2 and Codex). The fallback tiers needed six extra five-level layouts at setup, which raised setup to 4.5 to 4.7 seconds locally and 3.7 to 3.8 on the grader.

Choosing the depth from the clock



Before each layer the estimator asks one question. What is the deepest tier that still finishes this network in time? The picture follows one real decision before layer 12 in a *rehearsal*, a local run on a clock slowed 2.05 times (page 48).

1. How slow is this machine? The first twelve layers took t = 76.1 s here and t_{ref} = 36.4 s on our machine at the same tiers, from a table of local times measured in advance. We start by assuming a slowdown of 1.7, as if six seconds had already been measured at that rate (hatched), and let the measured time take over; $\hat{\rho}$ is the estimated slowdown.

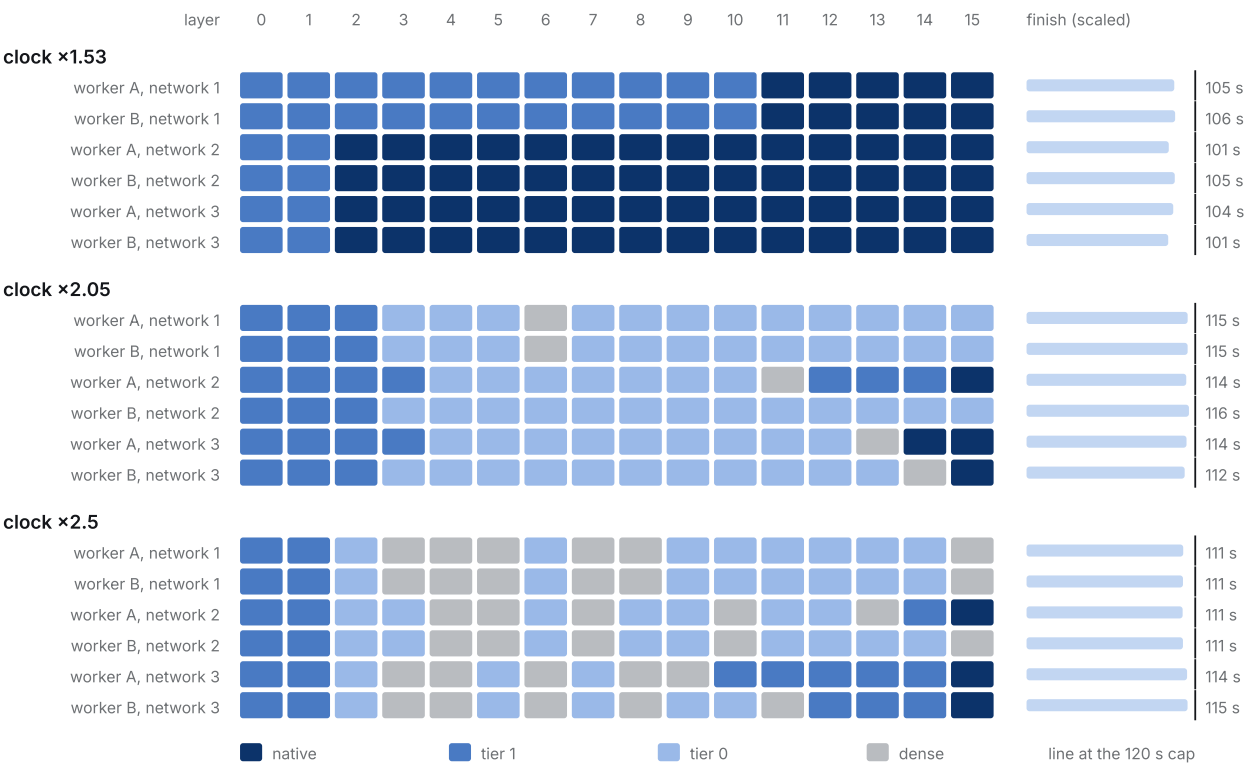
$$\hat{\rho} \;=\; \max\left(1,\; \frac{t + 1.7 \cdot 6\,\text{s}}{t_{\text{ref}} + 6\,\text{s}}\right) \;=\; \frac{76.1 + 10.2}{36.4 + 6} \;=\; 2.04, \qquad t + 1.03\,\hat{\rho}\,R_k \;\le\; 114\,\text{s}$$

2. When would the rest end? Each tier's local time for layers 12 to 15, R_k (thin bars), is stretched by $1.03\ \hat{\rho}$, the estimated slowdown plus a 3% margin, and added to t. Native would end at 119.0 s, past the 114-second plan; tier 1 at 112.5 s, inside it. Tier 1 also passes a safety check (this layer at tier 1 and the rest at tier 0, stretched by $1.12\ \hat{\rho}$, must end by 118 s), so it is chosen. If no tier passes, tier 0 runs if it ends by 115.5 s without the 3% margin; otherwise the layer runs dense.

3. The whole run. Native never fitted early on, so the first layers ran at tier 1; once the slowdown was clear, only tier 0 fitted; at layer 11 not even tier 0 would end in time, so that layer ran dense, three times faster. The time saved bought tier 1 for layers 12 to 14 and native for the last. The network ended at 113.6 s.

The clock steers only the choice of tier, so a slow grader costs FLOPs and leaves the accuracy unchanged apart from rounding. A worker's first network uses its own, slower table of local times. The constants came from replays of the grader's recorded times (page 45); a rule found by exhaustive planning over the same tables (dynamic programming) did no better.

Rehearsing slow graders



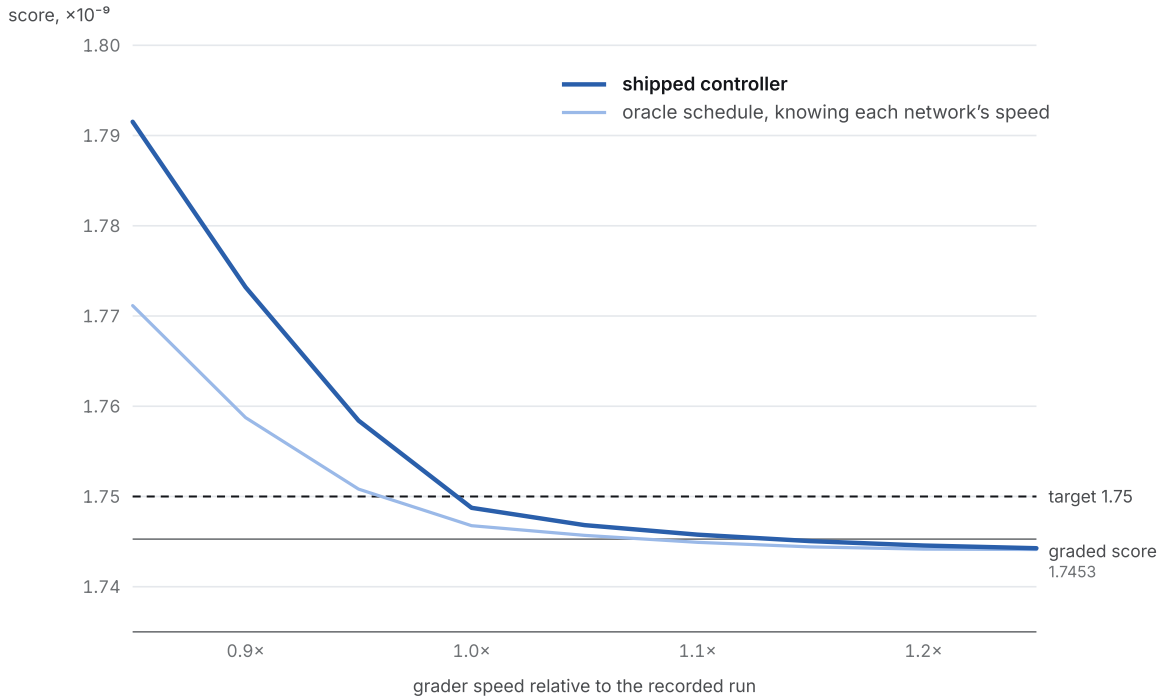
We could not try the controller on the grader before submitting, so we rehearsed. We ran the shipped package on the organisers' harness with its clock multiplied by 1.53, 2.05 and 2.5, a model of slower machines. The picture shows the tier chosen at every layer of three networks on each of two workers, and when each finished.

At 1.53, the median slowdown seen on the grader, the controller runs early layers at tier 1 while its estimate is still uncertain, then settles on the native tier. At 2.05 it runs tier 0 for most of the network and drops single layers to dense when a layer runs long. At 2.5, slower than any network we had seen, about a third of the layers run dense, and every network still finishes between 111 and 115 seconds.

Scaling the clock tests the rule under an assumed slowdown; it does not measure a slow machine.

The same harness confirmed the package at normal speed, where 10 of 10 networks passed, with wall times of 65.8 to 70.7 seconds, at most 343 ms of Python per network and 4.54 to 4.69 seconds of setup. On a fresh network the native tier's answers were identical, bit for bit, to those of the build it extends. The scaled runs also found the tightest case for the Python limit, a worker's first network that spent eleven layers at tier 1 and used 382 ms of the 400 allowed. On the grader, Python ran about 25% faster than locally.

What we expected



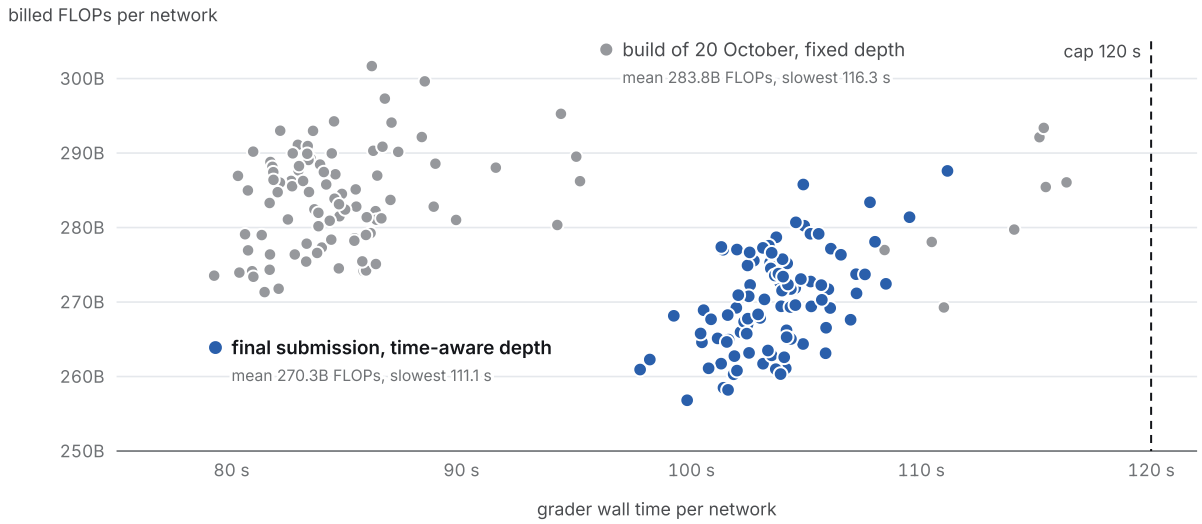
We simulated the controller to predict its score. Each simulated network starts from the measured time of every layer at every tier on the organisers' harness and the exact FLOPs of every layer at every tier, measured on a fresh network. Its slowdown comes from the grader's recorded times of page 45, and 10% random noise is added to every layer. The score is the fresh-network forecast for the native tier, 1.7442×10^{-9}, scaled by the FLOPs of the tiers chosen.

At the recorded speeds the controller scored about 1.749×10^{-9}, within 0.2% of an oracle schedule that knew every network's speed in advance. The slowest simulated network finished in 115.9 seconds; with arithmetic 20% slower and 20% noise per layer, in 117.1 seconds.

The picture shows how steeply the score depends on the grader's speed. With the grader 10% slower the expected score passes 1.77; 10% faster and it approaches the native tier's 1.744. The native tier at a fixed depth would pass only if every network ran at least 17% faster than in the recorded run.

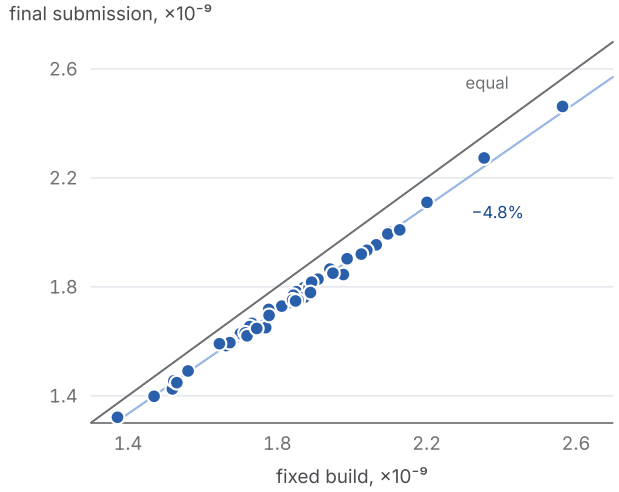
Codex replayed the controller independently before upload and obtained 1.7494×10^{-9}. A cheaper alternative that fell back only on the largest products simulated 1.757 to 1.760, with 7% of layers dense, and was rejected.

What the grader saw



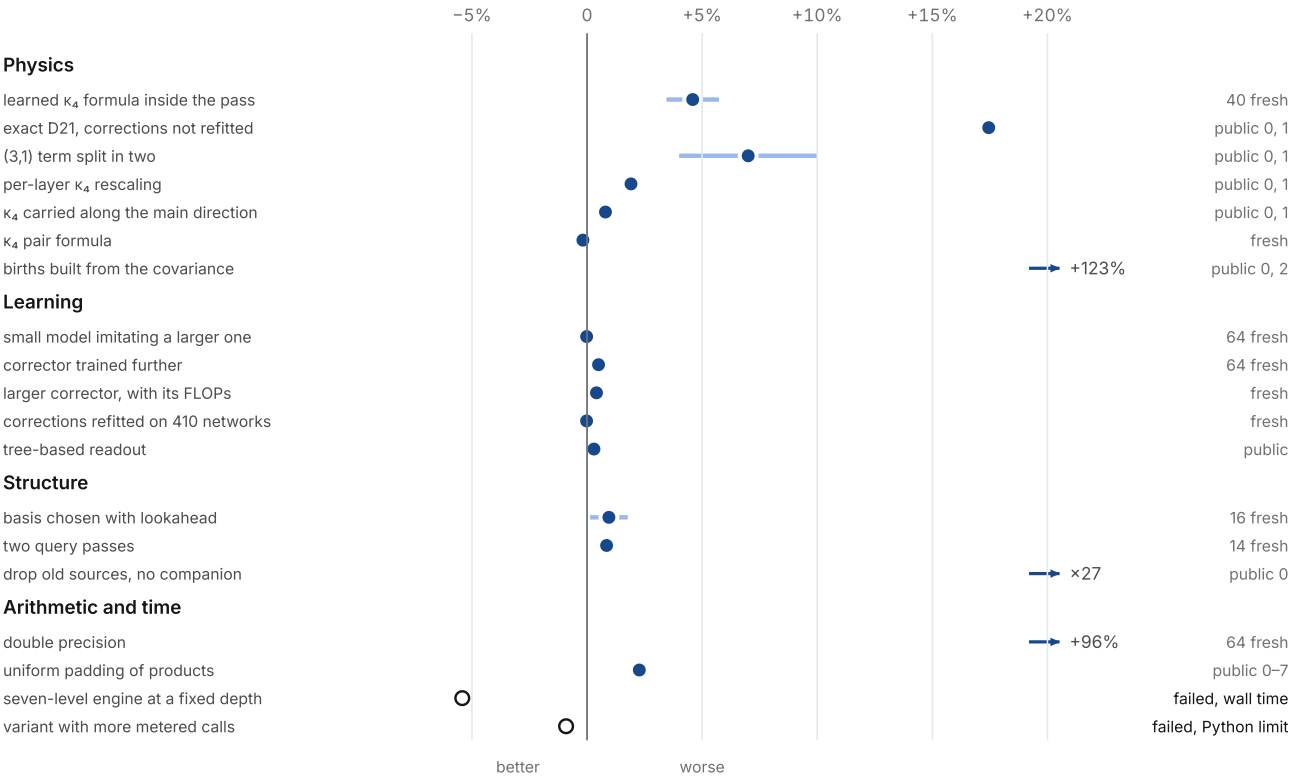
The final submission was graded on 21 October. The top picture shows every one of its 100 networks, by time and FLOPs, against those of the fixed build before it, uploaded on 20 October. The fixed build spread from 79 to 116 seconds, so its slowest network came within 4 seconds of the limit. The time-aware build ran the native tier wherever it fitted, and its slowest network took 111 seconds.

Mean FLOPs on the 50 leaderboard networks fell from 284.6 to 271.2 billion (283.8 and 270.3 over all 100, as in the top picture), and the compute factor from 0.1294 to 0.1233. The error was 1.415×10^{-8} against 1.416×10^{-8}. The picture below shows each leaderboard network under both, and all 50 improved.



Python time peaked at 279 ms against the 400 ms limit, and setup at 3.78 seconds against 5. The score was 1.7453×10^{-9}, 4.8% below that of the fixed build, close to the fast end of the simulation. The graded FLOPs, 4.7% lower and close to the 4.8% forecast for the native tier everywhere, show that almost every layer ran at the native tier. The two packages also carry different trained versions (checkpoints) of the recurrent corrector, so the comparison is between two packages that differ in more than one way.

What did not work



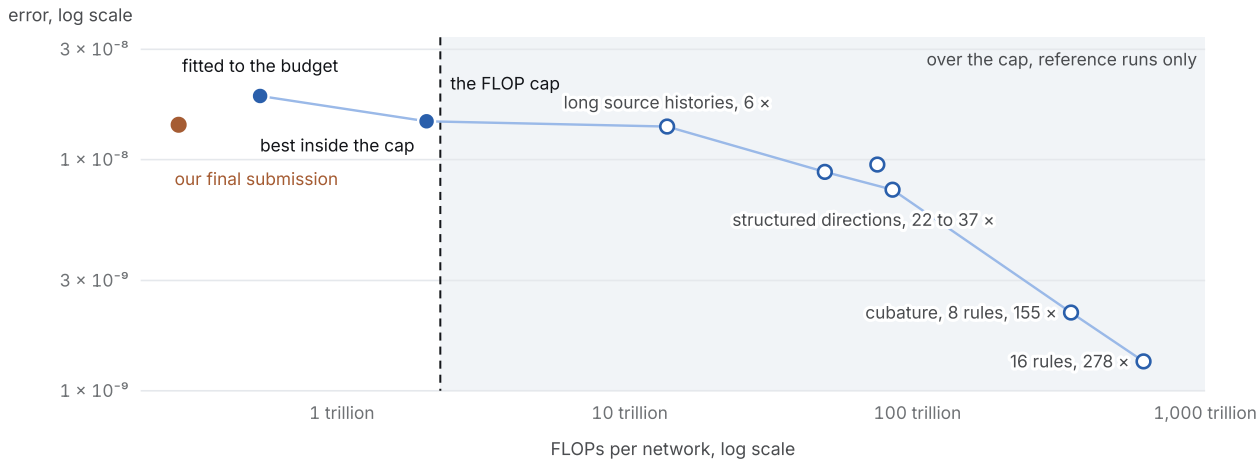
Many ideas that made an intermediate quantity more accurate made the final answer worse. A later learned formula for κ_4, meant to improve on the fitted model of page 35, tracked the true kurtosis 5 to 46% better inside the pass and still raised the final error by 2.3%, and the score by 4.6% once its extra FLOPs are counted (the picture shows changes in score). Feeding the exact D21 slices into one candidate, without refitting its learned corrections, made it 17% worse. The corrections downstream had learned the estimator's old errors, so a partial fix broke their compensation. A more accurate but different intermediate quantity can lose unless what comes after it is refitted.

Cheap reconstructions of the (3,1) and (2,2) slices of κ_4 (the (2,2) slices, $\kappa_4(i,i,j,j)$, say how the spreads of two neurons grow and shrink together) never recovered the 26% that both exact slices together were worth on one network, because the missing part is structure particular to each pair of neurons, which no simple formula carries.

The picture shows a selection of these closed lanes, ideas we tested and dropped, each with its measured change in score; open circles improved the score on fresh networks but broke a time limit.

What we would build next. A dense pair engine, which carries the covariance, the D21 slices and the (3,1) and (2,2) slices of κ_4 through every layer with exact products, about fourteen n^3 products per layer by a public account of the leading methods (Nurlanggs, forum 18252). With fast multiplication that is roughly 240 billion FLOPs per network, near the floor. We did not have time to test it.

A secondary avenue with accuracy first



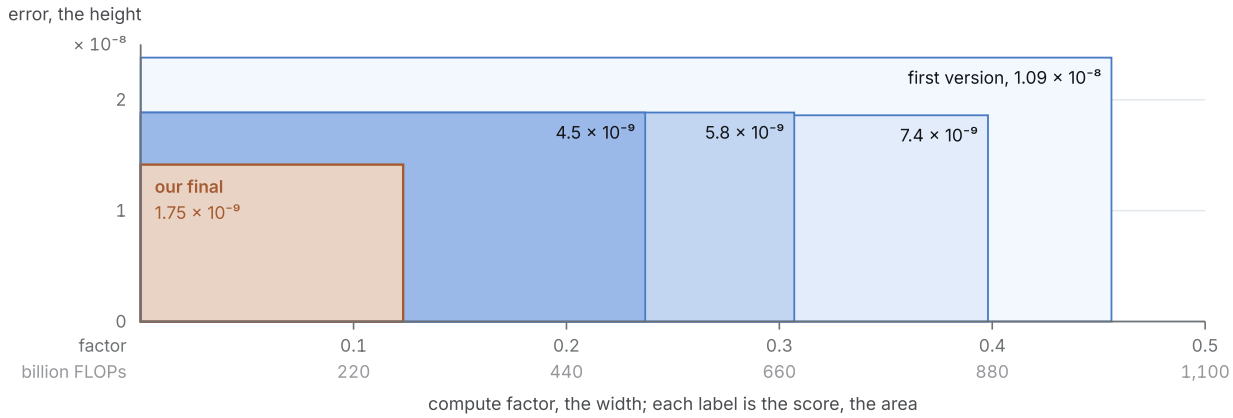
Alongside the main line, a second line of our own ran on another machine. It did not use V29. It grew from the organisers' covariance baseline, the Gaussian closure of page 15, and the propagation method of the paper ARC published with the challenge (arXiv:2605.05179), and its rule was the reverse of ours, accuracy first and compute after.

It carried each layer's means and full covariance, with the effect of the ReLU written as a Hermite expansion, a standard series for functions of bell-shaped variables. Its third cumulant was held as cubic sources in factored form, much like V29's triads but found independently, and each source kept its own legs for twelve to sixteen layers, or for its whole history, instead of being compressed into a shared basis. A solve weighted by the covariance restored the part that truncation dropped, an idea close to our later companion (page 37). Its fourth order was summarised by a single trace, the sum of a diagonal, and a linear readout fitted on 16 public networks adjusted its final means.

Its reference runs went far past the budget, to the right of the dashed cap in the picture. There the means were also estimated by structured cubature, fixed sets of over a million directions chosen so that averages come out exact for low-order terms, and merged with the analytic means layer by layer. With 16 such rules, 278 times the allowed compute, the error on one test network fell to 1.34×10^{-9}, against 1.4×10^{-8} for our final submission on average; with 8 rules, 155 times the compute, it was 2.2×10^{-9}. These runs could never be entries, but they show how much accuracy compute can buy, and at what price.

Inside the cap its best error was 1.47×10^{-8}, at 89.5% of the allowed FLOPs, measured, like every result of the second line on this page, locally on public networks.

Fitting our own line to the budget



Fitting the second line to the budget meant keeping its error while narrowing its compute, which in the picture means narrowing its rectangle, the score drawn as an area as on page 6. Its first version used 1.00 trillion FLOPs, 46% of the cap, at an error of 2.4×10^{-8}, which scored 1.09×10^{-8}.

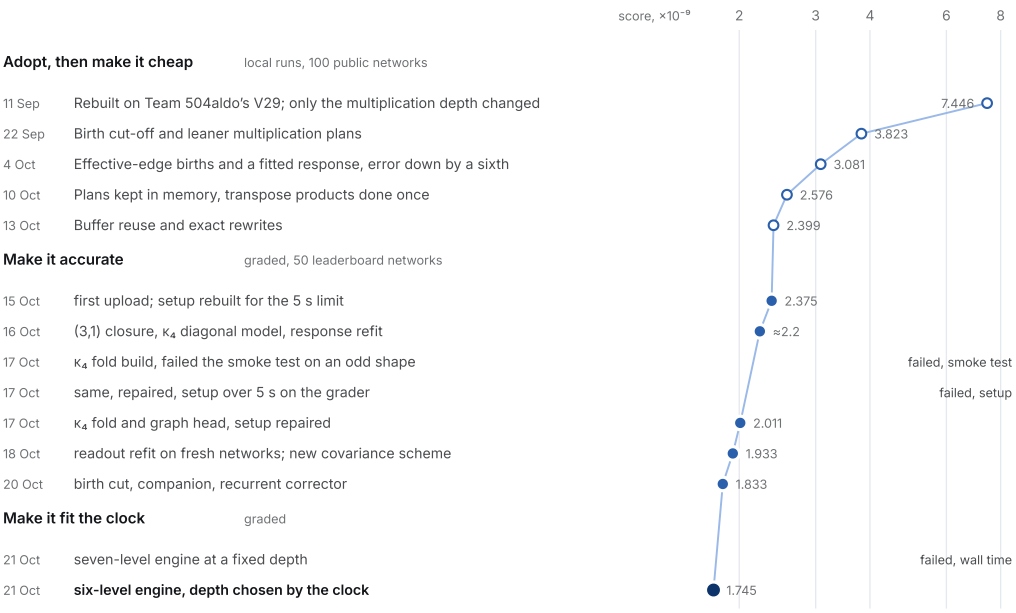
A few steps narrowed it. Terms for neurons that almost never fire, or almost always do, were dropped from its sources, much like our cuts (page 31); products were pooled and run through fast multiplication; and the last layer took shortcuts, since only its means count. In the end the error stood at 1.9×10^{-8} with 520 billion FLOPs (24% of the cap), for a score of 4.46×10^{-9}, and a sibling version scored 4.39×10^{-9}. Both met every clock on the networks tried, though one later test broke the Python limit on a single network, 447 ms against 400.

Our main line then stood at 2.58×10^{-9}, so we kept it and did not take the second line further, and its machine went on to run the main line's jobs. One idea crossed over and stayed. Its effective-edge birth, which folds the feedback of the lopsidedness already present into each new source, joined the main line on 4 October (page 32) and is part of the final submission.

What it changes is the leg A of each new source (page 21). Before, entry i of the column of a bent neuron c was C_{ic} times the slope of i, the covariance alone, and the lopsidedness that the input of c already carried reached the new source through a separate thin source of a few columns. Now the same entry also has terms in the D21 slices of the pair, $\kappa_3(i, i, c)$ and $\kappa_3(c, c, i)$, and in C_{ic}^2, weighted by the slopes, the kink heights and how far the bell of c sits from its kink. One leg carries both, and the thin source is gone.

Every result of the second line on these two pages was measured locally on public networks, from a slightly different release of them, and none was submitted.

How the work went



Most of our time on Phase 2 went into its final week. Before 15 October we came to it in short visits, a few days in late August and occasional sessions later, mostly to see where the competition stood. On our first day, 26 August, the organisers' covariance baseline, the Gaussian closure of page 15, scored 4.05×10^{-7}; two refinements took it to 1.3×10^{-7}. A bell has no room for the lopsidedness the ReLUs create, so we turned to the third cumulant; by early September our best complete method scored 3.5×10^{-8}. On 11 September we rebuilt on Team 504aldo's V29, changing only the depth of its fast multiplication. It scored 7.4×10^{-9}, a gain owed to their method, and light engineering cut its compute from 735 to 294 billion FLOPs by 13 October.

In parallel, a second line of our own, built without V29 and aimed at the lowest error first, reached about 4.4×10^{-9} on public networks (pages 52 and 53). We kept the main line, but one idea of the second line, the effective-edge birth, crossed over on 4 October and, with a fitted response, cut the error by a sixth.

The real investment of time came from 15 October to the deadline, and most of this report describes it. The work turned statistical and was measured on the grader. It added the (3,1) term and the κ_4 model, the quartic fold and the graph head, a readout refit on fresh networks, the birth cut, the companion and the recurrent corrector, and the error fell by a fifth, from 1.78 to 1.42×10^{-8}.

The last move was about time. On 21 October our deepest engine failed on the clock at a fixed depth, and a slightly shallower one, its depth chosen layer by layer by the controller, passed. Three of our nine uploads failed, each on a different check, the smoke test, the setup limit and the wall clock.

In the picture, on a log scale, open circles are our own runs on the 100 public networks and filled circles graded submissions. From our first V29 run to the graded mean the score fell 4.3-fold, 2.7 from FLOPs and 1.6 from the error.

On how we worked

THE FOUR CHECKS	TWO EXAMPLES OF CHANGES THROUGH THIS JOURNEY		WHAT IT STOPPED
	The companion kept in the final submission	**The seven-level engine** at a fixed depth; dropped	
1 **Check the algebra** exact tests at small width, before any network runs	417 tests passed of its new algebra	exactness checked same answers up to rounding	wrong derivations and bugs
2 **Measure on fresh networks** the same networks with and without it, sampled truths	score 1.00% better ± 0.36, on 23 networks	score 5.4% better on 96 networks	most of the forty closed lanes
3 **Qualify on the stock harness** the organisers' client and server, all limits timed	every limit met on 3 networks	every limit met 10 of 10 networks, 94–99 s	setup and Python-time failures
4 **Upload** authorised, uploaded and graded on their machines	graded on 20 October score 1.8327×10^{-9}	ran out of time the grader was slower	3 of our 9 uploads

I, Andrei Bulzan, was the only person on the team. I directed AI agents, which wrote nearly all of the code, ran the experiments and drafted this report. From 18 October every change passed the same four checks before it reached the grader (picture), and most ideas stopped at the second.

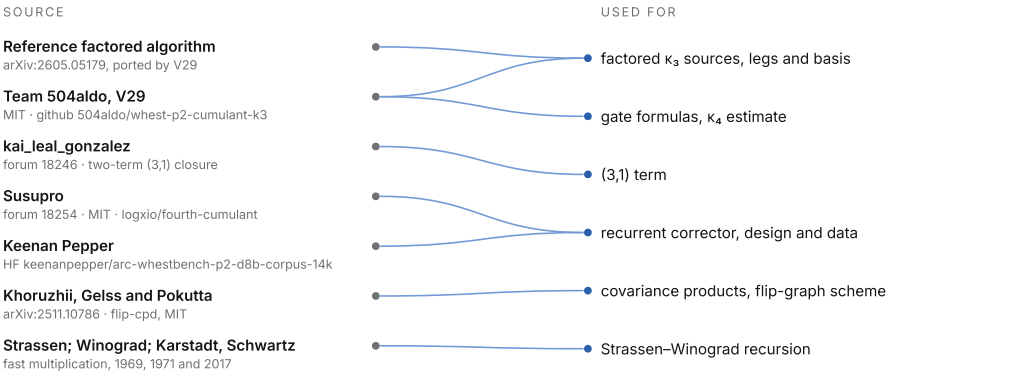
I set the direction, provided the hardware (a workstation, a machine with an RTX 5090 graphics card for the sampled truths, and Google Colab notebooks for the corrector's training data), chose what to pursue and authorised the uploads. Codex (OpenAI) set up the harness in late August and carried the engineering, from the multiplication engine, setup and Python-time work and exact rewrites to qualification on the stock harness and every upload. Claude (Anthropic) worked on the statistical additions, fresh-network measurement, training data for the corrector and the time-aware controller. Two agents in Antigravity (Google's agent coding tool), Argon and argon_2, built the corrector's trainer and module, integrated the companion, and built the deep-engine candidates and their layouts. A temporary extra Codex instance built the 17 October submissions.

Most of this happened in the final week. Until 15 October Phase 2 had only short visits, to try the harness and see where the competition stood; most of the additions on these pages, and all nine uploads, came after that.

Archives that had been released were left untouched, and further work went into copies of them. One panel of fresh networks was kept from the agent that built the candidates, so that its results could not steer the building (page 41). Each of the three failed uploads revealed something we had not yet checked (an unusual network shape in the smoke test, setup on the stock harness and the grader's slowest networks), and the last two led to the stock-harness check and to the rehearsals with a slowed clock (page 48).

Use of large language models. LLM involvement was extensive. The estimator's code, the fitted components, the experiments, the analysis and this document were produced by AI agents under my direction. The agents coordinated through shared message hubs, with well over a thousand messages between 16 and 22 October.

What we built on





The picture names every outside source in the package and what it contributed.

Team 504aldo released the V29 cumulant-propagation estimator under the MIT licence (github.com/504aldo/whest-p2-cumulant-k3; forum 18218). It is the core of our estimator, and its licence is kept in the package. V29 reimplements the factored algorithm of arXiv:2605.05179.

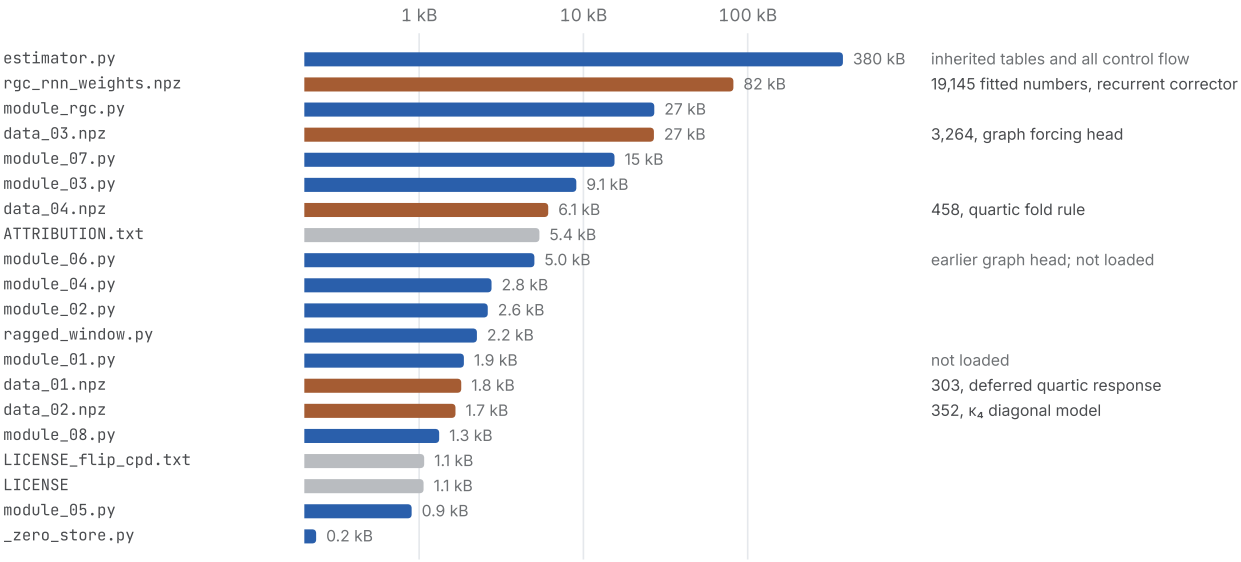
kai_leal_gonzalez published the two-term (3,1) closure (forum 18246); we recalibrated its weight. **Susupro** published the recurrent learned corrector (forum 18254; github.com/logxio/fourth-cumulant, MIT) whose design ours follows.

Keenan Pepper published on Hugging Face a corpus of 14,048 networks with sampled moments (picture), and most of the networks our corrector was trained on come from it. We are grateful to him. The corpus states no licence, and we have no permission from him beyond its public release. In Phase 1 he shared a similar dataset on the forum so that others would not have to repeat the computation (forum 18052), and other Phase 2 write-ups train on this corpus too. We hope he is content with this use, and we would welcome hearing from him if he is not. We sampled quite a few networks ourselves when we considered the training approach. But it was during the last few days of the challenge, and we realised there would not be sufficient time to produce data on his scale. The corrector was also better with more of his networks, removing 3.4% of the error with about 5,000 against 2.6% with 2,190 (page 40).

Khoruzhii, Gelss and Pokutta found the flip-graph multiplication scheme behind our covariance products (arXiv:2511.10786; flip-cpd, MIT). The fast-multiplication recursion is Strassen's (1969), in Winograd's variant and the alternative basis of Karstadt and Schwartz. **AIcrowd**, which hosts the challenge, and **ARC** (the Alignment Research Center) built the challenge, its harness (whestbench), the meter (flopscope) and the public networks.

Other teams' public write-ups shaped our choices even where we used no code.

Reproducing the result



The shipped archive holds 20 files, 574 kB in all (picture, by size), besides the manifest the organisers' packager adds. Every numerical operation runs through flopscope. The fitted numbers stored in data files are 23,522 values in five files, listed below; a handful of settings, such as the (3,1) weight, the cuts and the controller's constants, were chosen by measurement and are written in the code. The estimator reads no network identity, saved activation or reference value at prediction time. The organisers' harness reproduces the native-tier answers exactly; the mix of tiers, and so the bill, depends on the speed of the machine. The package carries the MIT licence of the code it extends and that of flip-cpd.

Submission	number 337559, accepted 21 October 2026 16:23 UTC, graded 17:15 UTC
Code	github.com/AndreiBulzan/arc-whitebox-submission-phase-2
Runtime	flopscope 0.12.3, whestbench 0.16.2; the organisers' client–server harness

Data file	Values	Component	Fitted on
`rgc_rnn_weights.npz`	19,145	recurrent corrector	about 5,000 networks with Monte Carlo truths, most from Keenan Pepper's collection
`data_03.npz`	3,264	graph forcing head	fresh networks (160 in the readout refit)
`data_04.npz`	458	quartic fold rule	public networks 0 to 3; rule chosen on eight more
`data_02.npz`	352	κ_4 diagonal model	public networks
`data_01.npz`	303	deferred quartic response	public networks 0 to 31; 15 per-layer scales on fresh networks

In Brief

Our estimator is the factored cumulant propagation of ARC's paper, in the form Team 504aldo released, with the additions below. Sampling would need about 81 times the allowed compute to reach the same error as our final submission (a score of 1.7453×10^{-9}, from an error of 1.4×10^{-8} at 271 billion FLOPs per network).

1. **The error.** Given exact cumulants, the Edgeworth form of the ReLU mean has an error of only 7×10^{-11}, so nearly all of ours comes from estimating the cumulants. Most of it, 69%, is made in layers 8 to 14, where many neurons are still in the bend. pages 12, 34

2. **Birth legs.** When a neuron bends, the lopsidedness its input already carries feeds the new source. Writing it into the new leg, where the original estimator kept it as a separate thin source, saved FLOPs and cut the error by 11%. pages 32, 53

3. **Cheap fourth-order terms.** A (3,1) term in the covariance update, after a formula kai_leal_gonzalez posted, and a fitted correction to each κ_4, with the response refitted to match, improved the score by 3.4%. Fourth-order births fit into an existing matrix product and improved it by a further 3.0% for 0.4% more FLOPs. pages 35, 36

4. **Compression recovery.** Compressing old sources onto a shared basis loses part of their third cumulant. One list of numbers per network, combined with the covariance, restores part of it and improved the score by 1.0%. pages 37, 38

5. **Learned corrections.** Three fitted corrections read a record of the pass and adjust the means it reports, removing 9% of the error; the biggest of them is a recurrent network after Susupro's design. Because they are fitted to the errors of the pass, a change that makes the pass more accurate can raise the final error unless they are refitted. pages 39, 51

6. **FLOPs and seconds.** Deeper fast multiplication bills fewer FLOPs but spends more time on additions, and the grader ran 1.44 to 2.10 times slower than our machine, unevenly from network to network. Choosing the depth of each layer from the time already spent kept every network inside the limit, the slowest at 111 of the 120 seconds. Every depth computes the same answer up to rounding, so the clock decides only how many FLOPs are billed. pages 44–50

7. **Noise-level changes.** Changes of one percent are easily lost in noise; the thread count of the linear-algebra library alone moved one network's error by 1.4%. We compared every change with and without it on the same networks, and a change below one percent needs 40 or more of them to be seen reliably. Several early gains shrank or reversed. page 41

8. **Next steps.** Every setting we could vary already sat at its knee. Exact pair quantities, the covariances and the D21, (3,1) and (2,2) slices, are worth 26 to 50% of the error in oracle runs, and carrying more of them is where we would start. pages 33, 42

Some ending words

I wonder if not long from now we'll start asking ourselves when reading studies – who's that 'we' in 'we can see that…', 'we find that…'?

Is it the author and colleagues? Is it the author and the AI? The other way around, the AI (or a horde of them) and the author? The author and the reader?

And perhaps not long after, we'll take it for granted that somewhere in that formula, there will always be, in some shape or form and without a need to specify it, an AI contribution. It might fade into the background as a constant substrate, similar to how the use of the PC and day-to-day software is seen as implicit when presenting one's work.

And the variant of 'we' denoting autonomous AI agents alone may end up being just as likely in the times to come, but I digress.

Because I used AI intensely here, any understanding of the mechanisms at play in the submitted solution came post hoc. That is to say, one of the largest struggles was shedding some faint light on the ideas as agents uncovered them and put them to the test, and understanding them at least insofar as knowing where there was still room for the error to yield cheaply. And I'm not convinced this gave any better results than simply giving all agents a '/goal' of 'Find the best solution you can, ideally 1e-10 or lower adjusted' and letting them work (though, make no mistake, there have been plenty of prompts that sounded just like that along the way).

As with Phase 1, setting aside the extra (especially cognitive) load this competition has put on me during its last week, I have had an absolute blast. And though I mention again and again AI and its immense involvement in all this, I cannot but hope that the advances and discoveries made in all fields moving forward can be carried out with a similar degree of joy and wonder involved in the process.

Thank you to everyone involved in the challenge, and in particular to the organisers for setting it up to be so reliable and high-quality an experience.

I still share many of the thoughts from Phase 1, which I will not repeat here, but will just say that I'm curious what next year's models might arrive at if they took a stab at the same challenge.

For the sake of temporal anchoring, as I am writing this, Gemini 4 Carbon has just been (not so publicly) released today by Google, with Fable 5.1 and Opus 5.5 being the latest publicly available models from Anthropic, and Astra 6 and Sol 6.1 the latest from OpenAI.

The effect of each change

CHANGE	ERROR	FLOPS	SCORE	MEASURED ON	PAGE
PHYSICS ADDED TO THE PASS					
Effective-edge birth a bending neuron's existing shape, folded into its new leg	−10.8%	–	–	8 matched networks	32
The (3,1) term $0.75\,\kappa_3(i,i,i)\kappa_3(i,i,j)/\sigma_i^2$ in the covariance update	–	–	−1.3%	58 held-out public, 50 better; response refitted	35
... and the κ_4 model a fitted correction to every neuron's κ_4	–	–	−3.4%	58 held-out public, 53 better; −3.9% on fresh; response refitted	35
Quartic fold young sources' fourth-order births, inside the D21 product	–	+0.4%	−3.0%	100 public networks	36
Companion one list standing in for what compression drops	−1.33%	–	−1.00% ± 0.36	23 fresh networks	38
FITTED CORRECTIONS AFTER THE PASS					
All three together deferred quartic response, graph head, recurrent corrector	−9.3%	+1.3%	–	64 fresh networks, final layer; FLOPs from the bill	28, 39
Recurrent corrector a small network run alongside the layers	−3.4%	–	–	trained on 5,157 networks, final layer	40
Refit on fresh networks the corrections refitted on the true means of fresh networks	–	–	−3.2%	18 fresh networks, held out	41
CHEAPER ARITHMETIC AND TIMING					
Fast multiplication, four levels Strassen–Winograd recursion four levels deep instead of one	–	−35.8%	–	100 public networks	32
Shape-aware plans, first birth cut a plan per product shape; $\alpha < -2.5$ left out of births	–	−17.5%	–	100 public networks	32
Memory and exact rewrites plans kept in memory, transpose products done once	–	−21.8%	–	100 public networks	32
Birth cut at −2.0 births stop at $\alpha = -2.0$ instead of −2.5, where queries and transport already did	+0.8%	−1.9%	−1.1%	63 fresh networks	31
Depth chosen from the clock a deeper engine, its recursion depth set per layer from the elapsed time	−0.05%	−4.7%	−4.8%	graded, 50 networks, all better; two packages compared	50
From the V29 rebuild to the final submission score 7.45 → 1.745×10^{-9}, 2.7-fold from FLOPs and 1.6-fold from the error	−36.5%	−63.1%	−76.6%	100 public, then graded	32, 50

Each row gives the effect of one change as measured when we decided to keep it, on the panel named in the row. Error is the final-layer squared error after subtracting the reference answers' own noise, FLOPs are the billed operations per network, and the score is their product; clay marks an improvement. The panels differ and the changes overlap, so the rows cannot be added. The last row covers the whole project, from our first run of V29 on the public networks to the graded final submission.

Where the remaining error lies

MADE EXACT INSIDE THE PASS	ERROR REMOVED	SHARE OF THE FINAL ERROR, 0 TO 80%
Post-ReLU covariances with the κ_3 and κ_4 diagonals public networks 0 and 1; fitted corrections left as shipped	74%	
Covariances and the D21 slices public network 0	50%	
Post-ReLU covariances public networks 0 and 1	43%	
The (3,1) and (2,2) slices of κ_4 one network	26%	
The (3,1) slices of κ_4 two public networks	12–21%	
The κ_4 diagonal 40 fresh networks, inside the loop	15%	
The κ_3 diagonal, layers 1 to 14 public networks 0 and 1, 8.4% and 20.4%	14%	
Every old source kept in full public network 0; 30% more FLOPs	2.8%	
Double precision throughout 64 fresh networks; twice the FLOPs	0.12%	

WHERE THE FINAL ERROR IS MADE, 160 FRESH NETWORKS	SHARE
Layers 1 to 7	10%
Layers 8 to 14	69%
The last gate	21%

EXACT CUMULANTS GIVEN TO THE RELU FORMULA	ERROR	ERROR, ON A LOG SCALE
Mean and variance	1.9×10^{-7}	
... and the skewness, κ_3	2.1×10^{-8}	
... and the kurtosis, κ_4	7×10^{-11}	
Our estimator, for scale the graded final-layer error	1.4×10^{-8}	

10^{-11} 10^{-10} 10^{-9} 10^{-8} 10^{-7} 10^{-6}

The first table lists oracle runs, each replacing one quantity by its true value inside the pass and recording the share of the final error that disappears. The shares overlap, and most were measured on one or two networks, so they indicate orders of size. The second carries each layer's new error forward with the slopes and weights, a linear approximation. The third gives the closed-form ReLU mean the exact cumulants of every final-layer neuron; with four of them its error falls two hundred times below ours, which places our remaining error in the estimated cumulants.

The settings and their trade-offs

SETTING	SHIPPED	TRIED	FLOPS	ERROR	SCORE	SCORE LOST, 0 TO 30%
Layers a source stays young	5	4	**−4.6%**	+12.9%	+7.7%	
		6	+7.2%	**−2.6%**	+4.4%	
Shared basis, directions	320	256	**−4.9%**	+10.4%	+5.0%	
		384	+5.1%	**−3.1%**	+1.9%	
Extra piece, the rank of each birth's (2,1) remainder the separable part is exact; 64 made no measurable difference on 16 fresh networks	6	8	+0.6%	0.0%	+0.6%	
Almost-always-on cut, α	2.3	2.0	**−1.7%**	+9.5%	+7.6%	
Birth cut, α 63 fresh networks; −2.0 was the change kept	−2.0	−2.5	+1.9%	**−0.8%**	+1.1%	
Every capacity setting enlarged 47 fresh networks	as above	all up	+10.6%	**−2.3%**	+8.1%	
Every old source kept in full public network 0	basis of 320	no basis	+30.0%	**−2.8%**	+26.4%	
Double precision 64 fresh networks	float32	float64	+96.2%	**−0.12%**	+96%	

OTHER SHIPPED SETTINGS	VALUE	NOTE
Cut for the sources, α	−2.5	below it the sources stop carrying a neuron, and its κ_3 diagonal is estimated by a small regression
(3,1) term weight	0.75	calibrated on public networks
Fast multiplication depth	6 levels, 7 for squares	chosen per layer by the clock; a full seven-level build failed on wall time
Controller plan, safety and fallback	114 / 118 / 115.5 s	margins 1.03 and 1.12 on the measured slowdown
Assumed slowdown before measuring	1.7, worth 6 s	overtaken by the measured time as layers complete

Each setting in the upper table was moved one at a time from its shipped value, on the panel used when it was decided and with the reference noise subtracted; the score change combines the two measured changes. Every move made the score worse. Shrinking a setting saved FLOPs but added more error, and enlarging one removed error but added more FLOPs, which is what we mean when we say the settings sit at their knee. The lower table lists settings that were chosen once from measurements and then left fixed.

Uploads and time limits

DATE	WHAT CHANGED	ERROR, $\times 10^{-8}$	FACTOR	SCORE, $\times 10^{-9}$	SCORE, 1.6 TO 2.45
15 Oct	first upload, the engineering build with setup rebuilt	1.778	0.1336	**2.3750**	
16 Oct	(3,1) term, κ_4 model, response refit rounded record	≈1.67	0.1336	**≈2.2**	
17 Oct	quartic fold and graph head failed the smoke test	–	–	–	
17 Oct	the same, with a shape guard failed the setup limit	–	–	–	
17 Oct	the same, setup repaired	1.519	0.1324	**2.0111**	
18 Oct	readout refit on fresh networks, new covariance scheme, cuts	1.477	0.1309	**1.9332**	
20 Oct	birth cut −2.0, companion, recurrent corrector	1.416	0.1294	**1.8327**	
21 Oct	seven-level engine at a fixed depth failed on wall time	–	–	–	
21 Oct	six-level engine, depth chosen by the clock the final submission	1.415	0.1233	**1.7453**	

LIMIT	CAP	GRADER	OUR MACHINE	WORST	SHARE OF THE CAP USED
Setup before the first network	5 s	3.69–3.78 s	4.54–4.69 s	76%	
Python time outside the metered calls	0.4 s	228–279 ms	slowest 343 ms	70%	
Wall time per network	120 s	97.8–111.1 s	65.8–70.7 s	93%	

Scores are means over the 50 leaderboard networks of each graded run of 100; for the upload of 16 October only the displayed values were recorded. Three uploads failed, each on a different check, and the last two failures became checks that later candidates passed before upload. The lower table compares the final submission with the three time limits, using the grader's slowest network for each. For the upload of 20 October the grader took 1.44 to 2.10 times our time per network, while setup and Python time ran faster there than on our machine.

Words used in this report

α	μ/σ, where a neuron's bell sits relative to the kink, in units of its own spread
β	the height of a neuron's bell at the kink, p(0), which sets how much new shape its ReLU makes
Kink	the point z = 0, where the ReLU's slope jumps from 0 to 1
Bend; in the bend	a ReLU folding a neuron's bell at the kink; a neuron whose bell straddles the kink, roughly $-2 < \alpha < 2.3$, is in the bend
Fires; firing share	gives a non-zero output; P(z > 0), how often, which is also the average slope
Lopsided; skewness	one side of a curve stretches further than the other; measured by κ_3/σ^3
Heavy-tailed; excess kurtosis	more weight far out than a bell of the same spread; measured by κ_4/σ^4
Lean	the direction and strength of a curve's lopsidedness
Cumulants	the ladder of shape numbers, κ_1 mean, κ_2 variance, κ_3 lopsidedness and κ_4 tail weight
Covariance matrix C	how every pair of neurons moves together; its diagonal holds the variances
Closure	a rule that fills in the numbers a method does not carry; the Gaussian closure sets κ_3 and κ_4 to zero
D3, D21	the views of the third cumulant a gate needs, $\kappa_3(i,i,i)$, a neuron's own lopsidedness, and $\kappa_3(i,i,j)$, how the spread of i follows j
(3,1) slices	$\kappa_4(i,i,i,j)$, how the lean of neuron i follows neuron j
Outer product; triad	a table or cube of products of one number from each list; a triad has three lists
Source	all the triads born at one layer's gate, the lopsidedness that layer adds to the network
Legs P and A	a source's two tables, with 1,024 rows and one column per neuron kept at its birth, all but those almost always off
Birth	the moment a layer's ReLUs create a new source
Young, old	a source in its first five layers, carried as full legs; after that, kept as factors in the basis
Query	a gate's reading of D3 and D21 from every source alive
Gate	the estimator's ReLU step, taking μ, σ^2, κ_3 and κ_4 in and giving output means, covariances and κ_4
Basis Q	the 320 shared directions onto which old sources are projected
Companion u	one list of 1,024 numbers that stands in for what the basis drops, read through C
Physics	our word for the mathematical model the estimator carries, as opposed to learned corrections
Cut	a line in α beyond which a neuron is left out of some of the work
Head; readout	a small fitted add-on to the estimator; the fitted part that adjusts the reported means
Oracle run	a run with one quantity replaced by its true value, to price its error
Monte Carlo	estimating an average by running the network on many random inputs
Fresh networks	networks we generated, with true means sampled 2^{25} or 2^{27} times
Panel	a fixed set of test networks
Knee	the point beyond which pushing a setting further buys little
FLOP	one arithmetic operation on a floating-point number, as counted by the meter
Fast multiplication	Strassen's method, seven block products instead of eight, applied recursively
Tier	a recursion-depth policy for one layer's products; every tier computes the same model
Compute factor	$\max(0.1, \text{FLOPs}/2^{41})$, the multiplier applied to the error
Harness; stock harness	the program that runs an estimator as the grader does; the organisers' own version
Wall time	the real seconds that pass while a network runs
Worker	one of the grader's parallel processes, each running its share of the networks in turn